Tritre francais

Davide FISSORE

Centre Inria d'Université Côte d'Azur

Présentée en vue de l'obtention du grade de docteur en Computer Science **d'**Université Côte d'Azur

Dirigée par : Jean-Paul HÉVIN, Chocolatier, Maison Hévin
Co-dirigée par : Antoine SANTOS, Chef pâtissier, École Criollo
Co-encadrée par : Laurent DUCHÊNE, Chef pâtissier, Maison Duchêne
Soutenue le : TODO

Devant le jury, composé de :
Paul BOCUSE, Grand chef cuisinier, L'Auberge du Pont de Collonges
Patrick LENÔTRE, Chef de cuisine, Pavillon des Princes, Paris
Christophe MICHALAK, Chef pâtissier, l'Hôtel Plaza-Athénée, Paris
Cédric GROLET, Chef pâtissier, Le Meurice
Michel GUÉRARD, Cuisinier, Les Prés d'Eugénie

TRITRE FRANCAIS

English Title

Davide FISSORE

Jury :

Président du jury
Paul BOCUSE, Grand chef cuisinier, L'Auberge du Pont de Collonges

Rapporteurs
Patrick LENÔTRE, Chef de cuisine, Pavillon des Princes, Paris
Christophe MICHALAK, Chef pâtissier, l'Hôtel Plaza-Athénée, Paris

Examinateurs
Cédric GROLET, Chef pâtissier, Le Meurice

Directeur de thèse
Jean-Paul HÉVIN, Chocolatier, Maison Hévin

Co-directeur de thèse
Antoine SANTOS, Chef pâtissier, École Criollo

Co-encadrant de thèse
Laurent DUCHÊNE, Chef pâtissier, Maison Duchêne

Membres invités
Michel GUÉRARD, Cuisinier, Les Prés d'Eugénie

Université Côte d'Azur

Davide FISSORE

Tritre francais

xiii+108 p.

Ce document a été préparé avec LaTeX2e et la classe `these-ISSS` version v. 2.10.

Impression : main.tex – 9/4/2026 – 13:14

Révision pour la classe : these-ISSS.cls,v 2.10 2020/06/24 14:16:37 mpelleau

*À toi lecteur <3 :**

Tritre francais

Résumé

Les macarons c'est bon par nature, c'est le propre même du macaron que d'être bon. Au vue de l'affluence constante aux divers magasins Ladurée on peut supposer que les macarons qu'on y trouve sont meilleurs qu'ailleurs. Cette thèse vise à donner des idées et pistes sur le pourquoi du comment que les macarons de Ladurée sont si bons.

Mots-clés : Pâtisserie, Macaron, Ladurée.

English Title

Abstract

Macarons are so good.

Keywords: Pastry, Macaron, Ladurée.

Acknowledgement

Merci !

Contents

CHAPTER 1

Introduction

1.1 Prologue

Automation everywhere in CS, we want to set up boilerplates to help our development.
Examples:
- Continuous integration
- Macros are everywhere
- Artifitial intelligence

We delegate tasks to some tools.
expert system →
example plus frappant:
artificial intelligence: based on probability but not reproductible, not predicatable
several other examples in computer sciences:
...

1.2 Automation is everywhere

In the domain of formal proof automation is everywhere, and it is formally called elaboration [58]
In formal proof, proofs are verbose and have lot of details: types, coercions, canonical structures, type-classes.
In theorem provers the user benefit of notations, implicit inference and tactics to ease his job.
Simply typed lambda calculus is an exmaple of automation: an algo checks if ...

1.3 Contributions

In this manuscript we focus on an alternative type-class solver written in the elpi meta-language.
Discimination tree,
Elpi semantics,
Higher order unification,
Alternative type-class solver in *Rocq-Elpi*
Rocq-Elpi: Higher-order unification support
Static determinacy checking for *Elpi*
First-order static determinacy checker formalization

CHAPTER 2

Context: languages and tools

At the core of this manuscript, there are two important pieces of software coming from different areas of computer science. On the one hand, we consider the *Rocq* proof assistant (formerly known as *Coq*); on the other hand, we consider the logic programming language *Elpi*.

Rocq [5] is one of the main proof assistants. It is based on intuitionistic logic and allows formal reasoning about mathematical statements and program specifications. In this setting, logical propositions are interpreted as types, and proofs are interpreted as programs (or terms) of these types. The underlying type theory of *Rocq* ensures that the coherence between a program and its type is rigorously checked, thus providing strong guarantees of correctness.

Proof development in *Rocq* is carried out using a system of tactics, which allows the user to construct proof terms interactively.

On the other hand, *Elpi* [14, 55] is a language from the field of logic programming. In particular, it is a dialect of *λ-Prolog* which features backtracking, higher-order programming and unification, backtracking control though the `Cut` operator, unification control throuh predicates modes. This language provides a plugin, called *Rocq-Elpi*, which allows one to extend *Rocq* with new commands and tactics [32, 59, 24].

Commands extend the *Rocq* language by introducing new definitions or theorems that are automatically generated by *Elpi* programs. Examples of libraries that intensively use this feature include *derive* [56], which derives equality theorems for inductive types, and *Hierarchy Builder*, which helps define complex algebraic structures while automatically generating associated theory. *Hierarchy Builder* is supports the Mathematical Components ecosystem [36] and mechanizations such as the Odd Order and Four Color theorems [23, 22].

Tactics extend the proof system of *Rocq*. They rely on the *Elpi* interpreter, which uses a Prolog-like SLD resolution strategy [21, Chap. 9] to search for proofs in a predefined *Elpi* database. For example, the *Trocq* [6, 9] library provides an *Elpi*-based tactic for automatic reasoning via proof transfer.

In the following sections, we introduce the key concepts that form the foundation of chapters 3 to 6. In particular, in section 2.1, we briefly discuss a central component of proof assistants such as *Rocq*: the elaborator, which mediates the interaction between the user and the system. In section 2.2, we present the core aspects of the *Elpi* language, including its syntax and semantics, which are necessary to understand its execution model. Finally, in section 2.3, we describe the main ideas behind the *Rocq*-elpi plugin.

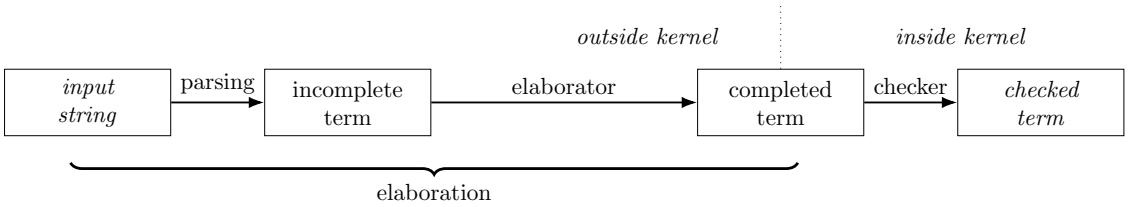


Figure 2.1: Elaboration scheme

2.1 Elaboration in the *Rocq* proof assistant

A standard introduction to *Rocq* usually starts with the foundations of the language, such as type theory, the λ-calculus, and the basic constructs used to define functions and state theorems. These topics are covered in detail books, such as [5], and papers [10]. Additional resources can be found at: `https://rocq-prover.org/papers`.

In this section, we focus on a more technical aspect that enables communication between the user and the *Rocq* system: the *elaborator*. This component is part of all proof assistants. Its purpose is to allow the user to write incomplete or ambiguous expressions, leaving the system to infer the missing information.

In the terminology of *Rocq*, missing information is called *holes*, usually denoted by underscores.

As a simple example, consider the term `length [1,0]`. This expression uses notations for natural numbers and lists, and it omits several implicit type annotations. The fully elaborated term is:

```
@length nat (@cons nat (S O) (@cons nat O (@nil nat)))
```

Thus, a short expression of 12 characters is expanded into a fully explicit term of 62 characters. This automation significantly reduces the effort required from the developer.

Figure 2.1, taken from [58], summarizes the main steps of the elaboration process. Starting from a raw input string, the system parses it into a *Rocq* term that may still be incomplete, and then elaborates it into a fully specified term. This final term is passed to the kernel, which checks its type correctness. If the check succeeds, the term is a valid inhabitant of its type.

The elaboration process includes several key tasks: notation resolution, type inference, coercion insertion, and inference of overloaded symbols.

Notation resolution removes syntactic ambiguities by replacing user-defined notations with their corresponding logical symbols in the abstract syntax tree (AST). The resulting term is unambiguous but may still contain holes, which are then filled during later stages.

Type inference is one of the main mechanisms used to fill these holes. The type-checker can infer missing types when symbols are not overloaded, that is, when they have a unique interpretation.

For example, assume that `cons` has type `∀ T, T → list T → list T`, `nil` has type `∀ T, list T`, and `O` has type `nat`. In this context, the type-checker can infer that the holes in the term `@cons _ O (@nil _)` are both `nat`.

Another important feature of the elaborator is the coercion mechanism [3]. Coercions allow one to implicitly convert a term of type t_1 into a term of type t_2 by inserting a function of type $t_1 \to t_2$.

For instance, suppose that `nat_to_bool` maps `0` to `false` and any other natural number to `true`, and is declared as a coercion from `nat` to `bool`. Then the term `negb 0`, where `negb` expects an argument of type `bool`, is automatically elaborated into `negb (nat_to_bool 0)`.

Beyond type inference and coercions, a key feature of elaboration is support for overloading. This allows the same symbol to be reused at different types. Since ad-hoc polymorphism plays an important role in this manuscript, we will examine this mechanism in more detail in the following subsection.

2.1.1 Ad-hoc polymorphism

Polymorphism is a fundamental feature of functional programming languages. In his lecture notes, Strachey [53] distinguishes between two main forms of polymorphism: parametric polymorphism and ad-hoc polymorphism. Parametric polymorphic functions are *uniform*, meaning that their behavior does not depend on the specific type at which they are instantiated. For example, the function *length*, with type `'a list -> int`, returns 7 when applied to any list of seven elements. In other words, the compiler reuses the same implementation of *length* for every call, regardless of the instantiation of `'a`.

Parametric polymorphism is widely used in functional programming. However, it is not suitable for modeling mathematical function overloading. In mathematics, the symbol + has a consistent, intuitive meaning, and no ambiguity arises in expressions such as $1 + 2$ or $\pi + e$. In the first expression, + denotes addition over integers; in the second, it denotes addition over real numbers. The same symbol is consistently used for operations that share common algebraic properties.

In contrast, parametric polymorphism does not allow to have multiple different type-dependent interpretations. This limitation appears in languages such as *OCaml*, where integer and floating-point additions are represented by distinct operators, + and +., respectively. Thus, one writes $1 + 2$ for integers and $\pi +. e$ [*] for floating-point numbers. In general, a different symbol must be introduced for each type implementing a given mathematical operation.

To address the need for a shared notation for operations (and properties in proof assistants) across types, Wadler and Blott introduced the notion of type classes in 1989 [61] and were soon implemented in *Haskell* [26] as a pervasive feature of the language. Then other languages, such as *Rocq* [52] also extended their type system to allow this kind of overloading.

Symbol overloading Type classes provide a principled form of ad-hoc polymorphism, enabling a single symbol to be associated with multiple type-specific implementations while preserving static type safety. They provide structure parameterized by one or more types t and declare the signatures of a set of functions, called methods, whose types may depend on t. An instance of a type class provides a concrete implementation of the type class for a specific choice of t, providing definitions for all of its methods.

Unlike parametric polymorphism, where a unique function is used for the execution of a call, type classes rely on instance resolution. Typeclasses and instances inhabit a database mapping classes to their instances. When compiling a call to a type class method, the elaborator finds the right implementation by selecting the instance of the type class associated with the given type in the given database.

[*] In *OCaml*, π is `Float.pi` and e is `Float.exp 1.0`

As an example, consider the type class `Add`, parameterized by a type `T` and equipped with a method `plus` to perform addition between two elements of type `T`. In *Rocq*, the definition of `Add` is given below.

```
Class Add T := {plus: T → T → T}.
```

The type of the method `plus` is `∀T, Add T → T → T → T`, where both the type parameter `T` and the instance argument `Add T` are implicit. The typechecking of the term `plus 3 4` gives:

```
@plus nat ?Add 3 4 : bool
where ?Add : [ |- Add nat]
```

The term has type `nat`, but the instance for `Add nat` cannot be inferred, leaving a metavariable `?Add`, this is beacuse in the type-class database, there is no instance associated to `Add`. To resolve this, we define the instance for natural numbers:

```
Instance addNat : Add nat := {plus := Nat.add}.
```

The declaration `addNat` has type `Add nat` and provides an implementation of the method `plus`. Registering this instance extends the instance database for `Add` with an entry for the type `nat`. Afterwards, the term `plus 3 4` can be elaborated successfully, with `?Add` instantiated to `addNat`.

Similarly, we may define an instance for booleans:

```
Instance addR : Add R := {plus := Rplus}.
```

The same symbol `plus` can then be applied to values of type `nat` or `R`. Therefore, extending the notation system, we are now allowed to write $1 + 2$ and $\pi + e$ knowing that the elaborator automatically selects the appropriate instance, `addNat` or `addR`.

A key feature of type classes is the ability to define *conditional* instances, whose availability depends on other instances. An example is the instance for the product type.

```
Instance addProd : ∀ T1 T2, Add T1 → Add T2 → Add (T1 * T2) :=
  {plus '(x1,y1) '(x2, y2) := (plus x1 x2, plus y1 y2)}.
```

The type of the instance `addProd` states that it implements `Add (T1 * T2)` provided that instances of `Add T1` and `Add T2` are available. In this way, addition on pairs is derived from addition on their components, that is, we can add $(\pi, 3)$ with $(e, 4)$, $(3, (\pi, 4))$ with $(7, (e, 0))$ ...

The type class `Add` can be defined in a similar style in *Haskell*, up to minor syntactic differences. In a proof assistant such as *Rocq*, however, type classes support not only function overloading but also *theorem overloading*.

For instance, one may enrich `Add` with the associativity specification:

```
Class Add T := { plus: T → T → T;
  assoc: ∀ a b c, plus a (plus b c) = plus (plus a b) c}.
```

An instance of this refined class must now provide both an implementation of the function `plus` and a proof that `plus` is associative. Then, if `addNat`, `addR` and `addProd` are implemented according the new definition of `Add`, the `assoc` symbol can be used as a proof that addition is associative over natural and real numbers and pairs.

In order to better structure type classes, the system can be extended so that type classes can inherit properties of existing classes. In group theory this feature consist, for example, in defining an object called `Magma` with an operation `op`, and a second opject `Semigroup` which extends `Magma`s by postulating the associative property on `op`.

This way for example, the `Add` class, in some developments is formalized as:

```
Class Magma T := {op : T → T → T}.
Class Semigroup T `{Magma T} :=
  {assoc a b c : op a (op b c) = op (op a b) c}.

Instance addNatM : Magma nat := {op := Nat.add}.
Instance addNatS : Semigroup nat := {assoc := Nat.add_assoc}.
```

Search strategy Type classes are used in several major *Rocq* libraries, such as *tlc*, *stdpp*, and *iris* [30]. They provide access to the type-class solver, which automatically infers instances by searching in a database of declared instances. This database is organized as an ordered list. Given a type-class goal, the solver tries to unify the goal with the conclusion of each instance. When unification succeeds, it then tries to solve the premises of that instance.

As an example, consider the instances defined previously for the type class `Add`. To solve the goal `Add (nat * R)`, the solver first tries `addNat`, which fails because its conclusion does not match the goal. The same happens with `addR`. The instance `addProd` can be applied if its premises, namely `Add nat` and `Add R`, can be solved. Since both are derivable, the resulting instance is `addProd nat R addNat addR`. By default, the solver follows a *depth-first search* (DFS): it explores the database deeply and backtracks when a branch fails.

The order of instances in the database is important, since it may affect both performance and termination. For instance, the goal `Add ?X`, where `?X` is an existential variable, can lead to non-termination if `addProd` has the highest priority. In such a case, the variable may be repeatedly instantiated with terms of the form $(((\cdots * \texttt{?X}_{n-2}) * \texttt{?X}_{n-1}) * \texttt{?X}_n)$, due to the recursive use of its first premise.

The priority of an instance can be specified at the end of its declaration using `| N`, where `N` is an integer. The instance is then placed at position `N` in the database. A smaller value of `N` means higher priority. If no priority is given, *Rocq* computes it automatically by assigning weight 1 to each premise that is itself a type-class constraint.

Several mechanisms are available to guide the solver. One common tool is *modes*, which enforce conditions on the shape of class arguments during search. There are three kinds of modes in *Rocq*. Informally, a mode is a list of symbols `+`, `!`, or `-`, indicating whether an argument is treated as an input, partially known (its head is fixed), or as an output.

Modes can be declared in two ways. One can write `#[mode="+"] Add T.`, which specifies that the first argument of `Add` must be instantiated in goals. Alternatively, one can declare it afterwards using `Hint Mode Add +`.

A type class may have several modes. During instance search, these modes are checked dynamically: the search proceeds if at least one mode is satisfied by the current goal.

A characteristic of type classes is that several instances may match the same goal. This requires backtracking in the solver. In some libraries, such as *stdpp*, this behavior is exploited to mimic logic programming. For example, the type class `SetUnfold` is used to transform set expressions into logical properties. For instance, an inclusion $X \subseteq Y$ can be turned into a statement of the form

$\forall x, P\ x \to Q\ x$, under suitable assumptions on X and Y. If no transformation applies, the default instance `set_unfold_default P : SetUnfold P P` is used, leaving the goal unchanged.

This default instance serves as a catch-all rule, as it applies to any goal. Although backtracking can be useful in such patterns, it may introduce ambiguities and lead to unexpected behavior that is difficult to control in practice.

In *Rocq*, backtracking can be disabled using the flag `Typeclasses Unique Instances`. This forces the system to commit to the first solution found during the recursive search. However, this mechanism is not class-specific: once enabled, non-backtracking is applied to all type classes involved in the search. In practice, a user may prefer a more precise behavior, where only certain type classes avoid backtracking, while others retain it. Therefore, at least in the libraries we have analized, whis flag is not used.

2.2 The *Elpi* language

Elpi [14] stands for *Embeddable Lambda-Prolog Interpreter*. It is a dialect of λ-*Prolog* [40, 41] that supports higher-order logic programming and integrates a goal suspension mechanism based on CHR (Constraint Handling Rules [19, 25]).

The term "embeddable" highlights that *Elpi* is designed to be used as an extension language. Its main application is the *Rocq-Elpi* plugin (see section 2.3), which enables communication between *Elpi* as the meta-language and the *Rocq* proof assistant as the object language.

A comprehensive presentation of the *Elpi* language and its applications can be found in Tassi's "habilitation à diriger les recherches" [57]. Here we briefly introduce the aspects of *Elpi* that are relevant to this document.

Sections sections 2.2.1 and 2.2.2 describe the syntax and operational semantics of *Elpi*. Our presentation slightly differs from [57], as it follows the conventions adopted later in chapters 5 and 6. In particular, the syntax and semantics presented here exclude the *CHR* component of *Elpi*; we discuss it informally in section 2.2.5.

In sections 2.2.3 and 2.2.4, we first present an example of execution, highlighting the role of the `Cut` operator in the *Elpi* language. We then discuss two main programming styles in *Elpi*, namely higher-order programming and dynamic programming, and explain how they interact with the *Elpi* semantics.

Finally, in sections 2.2.5 and 2.2.6, we outline two key features of the *Elpi* language: the constraint handling rules system, and rule naming and grafting.

2.2.1 Abstract syntax of *Elpi* (without *CHR*)

Syntax of *Elpi* programs Figure 2.2 presents the abstract syntax of *Elpi* programs.

The basic syntactic categories are data $\mathbb{K}$, predicates $\mathbb{P}$, unification variables $\mathbb{U}$, and nominal variables $\mathbb{N}$. Nominal variables represent binders introduced either by λ-abstractions or by `pi`-quantification.

A term $\mathbb{T}\mathbb{m}$ is either one of these basic constructs or an application. More precisely, terms are built by the binary application of a term to a λ-term $\mathbb{L}$. Lambda terms introduce abstractions; in the concrete *Elpi* syntax an abstraction is written `x\ t`. Function application follows a curried style: unlike *Prolog*, *Elpi* does not use parentheses between the functor and its arguments.

An atom $\mathbb{A}$ represents an executable unit. It can be:

Program	$\mathbb{K}$	$::= f, g, \ldots$	datum
	$\mathbb{P}$	$::= p, q, \ldots$	predicate
	$\mathbb{U}$	$::= X, Y, \ldots$	unification variable
	$\mathbb{N}$	$::= x, y, \ldots$	nominal variable
	$\mathbb{T}\mathtt{m}$	$::= \mathbb{K} \mid \mathbb{P} \mid \mathbb{U} \mid \mathbb{N} \mid \mathbb{T}\mathtt{m}\ \mathbb{L}$	term
	$\mathbb{L}$	$::= \lambda\mathbb{N}.\mathbb{L} \mid \mathbb{T}\mathtt{m}$	λ–term
	$\mathbb{A}$	$::= \mathtt{Cut} \mid \mathbb{T}\mathtt{m} \mid \mathtt{pi}\ \mathbb{N}.\ \mathbb{A} \mid \mathbb{R} \Rightarrow \mathbb{A}$	atom
	$\mathbb{R}$	$::= \mathbb{T}\mathtt{m}\ \mathtt{:-}\ \mathbb{A}^*$	rule
Types	$\mathbb{K}\mathtt{d}$	$::= \mathtt{kind}\ \mathbb{K}\ \mathbb{T}\mathtt{a}$	type declaration
	$\mathbb{T}\mathtt{a}$	$::= \mathtt{type} \mid \mathbb{T}\mathtt{a} \to \mathbb{T}\mathtt{a}$	type arity
	$\mathbb{K}\mathtt{s}$	$::= \mathtt{type}\ \mathbb{K}\ \mathbb{T}\mathtt{y} \mid \mathtt{type}\ \mathbb{P}\ \mathbb{T}\mathtt{y}$	constructor declaration
	$\mathbb{T}_\mathtt{b}$	$::= \mathbb{K} \mid \mathbb{N} \mid \mathbb{T}_\mathtt{b}\ \mathbb{T}\mathtt{y}$	type
	$\mathbb{T}\mathtt{y}$	$::= \mathbb{T}\mathtt{a} \xrightarrow{?\mathtt{m}} \mathbb{T}\mathtt{a} \mid \mathbb{T}_\mathtt{b}$	λ–type
	m	$::= \mathtt{input} \mid \mathtt{output}$	mode

Figure 2.2: Abstract syntax of *Elpi* programs

1. the cut operator (`!` in the concrete syntax),
2. a predicate call,
3. a `pi`-quantification, written `pi x\ t` in the concrete syntax, which introduces a fresh nominal variable `x` visible in `t`, or
4. the dynamic insertion of a rule $\mathbb{R}$, written `r => t`, which makes the rule `r` locally available while solving `t`.

A rule $\mathbb{R}$ consists of a head and a (possibly empty) body, which is a list of atoms. The atoms in the body are called the premises of the rule.

For simplicity, we omit constraints from the syntax, since they do not play an immediate role in the presentation. They are briefly described in section 2.2.5.

The syntax we propose allows writing hereditary Harrop formulas. In this abstract syntax, logical connectives such as negation and disjunction can be treated as first-class objects and implemented directly as *Elpi* rules.

Definition 2.2.1 (Atom polarity)**.** Polarity indicates whether a subformula appears in a *positive* or *negative* position within a formula.

The intuition behind polarity is simple: a formula or atom at the top level is in positive position. When crossing an implication ($\Rightarrow$), the left-hand side flips its polarity, while the right-hand side keeps the polarity of the original formula.
In the example on the right we explicitly annotate the polarity of each subformula.

$$\underbrace{\underbrace{(\underbrace{x}_{pos} \Rightarrow \underbrace{y}_{neg})}_{neg} \Rightarrow \underbrace{z}_{pos}}_{pos}$$

This notion of polarity will be used again in section 3.4 when discussing the encoding of instances.

Syntax of *Elpi* types In *Elpi*, users can define the algebraic data types required by their development as well as the signatures of predicates. A new algebraic data type is declared with the `kind` keyword, while constructors of that type are introduced using the `type` keyword. A type is ether a definition is either a type declaration, a variable, or the application of a type to another in the case of parametric types. The arrow symbol is used to type function space and in *Elpi* it take an optional mode information. The mode, `input` or `output`, tells the runtime which unification algorithm to use when unifying the arguments of two terms, we dealy this discussion in the next paragraph. By default, no mode information is compiled to `output`.

For example, the following declarations define Church naturals and polymorphic lists.

```
kind nat type.                    kind list type → type.
type z nat.                       type nil list A.
type s nat → nat.                 type cons A → list A → list A.
```

The type `nat` contains the constant `z` representing zero and the constructor `s` representing the successor of another natural number. A `list` is either empty `nil` or the constructor `cons` that combines a head element with a tail list. Lists are polymorphic: the type of the head element must coincide with the type of the elements stored in the tail.

A predicate is a symbol whose type ends in `prop`. The type `o` is an equivalent notation following the convention of [41]. *Elpi* provides a dedicated syntax for declaring predicates, which also allows the programmer to specify the modes of their arguments.

The declaration `pred p i:t1, o:t2.` where `t1` and `t2` are types, is equivalent to the declaration `type p t1 -> t2 -> prop` but it also specifies that the first argument of `p` is an input (`i:`) and the second is an output (`o:`). These modes are used at runtime to determine which unification strategy should be applied when matching the head of a rule with the current goal.

Program type checking *Elpi* is an (almost) strongly typed language. When a program is interpreted, the type checker verifies that it is well typed. Any type inconsistency is reported as an error, which typically helps identify and correct implementation bugs early.

2.2.2 Operational semantics of *Elpi* (without *CHR*)

A substitution σ of type Σ is a partial mapping from unification variables $\mathbb{U}$ to terms $\mathbb{T}\mathbb{m}$. The application of a substitution to a term t is written $\sigma\ t$. The empty substitution is denoted by ε. The domain $\mathrm{dom}\ \sigma$ of a substitution σ is the set of variables that are assigned in σ.

Definition 2.2.2 (Substitution extension)**.** A substitution σ' is an extension of σ iff

$$\exists \sigma''.\ \sigma + \sigma'' = \sigma' \ \wedge\ \mathrm{dom}\ \sigma'' \cap \mathrm{dom}\ \sigma = \varnothing.$$

A typing environment $\mathbb{E}$ maps predicates $\mathbb{P}$ to their signatures.

A program $\mathcal{P}$ of type $\mathbb{P}$ is represented as a record with the following structure:

$$\mathbb{P}\ :=\ \{\texttt{index: list } \mathbb{R}\texttt{; nvar: set } \mathbb{N}\texttt{; sig: } \mathbb{E}\ \}$$

The field `index` is an ordered list of rules representing the rule database. The field `nvar` contains the set of nominal variables that have already been introduced, and `sig` is the signature environment.

We write $x\#\mathcal{P}$ to denote that the nominal variable x is fresh with respect to $\mathcal{P}.\texttt{nvar}$. The notation $x +_\forall \mathcal{P}$ denotes the program obtained by adding x to $\mathcal{P}.\texttt{nvar}$. Similarly, $h +_\Rightarrow \mathcal{P}$ denotes the program obtained by prepending the rule h to $\mathcal{P}.\texttt{index}$.

$$\frac{}{\texttt{runE}\ ((\sigma,\ \varnothing) :: a) \rightsquigarrow (\sigma, a)}\ \texttt{runE}_\top$$

$$\frac{}{\texttt{runE}\ \varnothing \rightsquigarrow \square}\ \texttt{runE}_\bot$$

$$\frac{\texttt{runE}\ ((\sigma,\ g) :: a) \rightsquigarrow r}{\texttt{runE}\ ((\sigma,\ (\_,\ \texttt{Cut}, a) :: g) :: \_) \rightsquigarrow r}\ \texttt{runE}_!$$

$$\frac{\mathcal{B}_E(\mathcal{P}, (\sigma\ t), \sigma, a, g) = a' \qquad \texttt{runE}\ (a' +\!\!+\ a) \rightsquigarrow r}{\texttt{runE}\ ((\sigma,\ (\mathcal{P},\ t, \_) :: g) :: a) \rightsquigarrow r}\ \texttt{runE}_{\mathcal{B}}$$

$$\frac{y \# \mathcal{P} \quad \texttt{runE}\ ((\sigma,\ (y +_\forall \mathcal{P},\ t[x/y], a) :: g) :: a') \rightsquigarrow r}{\texttt{runE}\ ((\sigma,\ (\quad \mathcal{P} \quad,\ \texttt{pi}\ x.\ t, a) :: g) :: a') \rightsquigarrow r}\ \texttt{runE}_\forall$$

$$\frac{\texttt{runE}\ ((\sigma,\ (h +_\Rightarrow \mathcal{P},\quad \texttt{t} \quad, a) :: g) :: a') \rightsquigarrow r}{\texttt{runE}\ ((\sigma,\ (\quad \mathcal{P} \quad,\ h \Rightarrow t, a) :: g) :: a') \rightsquigarrow r}\ \texttt{runE}_\Rightarrow$$

Figure 2.3: Operational semantics of *Elpi*

A goal g of type $\mathcal{G} = \mathbb{P} \times \mathbb{A} \times \overrightarrow{\mathcal{A}}$ is a triple consisting of a program, an atom, and a list of *cut-to alternatives*. An alternative a of type $\mathcal{A}$ is a pair $\Sigma \times \overrightarrow{\mathcal{G}}$ composed of a substitution and a list of goals.

Figure 2.3 presents the derivation rules of the *Elpi* interpreter, defined in terms of the function

$$\texttt{runE} : \overrightarrow{\mathcal{A}} \to (\Sigma, \overrightarrow{\mathcal{A}}) + \square.$$

This semantics is inspired by [11, Sec. 4.3] and [57]. Execution is modeled as the evaluation of a stack of alternatives $\mathcal{A}$. The interpreter processes alternatives sequentially. The evaluation of the alternative list either succeeds, producing a substitution and a new list of alternatives, or fails, returning $\square$.

In figure 2.3, rule $\texttt{runE}_\top$ captures success: if the first alternative contains an empty list of goals, the interpreter returns its associated substitution together with the remaining alternatives. Conversely, rule $\texttt{runE}_\bot$ states that evaluation fails when the list of alternatives is empty.

The remaining rules of runE, namely $\texttt{runE}_!$, $\texttt{runE}_{\mathcal{B}}$, $\texttt{runE}_\forall$, and $\texttt{runE}_\Rightarrow$, apply when the first alternative contains a non-empty list of goals. Such alternative list has the form $((q :: g) :: a)$, where q is the current goal atom (or query). The derivation rule to use depends on the syntactic form of q, which can be one of the four forms of $\mathbb{A}$ listed in figure 2.2.

If q is a Cut, rule $\texttt{runE}_!$ discards the current alternatives a and replaces them with the cut-to alternatives stored in q. Evaluation then continues with the remaining goals g.

If q is a pi-quantification $\texttt{pi}x.t$, the interpreter generates a fresh nominal variable y not occurring in $\mathcal{P}$, replaces all free occurrences of x in t with y, and continues evaluation with the resulting atom.

If q corresponds to the dynamic insertion of a rule h, evaluation continues with atom t in the program extended with the local rule h. Intuitively $h \Rightarrow t$ amounts to a well-bracketed use of *Prolog*'s `assert/1` and `retract/1`.

Finally, if q is a predicate call, the interpreter performs *backchaining* over the rules stored in $\mathcal{P}.\texttt{index}$. Backchaining retrieves the rules that can be used on the query q. In standard *Prolog* this is done by selecting rules whose head unifies with q. In *Elpi* the process depends on predicate modes, which determine if a argument should be "matched" or "unified".

Elpi provides two procedures, `unify` and `matching`. Both have type

$$\mathbb{Tm} \to \mathbb{Tm} \to \Sigma \to \Sigma + \square,$$

taking two terms and an initial substitution σ. They either fail (returning $\square$) or succeed with an updated substitution σ' that is an extension of σ.

The procedure `unify` implements higher-order unification restricted to the pattern fragment [40] and is used for arguments in `output` position. The procedure `matching`, inspired by *B-Prolog*'s "matching clauses" [64], is used for arguments in `input` position.

Definition 2.2.3 (`unify`)**.** Two terms q and u *unify* under a substitution σ if there exists a substitution σ' extending σ such that $\sigma' q = \sigma' u$, and σ' is a most general unifier for the two terms.

Definition 2.2.4 (`matching`)**.** A query q *matches* a pattern u under a substitution σ if there exists a substitution σ' extending σ such that $\sigma' u = \sigma q$, where σ' assigns at most the free variables occurring in u.

Backchaining is defined by the function $\mathcal{B}$:

Definition 2.2.5 (Backchain, $\mathcal{B} : \mathbb{P} \times \texttt{set}\ \mathbb{U} \times \mathbb{Tm} \times \Sigma \to \overrightarrow{\Sigma \times \mathbb{A}}$)**.**

$$\mathcal{B}(\mathcal{P}, V, q, \sigma) = \left[(\sigma', b)\ \texttt{if}\ \mathcal{U}(\mathcal{S}(\mathcal{P}.\text{sig}, q), q, h, \sigma) = \sigma' \neq \square \;\middle|\; (h :\!- b) \in \text{fresh}(V, \mathcal{P}.\text{index}) \right].$$

Intuitively, $\mathcal{B}$ selects the rules that can be applied to the current query q under a substitution σ. The variables occurring in the program rules are assumed to be disjoint from those in the current execution state. To guarantee this, the program is refreshed using the function `fresh`, which renames its variables with respect to the set V provided as input to $\mathcal{B}$.

For each applicable rule, $\mathcal{B}$ returns the substitution obtained by matching/unifying the rule head with q, together with the corresponding rule body.

The definition relies on two auxiliary functions:

$$\mathcal{S} : \mathbb{E} \to \mathbb{Tm} \to \mathbb{Ty} + \square \tag{2.1}$$

$$\mathcal{U} : (\mathbb{Ty} + \square) \to \mathbb{Tm} \to \mathbb{Tm} \to \Sigma \to \Sigma + \square. \tag{2.2}$$

The function $\mathcal{S}$ retrieves the signature of the predicate occurring in a term. It returns the signature if the predicate is present in the environment, and $\square$ otherwise.

The function $\mathcal{U}$ tests whether the query q matches/unifies the head h of a rule under an initial substitution σ. The predicate signature determines the mode of each argument, which in turn selects the appropriate procedure: `matching` for `input` arguments and `unify` for `output` arguments. If all arguments satisfy the unification condition, $\mathcal{U}$ returns the resulting substitution σ'; otherwise it returns $\square$.

Definition 2.2.6 (*Elpi* backchain, $\mathcal{B}_E : \mathbb{P} \times \mathbb{Tm} \times \Sigma \times \overrightarrow{\mathcal{A}} \times \overrightarrow{\mathcal{G}} \to \overrightarrow{\mathcal{A}}$)**.**

$$\mathcal{B}_E(\mathcal{P}, t, \sigma, a, g) = \left[\sigma', ([(\mathcal{P}, q, a) \mid q \in b] +\!\!+ g) \;\middle|\; (\sigma', b) \in \mathcal{B}(\mathcal{P}, \text{get\_fv}\ (t, g :: a, \sigma), t, \sigma) \right].$$

runE rules	run rules
$\texttt{runE}_\top$	$\texttt{run}_\top$
$\texttt{runE}_\bot$	unreachable case
$\texttt{runE}_\mathcal{B}$	$\texttt{run}_{\circlearrowleft,\bot,\sigma,@}$ + IH
$\texttt{runE}_!$	$\texttt{run}_!$ and IH
$\texttt{runE}_\forall$	$\texttt{run}_\forall$ + IH
$\texttt{runE}_\Rightarrow$	$\texttt{run}_\Rightarrow$ + IH

(a) runE → run

run rules	runE rules
$\texttt{run}_\top$	$\texttt{runE}_\top$
$\texttt{run}_\bot$	$\texttt{runE}_\mathcal{B}$ and $\texttt{runE}_\bot$
$\texttt{run}_{\circlearrowleft}$	$\texttt{runE}_\mathcal{B}$ and IH
$\texttt{run}_@$	$\texttt{runE}_\mathcal{B}$ and IH
$\texttt{run}_\sigma$	$\texttt{runE}_\mathcal{B}$ and IH
$\texttt{run}_!$	$\texttt{runE}_!$ and IH
$\texttt{run}_\forall$	$\texttt{runE}_\forall$ and IH
$\texttt{run}_\Rightarrow$	$\texttt{runE}_\Rightarrow$ and IH

(b) run → runE

Table 2.1: High-level proof schema of theorem 2.2.1

The function $\mathcal{B}_E$, used by rule $\texttt{runE}_\mathcal{B}$ in figure 2.3, builds a new list of alternatives from the result of $\mathcal{B}$ (get_fv is the function retrieving the list of free variables in the term t, the list of alternatives $g :: a$ and the substitution σ). For each filtered rule, the atoms in the rule body become new goals whose cut-to alternatives are a and whose continuation is g. The substitution associated with each alternative is the one returned by $\mathcal{B}$.

Comparison with Tassi's hdr In the previous subsections, we presented the *Elpi* language, including its syntax and semantics. This formalization has been developed in collaboration with Enrico Tassi. A comprehensive description of *Elpi* can be found in his work *hdr* [57].

We highlight here the main differences between his presentation and the one adopted in this section. First, in the syntax, we use binary application for terms, instead of the n-ary application used in *hdr*. Second, in the semantics, programs carry not only the list of rules and nominal variables, but also the signature of declared predicates. We also omit the formal treatment of constraints, as it is not required for the remainder of this manuscript; a brief informal explanation is sufficient for our purposes (see section 2.2.5).

Another difference concerns the definition of runE. In Tassi's presentation, runE, called run, is defined as a procedure that takes two arguments: the current alternative and the list of future alternatives. This choice reflects closely the concrete implementation in *Elpi*, where execution starts from a query, and a query is represented as an alternative. In contrast, we define runE with a single input, namely a list of alternatives, from which the first element is extracted by pattern matching when needed. This design choice is motivated by the mechanization in *Rocq* (see chapter 6), where we need to prove the equivalence between two semantics and to represent the execution of runE on an empty list of goals. Despite these differences, as the theorem below claims, the two semantics are equivalent when executed on non-empty lists of alternatives, modulo the absence of constraints, which is forced to the empty list in the run semantic (this is performed by the mapping of one list to the other and putting the empty set, i.e. $\varnothing$, of constraints).

Theorem 2.2.1.

$$\begin{aligned}\forall\ a_0\ a_s\ r\ \sigma,\ &\texttt{let}\ (a_0', a_s') := \texttt{map}\ (\lambda x.(\varnothing, x))\ (a_0 :: a_s)\ \texttt{in}\\ &\texttt{let}\ r' := \texttt{map}\ (\lambda x.(\varnothing, x))\ r\ \texttt{in}\\ &\texttt{runE}\ (a_0 :: a_s) \rightsquigarrow (\sigma, r') \leftrightarrow \texttt{run}\ a_0'\ a_s' \rightsquigarrow (\varnothing, \sigma, r')\end{aligned}$$

```
pred f i:int, o:int.
f 1 2.
f 2 3.
pred r i:int, o:int.
r 0 0.
r 0 1.
r 0 2 :- !.
pred g i:int, o:int.
g A C :- r A B, f B C, !.
g D F :- f D E, f E F.
```

Figure 2.4: Running example of a *Elpi* program

Proof.

The proof is relatively simple. It is based on two inductions, one for each direction of the $\leftrightarrow$ operator, respectively on `runE` and `run`. The intuition is illustrated in table 2.1, where IH denotes the induction hypothesis.

The first subtable corresponds to the left-hand side of the implication, while the second one corresponds to the right-hand side. In both cases, the goal is to relate the constructors of the two semantics.

More precisely, we match each constructor of one semantics with the corresponding constructors of the other, and, since our semantics has fewer constructors, depending on the considered case, several constructors of `run` may be used on a single constructor of `runE`. We also point out, that the $\texttt{runE}_\bot$ case in 2.1a, is unreachable since we start from an non-empty list of goals.

□

2.2.3 Example of program execution in *Elpi* with `Cut`

We believe that the main difficulty in the semantics presented in the previous section lies in the treatment of choice points and their interaction with the `Cut` operator.

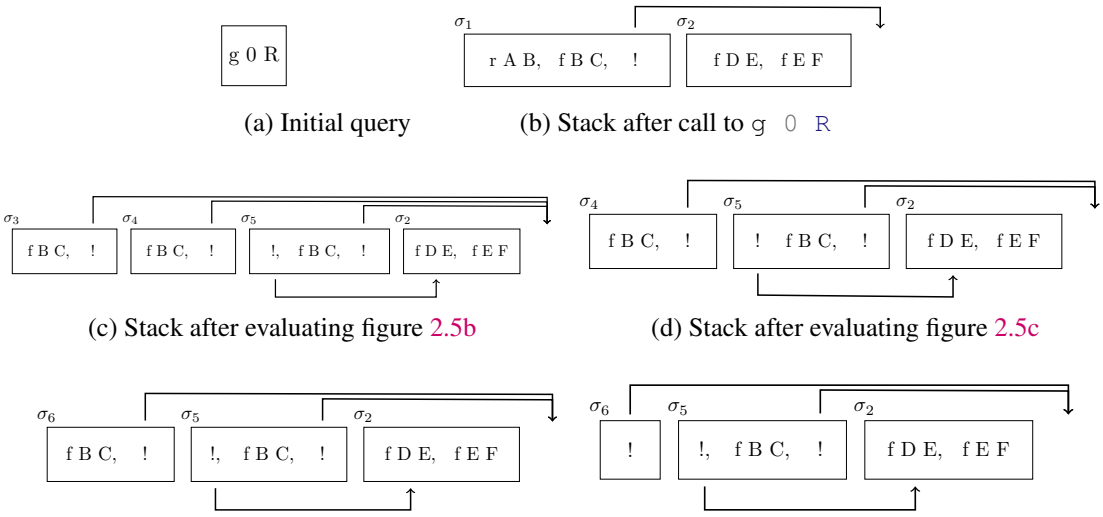
Several variants of the cut operator exist in the literature. The variant we consider, used in *Elpi*, is the so-called *hard cut* (see, for example, the documentation of *SWI-Prolog* [54]). When the `backchain` function returns a non-empty list of alternatives, the first alternative is executed immediately, while the remaining ones are stored as choice points.

Within a selected alternative, premises are evaluated from left to right, each atom being executed in sequence. When a `Cut` is encountered, it removes two kinds of choice points: (i) those generated by `backchain` for the current predicate call, and (ii) those created during the evaluation of premises to the left of the `Cut`.

As a running example, we consider the program shown in figure 2.4. For clarity, figure 2.5 illustrates its execution, explicitly showing the pointers to the cut-to alternatives. We only highlight the alternatives targeted by the `Cut` operator, since the alternatives stored for predicate calls do not play a role in the semantics discussed here.

Consider the program in figure 2.4 and the query `g 0 R`. Both rules for `g` apply to this query. Backchaining therefore returns the following two alternatives:

$$[(\sigma_1, [\texttt{r A B, f B C, !}]), (\sigma_2, [\texttt{f D E, f E F}])]$$



(a) Initial query (b) Stack after call to `g 0 R`

(c) Stack after evaluating figure 2.5b (d) Stack after evaluating figure 2.5c

(e) Stack after evaluating figure 2.5d (f) Stack after evaluating figure 2.5e

Figure 2.5: Example of execution

with $\sigma_1 := \{\texttt{A} \mapsto \texttt{0}, \texttt{C} \mapsto \texttt{R}\}$ and $\sigma_2 := \{\texttt{D} \mapsto \texttt{0}, \texttt{F} \mapsto \texttt{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_\nu$ any further.

Execution continues with the first premise of the first alternative, namely `r A B` under σ_1, i.e. `r 0 B`. The remaining alternatives are stored as choice points. Backchaining `r 0 B` returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [\texttt{!}])]$ where $\sigma_3 := \sigma_1 + \{\texttt{B} \mapsto \texttt{0}\}$; $\sigma_4 := \sigma_1 + \{\texttt{B} \mapsto \texttt{1}\}$ and $\sigma_5 := \sigma_1 + \{\texttt{B} \mapsto \texttt{2}\}$.

The next step of interpretation continues with `f B C` under σ_3, that is, `f 0 R`, and leaves $[(\sigma_4, []), (\sigma_5, [\texttt{!}])]$ as new choice points. This time, backchaining fails (no rule for `f` matches input 0). We therefore backtrack to the most recently introduced choice point and retry `f B C` under σ_4, i.e. `f 1 R`. This succeeds with $\sigma_6 := \sigma_4 + \{\texttt{R} \mapsto \texttt{2}\}$.

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for `r` and one from the second rule for `g` (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for `g` was selected. In contrast, a cut does not discard choice points created earlier; in this example, there are none. The final solution is: $\{\texttt{A} \mapsto \texttt{0}, \texttt{B} \mapsto \texttt{1}, \texttt{C} \mapsto \texttt{R}, \texttt{R} \mapsto \texttt{2}\}$.

2.2.4 Higher-order, dynamic and meta- programs in *Elpi*

One of the main features of *Elpi* is the possibility to work with higher-order and dynamic programs. In this section, we present some simple examples illustrating these two aspects. These examples will be used later as a reference when discussing determinacy checking in *Elpi* (see chapter 5).

Higher-order programming A well-known higher-order predicate in *Elpi* is `map`, whose implementation is shown below.

```
pred map i:(pred i:A, o:B), i:list A, o:list B.
map _ [] [].
map F [X|XS] [Y|YS] :- F X Y, map F XS YS.
```

This is a standard Prolog-style definition. The predicate takes a higher-order argument `F`, which is applied to each element of the input list. In the base case, the empty list is mapped to the empty list. In the recursive case, the head `X` is transformed into `Y` using `F`, and the predicate is then applied recursively to the tail `XS`.

Another common higher-order construct in *Elpi* is the predicate `once`, defined as follows.

```
pred once i:(pred).
once P :- P, !.
```

The predicate `once` acts as a wrapper around a goal. It executes the goal `P` and then removes all remaining choice points. As a consequence, only the first solution is returned. For example, the query `once (r 0 X)`, where `r` is defined in figure 2.4, produces a single result, binding `X` to `0`.

The behavior of the `Cut` operator depends on where it is placed. The following snippets illustrate similar programs with different uses of `Cut`.

```
pred p1 i:int, o:int.
p1 1 X :- r 0 X.
p1 1 3.
```

```
pred p2 i:int, o:int.
p2 1 X :- r 0 X, !.
p2 1 3.
```

```
pred p3 i:int, o:int.
p3 1 X :- once (r 0 X).
p3 1 3.
```

The query `p1 1 X` returns four solutions for `X`, namely `0, 1, 2`, and `3`. This is because no `Cut` is used, so all possible solutions are explored by backtracking.

The query `p2 1 X` returns only one solution, `X = 0`. In this case, after the first solution of `r 0 X` is found, the `Cut` removes all choice points, both for `r` and for `p2`.

Finally, the query `p3 1 X` returns two solutions, `0` and `3`. Here, the use of `once` removes the choice points generated by `r`, but it does not affect the second rule for `p3`.

Dynamic programming The domain where *Elpi* really shines is the manipulation of higher-order data represented via the so-called λ-tree syntax [42] also called HOAS [47]. Such data is pervasive in interactive provers based on Type Theory such as *Rocq*. In the paradigmatic example below `tm` is the data type for λ-terms.

```
kind tm type.                    % datatype of λ-terms in HOAS form
type app tm -> tm -> tm.         %  - binary application
type lam (tm -> tm) -> tm.       %  - λ-abstraction
```

In the HOAS encoding of syntax with binders we do not find a node for variables: we re-use the notion of bound variable of the programming language to encode the one of the syntax tree. At the level of types we see that the `lam` node carries an *Elpi* function over terms. It is by applying this function that one can recover the body of the λ-abstraction. A simple program manipulating higher-order data is the function performing a deep copy of a term:

```
pred copy i:tm, o:tm.
copy (app A B) (app C D) :- copy A C, copy B D.
copy (lam F) (lam G)     :- pi x\ copy x x => copy (F x) (G x).
```

The copy of an application is built from the copies of the subterms. What is more interesting is how one copies under a binder. The "`pi x\ copy x x =>`" instruction explains how to copy the new constant `x`. Finally the code performs a recursive call over `F x`: by applying the *Elpi* function `F` to the fresh symbol `x` one obtains the body of the λ-abstraction where the bound variable is replaced by the new symbol `x`.

In *Elpi*, all predicates are *dynamic*, or open, hence the code of `copy` can be augmented with rules at runtime.

The `copy` predicate, as given, implements the identity and may seem rather useless, but it is not! Since the program is open, one can add new rules for `copy` and change its behavior. For example the code below computes the weak head normal form of a λ-term by contracting the application of a λ-abstraction to an argument and uses `copy` to perform the substitution:

```
pred whd i:tm, o:tm.
whd (app H A) R :- whd H (lam F), !,          % we fire the redex
  pi x\ copy x A => copy (F x) S, whd S R.    %   and we continue
whd X X.                                      % or we stop
```

The argument `A` is put in place of the (previously) bound variable in `F` by replacing that variable with `x` and by running `copy` with the addition of a special rule copying `x` to `A`. Our static analysis understands this programming pattern and is able to recognize that `whd` is also a function. In particular the first rule, if taken, discard the second rule (due to the cut) and only calls functions.

Meta-programming In *Elpi*, code is a first-class object, meaning that it can be inspected and generated within the language itself. As an example, consider a program that performs the fusion of two calls to `map` [60].

```
pred comp i:(pred i:A, o:B), i:(pred i:B, o:C), i:A, o:C.
comp F G X Z :- F X Y, G Y Z.

pred fuse i:(pred i:list A, o:list B), i:(pred i:list B, o:list C),
          o:(pred i:list A, o:list C).
fuse (map F) (map G) (map (comp F G)).
```

The predicate `fuse` analyzes two programs. If both correspond to mapping a list, with functions `F` and `G`, it produces a new program of the form `map (comp F G)`, where `comp` denotes function composition.

This approach can improve efficiency. For instance, instead of executing the sequence `map F L1 L2, map G L2 L3`, which traverses the list twice, one can combine the two computations. By calling `fuse (map F) (map G) H`, a new program `H` is obtained. Then, `H L1 L3` computes the same result with a single traversal of the list.

More generally, meta-programming is widely used in *Rocq-Elpi*. It allows the generation of new rules that can be stored on disk and reused in later executions of the program.

2.2.5 *CHR*: goal suspension

Constraint Handling Rules (*CHR*, [19, 18]) is a declarative, rule-based language designed to manipulate a multiset of constraints. A *CHR* program consists of rules that match patterns of constraints, optionally guarded by conditions, and transform the current constraint store by adding or removing elements. Operationally, execution proceeds by repeatedly selecting applicable rules and applying them to the constraint store. This model offers a flexible way to express simplification, propagation, and their combination, known as simpagation.

From a conceptual point of view, *CHR* can be seen as a rewriting system over constraints, where rules describe how local configurations evolve. In contrast to standard logic programming

rules, which operate on a single goal, *CHR* rules may inspect and rewrite several constraints at once. This aspect is important when correctness depends on the interaction between constraints, rather than on their independent resolution. For this reason, *CHR* naturally supports a more global view of computation.

In *Elpi*, *CHR* is integrated as an extension of the underlying λ-*Prolog* engine. The motivation for this integration comes from the need to combine and refine suspended constraints generated during proof search.

We do not enter into the details of the syntax or the implementation of *CHR*, since they play only a minor role in this manuscript. A more complete presentation can be found in [25, Sec. 4.3] and [57, Sec. 2.4].

2.2.6 Rule names and insertion

In the *Elpi* database, the rules used during backchaining are stored in an ordered data structure, together with their implementation. Each rule can optionally be given a name by the user.

These names are useful when adding new rules that should have higher or lower priority than existing ones. In this way, the user can control the position of rules in the search order.

In *Elpi*, inserting a rule before or after an existing one is called *grafting*.

```
:name "nth-fail"
nth _ _ :- error "The list is too short".
```

The definition of `nth` above is a catch-all rule, since it applies to any query. This rule is named `nth-fail`. When defining additional rules for `nth`, one can write:

```
:before "nth-fail"
nth 0 [X|_] :- X.
```

This declaration places the new rule just before the catch-all rule. A similar syntax can be used with `:after` to insert a rule after an existing one.

The usefulness of grafting becomes clearer in section 3.4.1, where typeclass instances must be inserted at specific positions in the database to respect the intended priority.

2.3 *Rocq-Elpi*: A Logical Framework for *Rocq*

The *Rocq-Elpi* language is a plugin for *Rocq* that enables the use of *Elpi* within the *Rocq* environment. In the following sections, we first explain the idea behind encoding *Rocq* terms in *Elpi* (see section 2.3.1) using higher-order abstract syntax (HOAS) [47]. In section 2.3.2, we provide some important API that are widely used in *Rocq-Elpi*. Finally, in section 2.3.4, we provide a brief description of how commands and tactics are written in *Rocq-Elpi* and how they integrate with *Rocq*.

2.3.1 HOAS: λ-terms representation

In section 2.2.4, we sketched how to encode the syntax of a language using the λ-tree syntax of *Elpi*, also known as higher-order abstract syntax (HOAS) [47]. This syntax is inspired by the λ-calculus, which forms the core of the *Rocq* system.

```
kind term type.

type sort    sort → term.                    % Prop, Type@{i}
type global  gref → term.

type app     list term → term.               % app [hd|args]
type fun     name → term → (term → term) → term.
type prod    name → term → (term → term) → term.

...                        % Other constructors omitted
```

Figure 2.6: HOAS of *Rocq* terms in *Elpi*

Rocq terms	*Elpi* HOAS	description
`plus 0`	`app [<<plus>>, <<0>>]`	term application
`fun x => ?F a`	`fun `x` y\ app [G, <<a>>]`	λ-abstraction
`∀ x, p x`	`prod `x` z\ app [<<p>>, z]`	∀-quantif and binding

Table 2.2: Examples of *Rocq* terms encoded in *Elpi* using HOAS

The encoding of *Rocq* terms is performed using a special *Elpi* type called `term`. The snippet depicted in figure 2.6 shows the main constructors for terms.

The base constructors are `sort`, which encodes the **`Type`** and **`Prop`** terms of *Rocq*, and `global`. The `global` constructor represents user-defined symbols such as types, inductives, and constants.

These symbols are identified by a `gref`, which is an opaque object containing the unique identifier of the corresponding *Rocq* symbol. For better space management we use a more compact notation: opaque *Rocq* symbols are written using guillemets, avoiding the explicit use of the `global` constructor. For instance, the *Rocq* constant `nat` is represented in HOAS form as `<<nat>>`.

Term application in *Rocq* is represented by the `app` constructor, which takes a list of terms. λ-abstractions and ∀-quantifications are encoded using the `fun` and `prod` constructors, respectively. Each of these constructors carries the `name` of the *Rocq* binder (used only for pretty printing), the type of the binder, and an *Elpi* higher-order object of type `term → term`, which represents the abstraction of the bound variable in the term.

The *Rocq-Elpi* backend translates *Rocq* terms into HOAS *Elpi* terms and conversely when the two systems interact. In table 2.2, we show some examples of this encoding: the first column contains *Rocq* terms, and the second column their HOAS representation in *Elpi*.

The second and third rows illustrate how binders are handled for λ-abstraction and ∀-quantification. The original binder name from *Rocq* is stored in the `fun` and `prod` constructors, while the name of the bound variable in *Elpi* is not relevant. In the third row, the *Elpi* binder (called `z`) is reused to represent the abstraction of the object language when applied to the predicate `p`.

The second row shows a *Rocq* unification variable `?F`. It is translated into an *Elpi* unification variable `G`, which internally corresponds to `?F`. When `G` is instantiated during the execution of the *Elpi* program, the result is translated back into a *Rocq* term and assigned to `?F`.

Quotations and Antiquotations Writing *Rocq* terms can be tedious, as it requires explicitly constructing their full HOAS representation. As shown in table 2.2, the *Elpi* HOAS syntax is significantly more verbose than standard *Rocq* syntax. Moreover, *Rocq* often includes implicit subterms, whereas *Elpi* does not provide such a mechanism.

To simplify development, *Rocq-Elpi* offers a system of quotations and antiquotations. This mechanism allows embedding *Rocq* syntax within *Elpi* code, and conversely *Elpi* syntax within *Rocq* code, in a recursive manner.

An illustrative example, adapted from [57], is the following:

```
print {{1 + lp:{{app[<<S>>, {{ 0 }}]}}}}
Elpi  Rocq  Elpi  Rocq  Elpi  Rocq
```

Within an *Elpi* code block, the syntax `{{ ... }}` denotes a quotation that allows writing *Rocq* code. Conversely, within *Rocq* code, the syntax `lp:{{ ... }}` denotes a quotation for embedding *Elpi* code. These constructs can be nested arbitrarily.

2.3.2 Database and Main API in *Rocq-Elpi*

Thanks to the HOAS representation of *Rocq* terms in *Elpi*, the two languages become closer and can share a common representation. This makes their interaction easier and allows interleaving computations between them. The *Rocq* language exposes several APIs that can be used directly from *Elpi*.

```
pred coq.unify-eq i:term, i:term, o:diagnostic.
```

The predicate above is widely used in *Elpi* developments. It takes two terms and attempts to unify them. If unification fails, it returns a diagnostic explaining the error. Internally, this predicate relies on the *Rocq* unification algorithm. When *Elpi*coq.unify-eq is called, the HOAS representation of the terms is translated into *Rocq* terms before performing the unification.

Other predicates allow extending the *Rocq* system with new definitions, modules, modes, and more.

```
pred coq.TC.declare-class i:gref.
```

For example, the predicate above allows declaring a type class from a global reference in *Rocq* directly within *Rocq-Elpi*.

2.3.3 Declaring Databases in *Rocq-Elpi*

In [57, Sec. 4.4.3], a database is defined as a non-executable *Elpi* program. It contain a collection of code that can be included into commands or tactics by referring to its name.

When a program is executed, it does not use the state of the database at the moment when the command was defined. Rather, it accesses the current content of the database at execution time. This means that any update to the database is visible to all programs that include it.

An *Elpi* database can be dynamically modified by adding new predicate signatures and rules.

```
kind clause type.
%           Name      Grafting    Rule
type clause string → grafting → prop → clause.
```

```
%                              DB name   Indexing   PName
pred coq.elpi.add-predicate i:string, i:string, i:string,
                                    i:list (pair argument_mode string).
%                          Scope     DB name    Clause
pred coq.elpi.accumulate i:scope, i:string, i:clause.
```

The `coq.elpi.add-predicate` predicate takes as arguments the name of the database where the predicate is inserted, the indexing strategy[†], the predicate name, and its specification (argument modes and types).

The `coq.elpi.accumulate` predicate adds a new clause to a database within a given scope. A clause is a specific type with a single constructor, which contains the clause name, its grafting with respect to the existing database, and the rule itself.

These last two predicates are particularly useful in the context of sections 3.3 and 3.4.

2.3.4 Writing Commands and Tactics in *Rocq-Elpi*

We now describe how to implement commands and tactics in *Rocq-Elpi*. In table 2.3, we summarize how to define commands and tactics, how to invoke them, and what their entry points are. Both commands and tactics are typically built on top of an *Elpi* database, which they can read and, if needed, modify. A database is added using the command **`Elpi Accumulate`** `Db db_name`. The code of commands and tactics can be extended as follows:

```
Elpi Accumulate [command_name|tactic_name] lp:{{
  % New rules added here
}}.
```

As mentioned at the beginning of this chapter, commands are used to extend the *Rocq* language with new definitions and theorems. In other words, a command does not return a value: it is mainly used for printing or for modifying the internal state of the system. The entry point of a command is a predicate named `main`, with signature **`pred`** `main` **`i:list argument`**. An argument can be a string, an integer, a term, or an open term.

Tactics in *Elpi* are used to solve *Rocq* goals through the *Elpi* interpreter. Their declaration is similar to that of commands. The entry point of a tactic is the predicate `msolve`, with the following type:

```
pred msolve i:list sealed-goal, o:list sealed-goal.
```

[†]We do not discuss indexing in detail, as it is not central for this manuscript. It determines the data structure used to store rules in order to improve search efficiency.

Commands	Tactic	Description
`Elpi Command` `command_name.`	**`Elpi Tactic`** `tactic_name.`	Declaration
`Elpi` `command_name.`	`elpi tactic_name.`	Invocation
`main`	`msolve` and `solve`	Entry point

Table 2.3: Command and Tactic main commands

```
kind goal type.
kind sealed-goal type.
type nabla (term → sealed-goal) → sealed-goal.
type seal  goal → sealed-goal.

typeabbrev goal-ctx (list prop).
%          Context    RS     Ty       Sol     Arguments
type goal  goal-ctx → term → term → term →
           list argument → goal.
```

Figure 2.7: Representation of goals and sealed goals in *Rocq-Elpi*

The `msolve` predicate takes as input a list of *Rocq* sealed goals and returns an updated list of sealed goals. The type `sealed-goal`, shown in figure 2.7, represents goals that may contain abstractions. These abstractions are encoded using the `nabla` constructor.

Under a spine of `nabla` abstractions, the `seal` constructor contains a goal. A goal is a tuple with five components: a context C, a raw solution S_r, a type T_y, a final solution S, and a list of arguments L.

The context C maps free variables to their types. The term S_r is a trigger: when it is instantiated, it is elaborated against the type T_y. The type T_y is the type of the *Rocq* goal to be solved. The term S is the final solution, which must have type T_y. The list L contains arguments that control the behavior of the resolution process. In this work, we mainly use two projections of goals, namely the context and the goal type.

If the predicate `msolve L N` is not defined, or if it fails when applied to the list of goals `L`, then the system attempts to solve each goal in `L` individually using the predicate `solve`. In this case, `solve` is called on the content of the `seal` constructor, meaning that all `nabla`s have already been traversed.

```
pred solve i:goal, o:list sealed-goal.
```

The final point addressed in this section concerns the `refine` predicate.

Given a goal `G` and a proof term `P`, one may wish to typecheck `P` against the `G` and obtain the corresponding list of subgoals that must be solved recursively. The *Rocq* tactics performing this task is called `refine`.

In *Elpi*, the `refine` predicate exists and has the same behavior: it takes a proof term and a goal, it typechecks the proof agains its type and returns a list of subgoals on the for of a list of sealed-goals. It is customary for *Elpi* tactics to invoke `refine` in order to produce the output list the `solve` predicate.

For performance reasons, *Elpi* also provides a variant, `refine.no_check`, which skips typechecking and directly produces the list of sealed subgoals from the given term and goal. This results in faster, although less reliable, computation.

CHAPTER 3

Architecture of a Type-Class Solver in *Rocq-Elpi*

Type classes provide an elegant mechanism for implementing ad-hoc polymorphism [60]. As explained in section 2.1.1, they allow the same symbol to be reused in different contexts, and in a proof assistant like *Rocq*, instances of a type class preserve the same mathematical properties. From a Prolog-like perspective, a type-class goal of *Rocq* corresponds to resolving a query to a predicate that associates its inputs with an instance solving the type-class constraint. In *Rocq*, instances act as rules stored in a database, which are used to backchain and resolve type-class queries.

A meta-language integration for type-class resolution must provide two components: (1) a compiler that translates *Rocq* class and instance declarations into the meta-language representation, and (2) a type-class solver that animates the search by interacting with the resulting database.

In our setting, the meta-language used to implement the meta-solver is *Elpi*. Consequently, the main task is to implement the class and instance compiler, which incrementally builds the database on which the *Elpi* program operates.

As explained in section 2.1.1, the type-class solver itself typically follows a *Prolog*-like resolution strategy: instances are tried according to their priority, and backtracking may reconsider previous choices when a branch fails. In *Elpi*, this behavior comes for free, since the resolution mechanism of the type-class solver strongly imitates the standard execution model of the *Elpi* runtime.

The remainder of this work is organized as follows. We first demonstrate how *Rocq* terms can be compiled into an *Elpi* database and how type-class goals can be translated into *Elpi* queries, enabling proof automation through *Elpi*'s search mechanism. In section 3.2, we present a minimal instance compiler, showing how a basic type-class solver can be implemented with few lines of code. While simple, this approach lacks important features required for handling complex instances. We therefore introduce a more powerful compiler in sections 3.3 and 3.4.

Next, in sections 3.4.1 to 3.4.3, we discuss how rule priorities influence database construction, how *Rocq* goals are encoded as *Elpi* queries, and how users can extend the database with custom rules to tailor the solver's behavior.

We then report on our experimental evaluation in section 3.5, where the solver is applied to real-world libraries such as *stdpp* and *tlc*. Benchmarks presented in section 3.5.1 compare the performance of *Rocq* and *Elpi*, analyzing how execution time in *Elpi* is distributed across different

phases of the search. In section 3.5.2, we examine the differences between *Elpi* and *Rocq* modes and their impact on resolution. Finally, in section 3.5.3, we discuss the unification challenges that arise when using *Elpi*'s unification algorithm on *Rocq* terms. The discussion concludes in section 3.6.

3.1 Interfacing *Rocq* with *Elpi* for Type-Class Resolution

To address type-class unification problems, we extend *Rocq* with the following APIs (simplified here for clarity):

```
(* API for Compiler *)
val register_observer : string → (event → unit) → unit
val activate_observer : string → unit
val deactivate_observer : string → unit

(* API for Solver *)
val register_solver : string → tc_solver → condition → unit
val activate_solver : string → unit
val deactivate_solver : string → unit
```

This APIs allow one to declare new compilers and new solvers inside the *Rocq* system.

A compiler is implemented as an observer, which is triggered whenever a new class or instance is declared. To define a new `observer`, one provides a name and an event handler to `register_observer`. The observer is stored in a table mapping names to handlers. The handler is a higher-order function, invoked each time an event occurs. Its role is to process the received information and update its internal database. Observers can be activated or deactivated when needed.

A type-class solver is registered via `register_solver`, which takes a name, a value of type `tc_solver`, and a condition. The `tc_solver` is a higher-order function that receives a *Rocq* context and a list of goals, and returns an updated environment where some type-class goals may be solved. The condition explains if the current solved is meant to solve the current list of type-class goals. This is useful, for example, if the aim of a solver is meant to be used on particular type-class goals.

Solvers are also stored in a table indexed by their name. During resolution, the *Rocq* type-class mechanism scans the registered solvers and selects the first active one that satisfies its condition. If the condition does not hold, the next solver is considered. As for observers, solvers can be activated or deactivated by the user.

3.2 Simple instance compiler

Conceptually, a type-class goal can be translated into a query with two arguments: the goal itself and a concrete instance witnessing it. The search is expressed using the predicate `pred tc term -> term`.

By invariant, a query of the form `tc T I` ensures that the term `I` has type `T`.

As an example, consider the `Add` type-class introduced in section 2.1.1. For convenience, the full definition of the class and its instances is reproduced in figure 3.1.

```
Class Add T := {plus: T → T → T}.

Instance addNat : Add nat := {plus := Nat.add}.
Instance addR : Add R  := {plus := Rplus}.

Instance addProd : ∀ T1 T2, Add T1 → Add T2 → Add (T1 * T2) :=
  {plus '(x1,y1) '(x2, y2) := (plus x1 x2, plus y1 y2)}.
```

Figure 3.1: Add class and its instances

```
% Inst Ty Args Prems Res
pred comp term, term, list term, list prop → prop.
comp I {{lp:T → lp:Bo}} Ag P (pi y\ R y) :- !,   % r→
  pi x\ comp I Bo [x|Ag] [tc T x | P] (R x).
comp I {{∀ x, lp:(Bo x)}} Ag P (pi y\ R y) :- !, % r∀
  pi x\ comp I (Bo x) [x|Ag] P (R x).
comp I T Ag P (tc T Proof :- [true | Body]) :-   % rB
  mk-app I {rev Ag} Proof,
  rev P Body.

pred compile gref →.
compile G :- coq.env.typeof G Ty,
  comp (global G) Ty [] [] R,
  coq.elpi.accumulate _ "tc.db" (clause _ _ R).
```

Figure 3.2: Simple *Elpi* compiler

After an instance is declared, an observer triggers the instance compiler shown in figure 3.2. The compiler is similar to the example presented in [57, Sec. 4.5].

The entry point of the compiler is the predicate `compile`, which takes the global reference `G` of the instance as input. Using the `coq.env.typeof` API, the compiler retrieves the type `Ty` of `G`. Compilation then proceeds by invoking the auxiliary predicate `comp` to produce the rule `R`. Finally, `coq.elpi.accumulate` adds the clause `R` to the database, here called `tc.db`.

The predicate `comp` performs the core of the compilation. It takes four arguments: the instance term; its type, which is structurally consumed during recursion; and two accumulators storing the list of instance arguments and the list of premises that will form the body of the resulting rule. From these inputs, `comp` produces the *Elpi* rule corresponding to the compiled instance.

Compilation proceeds by analyzing the structure of the instance type. Three cases are relevant. Rules $r{\to}$ and $r{\forall}$ handle quantifications, while rule rB is the base case that constructs the final *Elpi* rule from the information accumulated during the recursion.

The base case is reached when the current type `Ty` is not a quantification. The compiler then constructs the final rule by assembling its head and body.

The head is the predicate `tc` applied to the instance type `T` and to a proof term `Proof`. The proof is obtained by applying the instance `I` to its arguments `rev Ag`. Since the arguments are accumulated in reverse order during the traversal, the list `Ag` is reversed before building the application.

The premises of the clause are stored in the accumulator `P`. As for the arguments, the list is reversed before constructing the final rule.

For example, compiling the instance `addNat` produces the rule

```
tc {{Add nat}} {{addNat}}.
```

When the current type `Ty` is a quantification, the compiler introduces a fresh nominal variable representing the bound variable in the recursive call. Two situations must be distinguished depending on the shape of the quantification.

If the quantification corresponds to a premise of the instance, rule $r{\to}$ is used. This rule introduces a fresh variable `x` representing the conditional instance with type `T`. The premise `tc T x` is added to the list of premises `P`, and the variable `x` is added to the list of instance arguments `Ag`.

Rule $r{\forall}$ handles dependent quantification corresponding to parameters of the instance, that are supposed not to have a class as type. In this case the abstraction is simply consumed: the list of premises `P` remains unchanged, while the variable `x` is added to the list of instance arguments.

Both rules $r{\to}$ and $r{\forall}$ return a term of the form `(pi y\ R y)`, where `R` is the result of the recursive call to `comp`. Since `R` has type `term -> prop` and abstracts the local variable `x`, the compiler introduces a fresh variable `y` to quantify over the corresponding argument. This ensures that the resulting rule is closed (ground), as required by the `coq.elpi.accumulate` predicate.

As an example, consider the instance `addProd`, whose type is

$$\forall \underbrace{x\ y}_{r\forall}, \underbrace{\texttt{Add}\ x \to \texttt{Add}\ y}_{r\to} \to \underbrace{\texttt{Add}\ (x * y)}_{rB}.$$

Compiling this instance yields the following *Elpi* rule:

```
pi L R PL PR\
  tc {{Add (lp:L * lp:R)}} {{addNat lp:L lp:R lp:PL lp:PR}} :-
    tc {{Add lp:L}} PL, tc {{Add lp:R}} PR.
```

This rule provides a proof for `Add (x * y)` provided that the two premises `Add x` and `Add y` can be solved.

To also solve type-class goals, we define an *Elpi* tactic containing the following rule:

```
solve (goal _ _ Ty _ _ as G) S :- tc Ty P, refine P G S.
```

Given a *Rocq* goal `G`, the tactic retrieves its type `Ty`, performs a backchaining step on `tc Ty P` to obtain a proof `P`, and refines the goal `G` with `P`. The refinement produces a list of subgoals `S`, which are returned to *Rocq*.

The full code, together with a working example, can be found in appendix A.

The implementation presented in this section illustrates the expressive power of the *Elpi* language: the core of the compiler fits in roughly eight lines of code, while a similar number of lines is required to complete the *Elpi* command for the compiler. The solver itself consists of a single rule. However, this simplified implementation has some limitations it is worth to point out.

The predicate `tc` does not expose the class arguments as *Elpi* arguments. Consequently, the *Rocq* modes (see section 2.1.1) declared on a class are not preserved in the *Elpi* solver. The compiler should therefore generate a specialized predicate for each class, with arguments and modes matching those of the class so that rule backchaining can be performed correctly. This problem is addressed in section 3.3.

Rule $r\forall$ does not account for the possibility that the quantified variable x is itself a premise of the class. In complex instances, x may have a class as its type and should therefore appear as a premise in the generated rule. This problem is addressed in section 3.4.

The compiler does not handle instances of the form $(c_1 \to c_2) \to c_3$, where the implementation of c_3 requires a proof of c_2 in a context extended with a local instance of c_1. Supporting such cases requires generating hereditary Harrop formulas and therefore tracking the polarity (see section 2.2.1) of the instance being compiled. This problem is addressed in section 3.4.

The `solve` rule of the solver ignores the context of the goal. As discussed in section 2.3.4, the first projection of a `goal` in *Rocq-Elpi* is its context, which may contain local instances that should be compiled and used during resolution. This problem is addressed in section 3.4.2.

3.3 Class compilation

When a class c is declared in *Rocq*, the compiler generates a corresponding *Elpi* predicate. If the class has n parameters, the generated predicate has $n + 1$ arguments: the first n arguments correspond to the parameters of c, while the final argument represents the instance witnessing the class in the current rule.

From the perspective of the *Elpi* solver, two aspects of a class declaration in *Rocq* are particularly relevant: the global reference identifying the class and the modes associated with its parameters. In *Rocq*, users may explicitly declare modes using the attribute `#[mode="..."]`. If no mode declaration is provided, the parameters of the class are considered outputs by default.

```
pred dft-class-mode term → list mode-type.
dft-class-mode (prod _ _ B) [pr out "term" | L] :- !,
  pi x\ dft-class-mode (B x) L.
dft-class-mode _ [pr out "term"].

pred str→mode string → mode-type.
str→mode "-" (pr out "term").
str→mode "+" (pr in "term").
str→mode "!" (pr in "term").

pred str→modes gref, string → list mode-type.
str→modes C "" M :- !, dft-class-mode {coq.env.typeof C} M.
str→modes _ S  M' :-
  map {rex.split " " S} str→mode M,
  append M [pr out "term"] M'.

pred add-class-pred gref, string →.
add-class-pred C S :-
  gref→pred-name C N,
  str→modes C S M,
  coq.elpi.add-predicate "tc.db" _ N M.
```

Figure 3.3: Class compiler

The compiler responsible for generating the corresponding *Elpi* predicate is shown in figure 3.3.

The entry point of the compiler is the predicate `add-class-pred`, which takes two arguments: the global reference `C` of the class and a string `S` describing its modes. The string `S` encodes the modes using the three *Rocq* symbols `-`, `+`, `!`, separated by spaces. If the user does not provide an explicit mode declaration, `S` is the empty string.

Predicates can be created dynamically in *Elpi* using the API `coq.elpi.add-predicate`. This primitive takes four arguments: the database in which the predicate will be stored (in our case `tc.db`), the indexing strategy, the predicate name `N`, and a list `M` of values of type `mode-type`. Each element of `M` specifies the mode and the type of one argument of the predicate. The predicate name is constructed from the global reference of the class using the predicate `gref->pred-name`. The list of modes `M` is obtained by processing the string `S` with the predicate `str->modes`.

Two cases must be distinguished depending on whether the string `S` is empty. If `S` is empty, the compiler derives the modes from the type of the class. It retrieves the type of `C` and creates a pair `pr out "term"` for each quantification in that type. In the base case, an additional output argument corresponding to the instance is added, again represented by the pair `pr out "term"`.

As an example, consider the declaration of the class `Add` (cf. figure 3.1). When this class is declared, the *Elpi* class compiler is triggered and receives the *Rocq* symbol `indt «Add»` as input. Since no mode attribute is provided, the compiler receives the empty string.

From the symbol `Add` the compiler retrieves its type and determines the number of parameters. In this example, the HOAS representation of the type of `Add` in *Elpi* is `{{Type → Type}}` which indicates that the class has a single parameter.

From this information the compiler generates the predicate associated with the class:

```
pred tc-Add o:term, o:term.
```

If `S` is not empty, the user has provided explicit modes for the class. In this case, each *Rocq* mode is translated into the corresponding *Elpi* mode. This translation is performed by the predicate `str->mode`, which maps `-` to the *Elpi* output mode, and both `+` and `!` to the *Elpi* input mode.

3.3.1 *Rocq* and *Elpi* mode discrepancies

Our typeclass solver is not intended to replicate the *Rocq* solver. Instead, it aims to exploit, whenever possible, the features of the *Elpi* language. A natural claim is that mapping two *Rocq* mode to only one in *Elpi* is that the *Rocq* and *Elpi* type-class solvers will have different runtime behaviors.

We can note that there is no direct correspondence between *Rocq* modes and *Elpi* modes. While the *Rocq* mode `-` corresponds naturally to the *Elpi* output mode, neither `+` nor `!` has a precise counterpart among the *Elpi* input mode. Furthermore, a *Rocq* class may declare multiple mode specifications, whereas an *Elpi* predicate admits only a single mode declaration.

We recall that the *Rocq* modes `+` and `!` impose conditions on the shape of the argument: `+` requires a ground term, while `!` requires a term with a rigid applicative head. During the resolution phase, if a goal does not satisfy the mode constraint imposed by the class declaration, a failure immediately stops the instance search. In contrast, *Elpi* modes do not check the shape of the argument before backchain the query into the database, it, instead, modifies the unification strategy by using the `matching` procedure (see section 2.2.2).

In order to see the different mode behavior of the *Rocq* and *Elpi* modes, we rework the implementation of the `Add` type class.

```
#[mode="+"] Class Add T :=

Instance addNat : Add nat.
Instance addDummy T : Add T.
```

Consider a goal g = `Add ?X` where `?X` is a meta-variable (evar in *Rocq* jargon). With the *Rocq* mode system, this goal has no solution because the argument of the class is a non-ground term and therefore does not satisfy the mode constraint. In contrast, under the *Elpi* mode interpretation the instance `addDummy` is a valid solution.

The instance `addDummy` is considered as a catchall: the first argument of the `Add` class is a variable, i.e a pattern that can be used a on any goal for `Add`. It means that `addDummy` can be used as a solution for g.

3.4 Instance compilation

As explained in section 3.1, instance compilation is triggered immediately after an instance declaration. Figure 3.4 shows an instance compiler implemented in *Elpi* where we take into account the type of dependent quantification and the polarity of the instance being compiled is an essential ingridient for compiling deep hereditary Harrop formulas.

Similarly to figure 3.2, we define a `compile` predicate which calls the `comp` auxiliary function. `comp` this time takes one more argument which is a boolean and tells the polarity of the term being compiled.

```
pred build-rule bool, prop, list prop → prop.
build-rule tt Head Prems (Head :- Prems).
build-rule ff Head Prems (Prems => Head).

%          Pol    Inst   Ty     Args        Prems          Res
pred comp bool, term, term, list term, list prop → prop.
comp B I {{∀ x: lp:Ty, lp:(Bo x)}} Ag P (pi y\ R y) :- % r→
  is-class? Ty, !,
  pi x\
    comp {¬ B} x Ty [] [] (M x),
    comp B I (Bo x) [x|Ag] [M x | P] (R x).
comp B I {{∀ x, lp:(Bo x)}} Ag P (pi y\ R y) :- !,      % r∀
  pi x\ comp B I (Bo x) [x|Ag] P (R x).
comp B I Ty Ag P R :-                                   % rB
  safe-dest-app Ty C CAg,
  mk-app I {rev Ag} Proof,
  make-rule-head C {append CAg [Proof]} Head,
  build-rule B Head {rev P} R.

pred compile gref →.
compile G :- coq.env.typeof G Ty,
  comp tt (global G) Ty [] [] R,
  coq.elpi.accumulate _ "tc.db" (clause _ _ R).
```

Figure 3.4: Instance compiler

The compilation proceeds by analyzing the structure of the instance type. There are three relevant cases. Rules `r1` and `r2` handle quantifications, while rule `r3` is the base case that constructs the final *Elpi* rule from the information accumulated during the recursive traversal.

When the current type has the form $\forall$`x` : `Ty`, `Bo x`, the abstraction binds a variable of type `Ty` in the body `Bo`. The test `is-class? Ty` checks whether `Ty` has the shape $\forall x_1 \ldots x_n, T$ (possibly with zero quantifiers), where `T` is a term whose applicative head is a type class.

If this condition holds, we are compiling a conditional instance and rule `r→` is applied. The rule introduces a fresh nominal variable `x` representing both the abstracted variable in `Bo` and the local instance with type `Ty`. To construct the premise corresponding to `x`, the compiler recursively invokes `comp` on `x` while switching the polarity of the rule. This recursive call produces the premise `M`, which abstracts `x`. Compilation then continues on `Bo x`, while extending the accumulators with the new instance argument `x` and the new premise `M x`.

Rule `r∀` handles the case where the quantified type is not a type class. In this situation, the compiler behaves exactly as rule `r∀` in figure 3.2: it simply consumes under the abstraction, leaving the list of premises `P` unchanged and adding the variable `x` to the list of instance arguments.

As in figure 3.2, the base case of `comp` is reached when the current type `Ty` is not a quantification. In this case, `Ty` is a type class `C` applied to a list of arguments `CAg`. The compiler then constructs the final rule by building its head and premises.

To build the head `Head`, the compiler first constructs the proof term `Proof`, obtained by applying the instance `I` to its arguments `rev Ag`. The list `Ag` is stored in reverse order during traversal, so it is reversed before constructing the application. The predicate `make-rule-head` takes the

```
Inductive a := p | q | r.
Variable to_prop : a → Prop.

Inductive mlogic :=
  | atom : a → mlogic
  | and : mlogic → mlogic → mlogic
  | impl : mlogic → mlogic → mlogic.

Fixpoint interp (p: mlogic) : Prop :=
  match p with
  | atom a => to_prop a
  | impl a b => interp a → interp b
  | and a b => interp a /\ interp b
  end.

Class Provable (T : mlogic) := { proof : interp T }.

Instance Pand F1 F2 :
  Provable F1 → Provable F2 → Provable (and F1 F2).

Instance Pimpl F1 F2 :
  (Provable F1 → Provable F2) → Provable (impl F1 F2).
```

Figure 3.5: Example with simple logic

class name as its first argument in order to generate the predicate corresponding to the class. The arguments of this predicate are obtained by appending the proof term to the list of class arguments.

The premises of the clause are stored in the accumulator `P`. As with the instance arguments, they are reversed before constructing the final rule.

Finally, the predicate `build-rule` assembles the rule according to the current polarity. With positive polarity, it produces a standard clause `Head :- Prems`. With negative polarity, it produces the implication `Prems => Head`.

Given the instance in figure 3.1, this version of the compiler produces the following rules:

```
tc-Add {{nat}} {{addNat}}.
tc-Add {{R}} {{addR}}.
pi L \ pi R \ pi PL \ pi PR \
  tc-Add {{lp:L * lp:R}} {{addProd lp:L lp:R lp:PL lp:PR}} :-
    tc-Add L PL, tc-Add R PR.
```

These rules are similar to those produced by the compiler in section 3.2, except for the specialization of the predicate `tc-Add` for the `Add` class.

An interesting example of compilation, where polarities play a central role (and are incorrectly handled by the compiler in section 3.2), is shown in figure 3.5. The code illustrates an encoding of a fragment of propositional logic using the *Rocq* typeclass mechanism as a proof search engine.

We define an inductive type `mlogic` representing a restricted language of formulas consisting of atoms, conjunctions, and implications. The constructor `atom` corresponds to logic atoms, while `and` and `impl` correspond to conjunction and implication, respectively.

The function `interp` assigns a semantics to `mlogic` formulas by translating them into *Rocq* propositions. This interpretation is compositional: an atom evaluates to a proposition using the `to_prop` function, a conjunction is interpreted as a logical conjunction, and an implication as a function type.

Automatic reasoning is expressed via the typeclass `Provable T`, which packages a proof of `interp T`. This allows *Rocq*'s typeclass resolution mechanism to act as a proof search procedure.

The inference rules of the logic are encoded as typeclass instances. The instance `Pand` corresponds to conjunction introduction: if `F1` and `F2` are provable, then `and F1 F2` is also provable.

Similarly, the instance `Pimpl` captures implication introduction. It states that to prove `impl F1 F2`, it suffices to show that `F2` is provable under the assumption `F1`, represented as `fact F1`.

The instance `Pand` is compiled into:

```
pi L \ pi R \ pi PL \ pi PR \
  tc-Provable {{and lp:L lp:R}} {{Pand lp:L lp:R lp:PL lp:PR}} :-
    tc-Provable L PL, tc-Provable R PR.
```

This rule closely mirrors the rule for `addProd`, since both rely on the same search principle: constructing a proof requires recursively constructing proofs for the subcomponents (the arguments of `and`, respectively `*`).

The rule corresponding to the instance `Pimpl` is more subtle, since it involves a higher-order hypothesis. Its type is:

$$\forall \texttt{F1 F2},\underbrace{\big(\underbrace{\texttt{Provable (fact F1)}}_{positive} \to \underbrace{\texttt{Provable F2}}_{negative}\big)}_{negative} \to \underbrace{\texttt{Provable (impl F1 F2)}}_{positive}$$

The compiled rule is:

```
pi F B Q \
  tc-Provable {{impl lp:F lp:B}} {{Pimpl lp:F lp:B lp:Q}} :-
    pi p \ tc-Provable {{fact lp:F}} p => tc-Provable B {{lp:Q lp:p}}.
```

The structure of this rule matches the intended semantics: the head targets `impl F B`, while the body attempts to construct a proof of `B` in a context extended with the assumption `F`.

3.4.1 Rule Priorities

When declaring instances, the priority assigned to each rule plays a fundamental role in the performance of the resolution phase. In *Rocq*, every instance is associated with a priority at declaration time.

To support rule priorities, we extend the instance trigger so that it also propagates priority information to the *Elpi* compiler. To reproduce a similar behavior in *Elpi*, we exploit its grafting mechanism (see section 2.2.6), which allows rules in the database to be named and enables new rules to be inserted before or after existing ones.

```
pred get-ITy prop → term, term.
get-ITy (decl I _ Ty) I Ty.
get-ITy (def I _ Ty _) I Ty.

pred compile-ctx prop → prop.
compile-ctx C R :- get-ITy C I Ty, is-class? Ty, comp tt I Ty [] [] R.

solve (goal C _ Ty _ _ as G) S :-
  map-filter C compile-ctx H,
  comp ff P Ty [] H R, R,
  refine P G S.
```

Figure 3.6: Goal compiler

In the *Elpi* compiler, we predeclare a fixed number of hooks, each associated with an integer n representing a grafting position. We account for this mechanism by modifying the `compile` rule in figure 3.4 so that it takes this integer as input. When a rule is added to the database, we explicitly specify that it must be grafted `:before` the hook named n.

3.4.2 Goal Encoding

With the database compilation in place, we now define the tactic used to solve type-class goals. The implementation of the `solve` predicate is shown in figure 3.6.

A *Rocq* goal in *Elpi* contains, among other information, the context `C` in which the goal is defined and a term `Ty` representing the type to be solved. The first step is to traverse the context, select the hypotheses that correspond to type classes, and compile them using the `comp` procedure. This produces a list of local rules `H`, which are added to the program during resolution.

Next, we compile the goal type `Ty`. As in hereditary Harrop formulas, a goal corresponds to an atom in negative position. Therefore, the compilation is performed with polarity `ff`. Concretely,

```
comp ff P Ty [] H R
```

constructs an atom in negative position. Here, `P` is a unification variable representing the proof term, `Ty` is the goal to be compiled, the list of instance parameters is empty, and `H` provides the premises extracted from the context. The result of the compilation is an atom `R` of the form `H => Q`, where `Q` is a query corresponding to a type class whose proof argument is `P`.

Evaluating `R` instantiates the proof term `P`, which is then used to refine the original *Rocq* goal.

3.4.3 User-Defined Rules

In addition to the automatic compilation of rules, users can extend the database with custom rules to implement specialized or more efficient resolution strategies in specific situations.

Using the `\elpi Accumulate` command, users can define ad hoc rules, such as the one shown below:

```
tc-Add {{lp:A * lp:A}} P :-
  P = {{let x := lp:A in let p := lp:Pa in addProd x x p p}},
  tc-Add A Pa.
```

This rule matches products whose two components are identical. When searching for an instance of the class `Add` applied to a type `A`, both components of the pair yield the same result, so the search is performed only once. Moreover, the resulting proof term shares memory between subproofs and their types, improving space and time efficiency.

In the simple rule presented above, we illustrate the expressive power of meta-programming in *Elpi*. Since *Elpi* is a Turing-complete language, users can arbitrarily customize the proof search by defining their own rules. In particular, there are no restrictions on these extensions: users are free to implement any custom rule that suits their needs.

Extending the database with ad-hoc rules is possible in the *Rocq* system thanks to the **`Hint`** system, that allows to tweak somehow the search engine, for example the **`Hint`** `Extern` command allows to apply a list of tactics on goal matching a particular pattern. This approach, however, asks the user to write ad-hoc rules in a language which is not the same of the *Rocq* solver, and which, is some case as not the power of expressivness of *Elpi*.

3.5 Experimental results

3.5.1 Benchmarks

We evaluate our solver on existing libraries, in particular *tlc* [7] and *stdpp* [31]. Both libraries use type-class inference as a main implementation tool. Studying them helps us understand the practical needs of *Rocq* users. This can reveal limitations of our implementation and suggest directions for future work. An important goal of this evaluation is also to compare our solver with the native *Rocq* solver.

As a first benchmark, we consider a set of classes and instances from *stdpp*. We focus on the `Inj` type class, defined as follows:

```
Class Inj A B (f : A → B) :=
  inj x y : f x = f y → x = y.
```

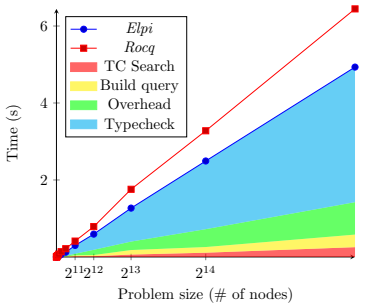
The type class `Inj` expresses that a function is injective. A function f is injective if, for all x and y, $f\ x = f\ y$ implies $x = y$.

The library provides 13 instances of this class. These include the identity function (`id`) and the constructors `inl` and `inr`, which are the injections of a sum type. In this benchmark, we focus on function composition. The composition (written $\circ$) of two functions `f` and `g` is injective if both functions are injective.

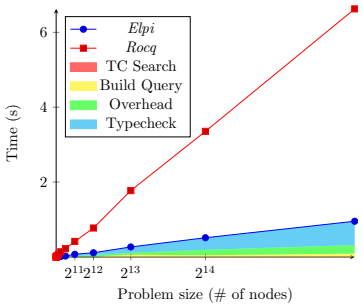
To evaluate performance, we measure the time needed by the instance solver to solve goals of increasing size. This helps us understand where time is spent and compare *Elpi* with *Rocq*. The benchmark goals are generated by the following function. It builds a balanced tree where internal nodes are compositions and leaves are a fixed injective function f:

$$g_n = \begin{cases} f & \text{if } n = 0, \\ g_{n-1} \circ g_{n-1} & \text{if } n > 0. \end{cases}$$

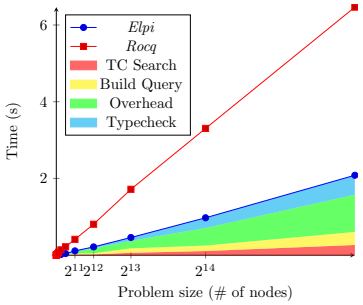
For a given n, we submit the goal `Inj eq eq` g_n to the solver. The size of this goal grows exponentially with n. Figure 3.7a shows the resolution time for n from 0 to 15. The x-axis is the number of nodes (logarithmic scale), and the y-axis is the time in seconds. The plot compares *Rocq* with a detailed analysis of the *Elpi* solver.



(a) With typechecking



(b) With memory sharing



(c) Without typechecking

Figure 3.7: Benchmarks of *Elpi* solver

For *Elpi*, we split the total time into four parts: type-class resolution, query construction (translation of *Rocq* terms into *Elpi*), communication between *Elpi* and *Rocq*, and typechecking of the final proof.

For *Rocq*, it is harder to separate these costs. Query construction and communication do not appear. However, instance search and typechecking are mixed together. In fact, partial typechecking is done after each instance selection. For this reason, we cannot easily measure them separately.

The results in figure 3.7a show that instance search is very fast. For goals with 2^{15} nodes, it takes about 0.25 seconds out of a total of 4.9 seconds. On the other hand, query construction and communication grow with the size of the goal. This cost comes from the interaction between *Rocq* and *Elpi* and does not depend on the *Elpi* interpreter itself.

The main cost is typechecking the proof term. It represents about 77% of the total time. We consider two ways to reduce this cost. First, we can introduce ad hoc rules for the specific problem. Second, we can skip typechecking of the produced proof.

The first approach also produces smaller proof terms, which are faster to check. As explained in section 3.4.3, memory sharing can reduce the complexity from exponential to linear. This effect is shown in figure 3.7b: smaller terms give faster typechecking and faster translation from *Elpi* to *Rocq*. However, this solution is not very portable, since it depends on the specific goal.

The second approach, i.e. skipping typechecking, reduces the time of the proof refinement to the goal by about a factor of 5, as shown in figure 3.7c. In most cases, this does not cause problems. The proof term should be correct, because the rules in the database are compiled in a context where they already typecheck. The main problem comes from universe constraints.

In the *Elpi* solver, universes are removed from the compiled clauses to simplify unification. For example, the *Rocq* term `Type@{i}` corresponds to the HOAS term `sort (typ i)`. During compilation, the level i is replaced by a unification variable. This allows matching terms at different levels at runtime.

However, removing universes from the database can introduce inconsistencies or may forget to state some universe constraint. These are usually detected during typechecking. A possible solution, which we do not implement, is to add universe constraints explicitly in the compiled rules. In this way, part of typechecking would be done during the search. This approach is similar to the strategy used by the *Rocq* solver.

3.5.2 Modes: *Elpi* vs *Rocq* in *stdpp*

We previously discussed discrepancies between *Elpi* and *Rocq* modes in section 3.3.1. Here, we highlight a family of implementation choices in the *stdpp* library that make the *Elpi* mode system inadequate. As a running example, consider the following class.

```
#[mode="- !"] Class Union A M :=
  union: (A → A → option A) → M → M → M.
```

The class is parametrized by two types, `A` and `M`, representing respectively the type of keys and the type of collections. The mode declaration requires `M` to have a rigid applicative head, while no constraint is imposed on `A`. The class provides the overloadable symbol `union`, which computes the union of two collections. In case of ambiguity, a higher-order function f is used to select between conflicting elements. A typical example arises with dictionaries: when a key appears in both inputs, f determines which value to retain.

We now consider an instance of `Union` from *stdpp*, together with a lemma that relies on it.

```
Instance ounion {A} : Union A (option A) := ...
Lemma union_Nr A (f: A → A → option A) mx : union f mx None = mx.
```

In this instance, `union` behaves as follows: it returns `None` if both arguments are `None`; it returns `f x y` if the arguments are `Some x` and `Some y`; otherwise, it returns the non-`None` argument.

In the lemma, an implicit type-class constraint must be resolved. The goal is `Union A (option ?X)`, where `?X` is a unification variable corresponding to the implicit argument of the `None` constructor. At this point, the typechecker lacks sufficient information to instantiate `?X`. The expected behavior is that type-class resolution both selects an instance and determines the value of `?X`.

In *Rocq*, the goal is solved using the instance `ounion`. In *Elpi*, however, the same instance cannot be applied because of mode restrictions. The input mode is declared on the second argument of the class, in particular, `matching` the query `tc-Union {{A}} {{option lp:X}}` against the clause head for `ounion` fails, since *Elpi* does not assign variables in input positions during matching.

Another subtle point arises when additional instances are present. If, before the lemma `union_None_r`, the database is extended with the following instance,

```
Instance oounion {A} : Union A (option (option A)).
```

then, under *Rocq* modes, it becomes unclear which instance is selected to solve the goal. Indeed, both instances match the constraint `Union A (option ?X)`. Depending on their priorities, `?X`

may be instantiated differently. As a result, developers may be unaware of which instance has been chosen, potentially leading to type errors later in the development.

In contrast, in *Elpi*, even with the additional instance `oounion`, type-class resolution still fails to make progress. In particular, the query `tc-Union {{A}} {{option lp:X}}` has no solution. To overcome this limitation, the type of the argument `M` must be provided explicitly in the lemma.

```
Lemma union_Nr A (f : A → A → option A) mx : union f mx (@None A) = mx.
```

3.5.3 Unification problems

As explained in section 3.1, the *Elpi* solver can be used as an alternative – not a replacement – to the *Rocq* solver. In a development, one can enable or disable either solver depending on the needs.

When running the *Elpi* solver on existing libraries, one may observe failures in situations where the *Rocq* solver succeeds, and vice versa, although the latter case is rare. In our setting, we focused on making the target library compile with the *Elpi* solver. Note that in this library, all type-class problems are solvable with the *Rocq* solver.

When a type-class resolution fails in *Elpi*, the cause can be investigated by inspecting the execution traces of both *Elpi* (using the *Elpi* trace browser [57, Sec. 3.5]) and *Rocq* (using the `Set Typeclasses Debug` option). Our analysis shows that most discrepancies arise from unification issues. We distinguish two main categories encountered in our study: failures related to $\eta\beta$-reduction, and failures related to δ-reduction (see [65]).

A typical $\eta\beta$-unification example is the term `fun x y => ?F y x` unified with `map`*, where `?F` is a unification variable. In *Rocq*, this succeeds by instantiating `?F` with `fun x y => map y x`. In contrast, *Elpi* represents these terms as HOAS expressions, namely `fun _ _ x\ fun _ _ y\ app[F, y, x]` and `global (const «map»)`. Since these terms have different rigid heads, unification fails.

This behavior is expected from the *Elpi* unification algorithm, but it is problematic in the context of instance resolution, where equivalent problems – expressed differently – should be solvable. For example, using *Elpi*-style λ-abstraction and application, the term `(x\ y\ F y x)` unifies with `map`, assigning `F` to `x\ y\ map y x`. To address this, we extend the instance compiler to detect potential $\eta\beta$-unification issues. Such terms are replaced with a fresh unification variable v, simplifying unification in *Elpi*. The variable v is then linked back to the original *Rocq* term, so that instantiations in *Elpi* are reflected in *Rocq*. This approach led to our contribution presented in [16], described in detail in chapter 4, which provides an automatic solution to $\eta\beta$-unification issues.

A concrete example of $\eta\beta$-unification failure in the *Elpi* solver arises from the compiled rule for the instance `Pimpl`, introduced at the end of section 3.4. Consider the goal `Provable (impl a (fact a))` for some proposition `a`. This goal has the solution `Pimp a (fact a) (fun x => x)`. However, the compiled rule fails to find this solution due to $\eta\beta$-unification.

After applying the rule for `Pimpl`, the sub-query becomes

```
tc-Provable {{fact a}} {{lp:Q lp:p}}
```

*The standard list map function

in a context extended with the hypothesis h = `tc-Provable {{fact a}} p`. Solving the subquery requires unifying it with h. However, the HOAS term corresponding to `{{lp:Q lp:p}}` is `app[Q, p]`, which cannot be unified with `p`, whose HOAS representation is simply `p`.

The second class of issues concerns δ-unification, which is not handled by the *Elpi* unification mechanism. In *Rocq*, definitions can be unfolded during unification. For example, the function id unifies with `fun x => x` after unfolding its definition. In contrast, *Elpi*– and more generally *Prolog*-like systems – does not support definitional unfolding during unification.

An example of failure due to missing δ-unification is given by the following code:

```
Class Extens A := ...
Instance Extens_f1 A1 A2: Extens (∀ x1 : A1, A2 x1) := ...
Definition map A B := A -> option B.
```

The *Rocq* solver can solve the goal `Extens (map A B)` using the instance `Extens_f1`. However, *Elpi* fails because `map A B` does not syntactically match a term starting with a quantifier.

Providing a general and automatic solution for δ-unification in *Elpi* is non-trivial. In some cases, the absence of δ-unification is actually desirable, since unfolding definitions can make unification unpredictable and harder to debug. Moreover, it may introduce performance issues [65].

To address δ-unification issues, the user has two options. First, one can declare `map` as a notation for `A -> option B`, since notations are unfolded by the elaborator, creating no ambiguity in the unification algorithm. Alternatively, one can add an ad-hoc rule to the solver:

```
\elpi Accumulate TC.Solver lp:{{
  tc-Extens {{map lp:A lp:B}} R :-
    tc-Extens {{lp:A -> option lp:B}} R.
}}.
```

This rule manually implements unfolding: when the first argument of `Extens` is `map`, the solver recursively processes the unfolded definition.

3.6 Discussion

The previous sections highlighted the flexibility of the *Elpi* language and its suitability for compiling *Rocq* terms into *Elpi* rules. In particular, *Elpi* provides a natural framework for proof automation: its *Prolog*-like search engine acts as an inference mechanism, provided that the program contains the derivation rules required to solve a given query. Although the type-class engine in *Rocq* exhibits a similar behavior, our goal was not to faithfully reproduce the *Rocq* solver. Instead, we aimed to leverage the distinctive features of *Elpi* whenever possible.

A key advantage inherited from *Elpi* is the control the user have over the search process. This control is achieved through several mechanisms, including rule priorities, predicate modes, the `unify`/`matching` unification algorithm, and the `Cut` operator. All these aspects have been discussed in previous sections, with the exception of the `Cut` operator.

We have deliberately postponed the discussion of `Cut`, as it has no direct counterpart in the *Rocq* type-class solver. As explained in section 2.2.2, the `Cut` operator allows the user to commit to a solution once certain conditions (e.g., rule guards) are satisfied. To fully exploit this feature, the *Rocq* language would require a syntax extension capable of indicating where `Cut` should

be introduced during compilation to *Elpi* rules. Discussions with other researchers suggest that `Cut` may provide an effective approach to addressing a well-known issue in type-class resolution: determinacy.

In logic programming, determinacy is the property that a predicate produces at most one solution for any given call. The `Cut` operator is a central mechanism for enforcing this property. Transposed to type-class resolution, determinacy would ensure that a class declared as deterministic remains so, regardless of the rules added to the database.

In *Rocq*, the flag `Typeclasses Unique Solutions` enforces a related behavior: it computes all possible solutions to a type-class query and returns a result only if exactly one solution exists; otherwise, it raises an error. However, as noted in the *Rocq* manual [49], this approach is computationally expensive, since it requires exploring the entire search tree.

In contrast, determinacy has been extensively studied in logic programming (e.g., [12, 35, 28]) and can be verified statically using abstract interpretation. This approach incurs no runtime cost during goal execution. Moreover, since determinacy is a general property of logic programs rather than being specific to type-class resolution, we extend *Elpi* with a determinacy checker implemented at the compiler level. As a result, all applications built on *Elpi*, including the type-class solver, can benefit from this analysis.

We presented this work in [17], where we describe the implementation of a determinacy checker for *Elpi* that accounts for both the `Cut` operator and higher-order features such as local rule insertion. A detailed discussion is provided in chapter 5. Finally, we formalize the first-order fragment of the checker within *Elpi* in the *Rocq* system; this formalization is described in chapter 6.

CHAPTER 4

$\eta\beta$-unification support for meta-programs

In the previous chapter, and in particular in section 3.5.3, we have talked about the unification issues that we incounter in the *Elpi* class solver when unifying certain *Rocq* terms. Unification up to δ is not possible in *Elpi*, but the $\eta\beta$-conversion rules are part of the *Elpi* unification algorithm. In this chapter we propose a framework allowing to ease the unification of the HOAS representation of the object language terms in the meta language. We take from our paper [16] the sentence which better describe the aim of this chapter: *how to reuse the higher-order unification procedure of the meta language in order to simulate a higher-order logic program for the object language*.

The example we have given in section 3.5.3 providing a source of $\eta\beta$-unification problem is due to the instance `Pimpl`.

```
tc-Provable {{impl lp:F lp:B}} {{Pimpl lp:F lp:B lp:Q}} :-
  pi p \ tc-Provable {{fact lp:F}} p => tc-Provable B {{lp:Q lp:p}}.
```

The variable `Q` in *Rocq* is an object living in the function space, while in the HOAS representation of *Elpi* `Q` is a first-order variable of type `term`. As shown in section 3.5.3, the *Rocq* statement `Provable (impl a (fact a))` can be solved by the *Rocq* type-class solver using the instance `Pimpl`. On the contrary, *Elpi* fails: the rule above would be used on the query `tc-Provable {{impl a (fact a)}} S` but the sub-query `tc-Provable {{fact a}} {{lp:(Q p)}}` will fails since not unifying with the local rule `tc-Provable {{fact a}} p`.

The root cause of the problems we outlined in this example is a subtle mismatch between the equational theories of the meta language and the object language, which in turn makes the unification procedures of the meta language weak. The equational theory of the meta-language *Elpi* encompasses $\beta\eta$-equivalence and its unification procedure can solve higher-order problems in the pattern fragment. Although the equational theory of *Rocq* is much richer, for efficiency and predictability reasons automatic proof search procedures typically employ a unification procedure that only captures a $\beta\eta$-equivalence and only operates in the pattern fragment.

A more sophisticated encoding of *Rocq* terms that, on a first approximation, would reshape the compiled rule for `Pimpl` as follows:

```
tc-Provable {{impl lp:F lp:B}} {{Pimpl lp:F lp:B lp:Q}} :-
  link Qm Q, % relating the local variable Qm with Q
  pi p \ tc-Provable {{fact lp:F}} p => tc-Provable B (Qm p).
```

```
kind to type.
type o-app list to -> to.
type o-lam (to -> to) -> to.
type o-con string -> to.
type o-uva addr -> to.
```

```
kind tm  type.
type app list tm -> tm.
type lam (tm -> tm) -> tm.
type con string -> tm.
type uva addr -> list tm -> tm.
```

Figure 4.1: The $\mathcal{O}$ and $\mathcal{M}$ languages

In the rule body, we have added a new higher-order *Elpi* variable `Qm` which has the type `term → term`. In the recursive call to `tc-Provable` we pass as argument the variable `Qm` applied to `p`, allowing the the unification problem `Qm p = p` to have a solution, namely, `Qm = x\ x`.

The role of the `link` predicate is to relate the *Elpi* variable `Qm` back to the *Rocq* variable `Q`. For instance, when `Qm` is assigned, then `Q` is assigned to the HOAS term `fun x => x`. With this encoding, the query `tc-Provable {{imp a (fact a)}} S` has a solution.

The general idea of this compilation is to identify at compilation time all the subterm that could create unification problems up to $\beta\eta$. These subterms t are replaced by unification variables v, to ease unification in the meta-langauge and an additional premise is added to relate t to v.

The idea of our compilation strategy is presented via two languages called called $\mathcal{O}$ for object language and $\mathcal{M}$ for the meta language. We detail the encoding of $\mathcal{O}$ in $\mathcal{M}$, and we show that the higher-order unification procedure of the meta language $\simeq_m$ can be used to simulate a higher-order unification procedure $\simeq_o$ for the object language that features $\beta\eta$-conversion.

Section 4.1 states the problem formally and gives the intuition behind our solution; section 4.2 sets up a basic simulation of first-order logic programs, section 4.3 and section 4.4 extend it to higher-order logic programs in the pattern fragment while section 4.6 goes beyond the pattern fragment. Section 4.7 discusses the implementation in *Elpi*.

4.1 Problem statement and solution

Even if we encountered the problem working on *Rocq*'s Calculus of Inductive Constructions, we devise a minimal setting to ease its study. In this setting, we have a language $\mathcal{O}$ with a rich equational theory and a $\mathcal{M}$ language with a simpler one.

Our encoding of $\mathcal{O}$ and $\mathcal{M}$ is given in the *Elpi* language. In figure 4.1, we provide in the syntax of the terms of $\mathcal{O}$, called `to` and the terms of $\mathcal{M}$, called `tm`. As claimed before, our framework is general, and therefore not proper to *Rocq* and *Elpi*. In our case, `to` corresponds to the `term` type of *Rocq* terms in *Elpi* and `tm` corresponds to the constructs of the λ-calculus of *Elpi*.

The only difference between the two term representations is the encoding of variables. Variables contain a pointer to a memory address. In $\mathcal{O}$, the variables are first-order, i.e. they have no term in their scope, while in $\mathcal{M}$, the variables have a scope which is represented as a list of term. We use this encoding, since we implement and test the full unification strategy in an *Elpi* development available at https://github.com/FissoreD/ho-unif-for-free.

When we write $\mathcal{M}$ terms outside code blocks we follow the usual λ-calculus notation, reserving f, g, a, b for constants, x, y, z for bound variables and X, Y, Z, F, G, H for unification variables. However, we need to distinguish between the "application" of a unification variable to its scope and the application of a term to a list of arguments. We write the scope of unification

```
type (=o) to -> to -> o.
o-con X =o o-con X.
o-app A =o o-app B :- forall2 (=o) A B.
o-lam F =o o-lam G :- pi x\ x =o x => F x =o G x.        % r_λλ
o-uva N =o o-uva N.
o-lam F =o T :-                                          % r_ηl
  pi x\ beta T [x] (T' x), x =o x => F x =o T' x.
T =o o-lam F :-                                          % r_ηr
  pi x\ beta T [x] (T' x), x =o x => T' x =o F x.
o-app [o-lam X|L] =o T :- beta (o-lam X) L R, R =o T.    % r_βl
T =o o-app [o-lam X|L] :- beta (o-lam X) L R, T =o R.    % r_βr
```

Figure 4.2: $\mathcal{O}$ equality implementation

variables in subscript while we use juxtaposition for regular application. Here are few examples:

$f\,a$	`app [con "f", con "a"]`
$\lambda x.\lambda y.F_{xy}$	`lam x\ lam y\ uva F [x, y]`
$\lambda x.F_{x\,a}$	`lam x\ app [uva F [x], con "a"]`
$\lambda x.F_{x\,x}$	`lam x\ app [uva F [x], x]`

When it is clear from the context, we shall use the same syntax for $\mathcal{O}$ terms (although we never use subscripts for unification variables). We use s, s_1, ... for terms in $\mathcal{O}$ and t, t_1, ... for terms in $\mathcal{M}$.

4.1.1 Equational theories and unification

Term equality: $=_o$ and $=_m$ For both languages, we extend the equational theory over ground terms to the full language by adding reflexivity for unification variables, i.e. a unification variable is equal to itself but different to any other term. The implementation is given in figure 4.2.

The first four rules are common to both equalities and define the usual congruence over terms. Since we use an HOAS encoding, they also capture α-equivalence. In addition to that, $=_o$ has rules for η and β-equivalence.

```
type (=m) tm -> tm -> o.
con C =m  con C.
app A =m  app B :- forall2 (=m) A B.
lam F =m  lam G :- pi x\ x =m  x => F x =m  G x.
uva N A =m  uva N B :- forall2 (=m) A B.
```

Both procedures use implication to assume that the fresh nominal constants introduces by `pi` are equal to themselves.

The main point in showing these equality tests is to remark how weaker $=_m$ is, and to identify the four rules that need special treatment in the implementation of $\simeq_o$. For brevity, we omit the code of `beta`: it is sufficient to know that `beta F L R` computes in `R` the weak-head normal form of `o-app [F | L]`.

Substitution: ρs and σt We write $\sigma = \{ X \mapsto t \}$ for the substitution that assigns the term t to the variable X. We write σt for the application of the substitution to a term t, and $\sigma X = \{ \sigma t \mid t \in X \}$

when X is a set of terms. We write $\sigma \subseteq \sigma'$ when σ is more general than σ'. We shall use ρ for $\mathcal{O}$ substitutions, and σ for the $\mathcal{M}$ ones. For brevity, in this section, we consider the substitution for $\mathcal{O}$ and $\mathcal{M}$ identical. We defer to section 4.2.1 a more precise description pointing out their differences.

Term unification: $\simeq_o$ vs. $\simeq_m$ $\mathcal{M}$'s unification signature is:

```
type (≃m) tm -> tm -> subst -> subst -> o.
```

We write $\sigma t_1 \simeq_m \sigma t_2 \mapsto \sigma'$ when σt_1 and σt_2 unify with substitution σ'. Note that σ' is a refined (i.e. extended) version of σ; this is reflected by the signature above that relates two substitutions. We write $t_1 \simeq_m t_2 \mapsto \sigma'$ when the initial substitution σ is empty. We write $\mathcal{L}$ as the set of terms that are in the pattern-fragment, i.e. every unification variable is applied to a list of distinct names.

The specification of a "good" unification procedure restricted to a domain $\mathcal{D}$ is the following:

Proposition 4.1.1 (Good unification)**.** *A good unification $\simeq$ for equality $=$ in the domain $\mathcal{D}$ satisfies the following properties:*

$$\{ t_1,\ t_2 \} \subseteq \mathcal{D} \Rightarrow t_1 \simeq t_2 \mapsto \rho \Rightarrow \rho t_1 = \rho t_2 \tag{4.1}$$

$$\{ t_1,\ t_2 \} \subseteq \mathcal{D} \Rightarrow \rho t_1 = \rho t_2 \Rightarrow \exists \rho', t_1 \simeq t_2 \mapsto \rho' \wedge \rho' \subseteq \rho \tag{4.2}$$

The meta language of choice is expected to provide an implementation of $\simeq_m$ that is a good unification for $=_m$ in $\mathcal{L}$.

Even if we provide an implementation of the object-language unification $\simeq_o$ in section 4.2.6, our final goal is to simulate the execution of an entire logic-program.

4.1.2 The problem: logic-program simulation

We represent a logic program *run* in $\mathcal{O}$ as a sequence of n *steps*. Each step p consists in unifying two terms, $\mathbb{P}_{p_l}$ and $\mathbb{P}_{p_r}$, taken from the list of all unification problems $\mathbb{P}$. We write $\mathbb{P}_p = \{ \mathbb{P}_{p_l}, \mathbb{P}_{p_r} \}$ and $s \in \mathbb{P} \Leftrightarrow \exists p, s \in \mathbb{P}_p$. The composition of these steps starting from the empty substitution ρ_0 produces the final substitution ρ_n, which is the result of the logic program execution.

$$\mathrm{step}_o(\mathbb{P}, p, \rho) \mapsto \rho' \stackrel{def}{=\!=} \rho\mathbb{P}_{p_l} \simeq_o \rho\mathbb{P}_{p_r} \mapsto \rho'$$

$$\mathrm{run}_o(\mathbb{P}, n) \mapsto \rho_n \stackrel{def}{=\!=} \bigwedge_{p=1}^{n} \mathrm{step}_o(\mathbb{P}, p, \rho_{p-1}) \mapsto \rho_p$$

In order to simulate a logic program we start by compiling $\mathbb{P}$ to $\mathbb{Q}$, i.e. each $\mathcal{O}$-term $s \in \mathbb{P}$ is translated to a $\mathcal{M}$-term $t \in \mathbb{Q}$. We write this translation as $\langle s \rangle \mapsto (t, m, l)$. The implementation of the compiler is detailed in sections 4.2, 4.4 and 4.6, here we just point out that it additionally produces a variable map m and a list of links l. The variable map connects unification variables in $\mathcal{M}$ to variables in $\mathcal{O}$ and is used to "decompile" the substitution, that is written $\langle \sigma, m, l \rangle^{-1} \mapsto \rho$. Links are an accessory piece of information whose description is deferred to section 4.1.3.

Then we simulate each run in $\mathcal{O}$ with a run in $\mathcal{M}$ as follows:

$$\begin{aligned}
&\mathrm{step}_m(\mathbb{Q}, p, \sigma, \mathbb{L}) \mapsto (\sigma'', \mathbb{L}') \stackrel{def}{=} \\
&\quad \sigma\mathbb{Q}_{p_l} \simeq_m \sigma\mathbb{Q}_{p_r} \mapsto \sigma' \wedge \mathrm{progress}(\mathbb{L}, \sigma') \mapsto (\mathbb{L}', \sigma'') \\
&\mathrm{run}_m(\mathbb{P}, n) \mapsto \rho_n \stackrel{def}{=} \\
&\quad \mathbb{Q} \times \mathbb{M} \times \mathbb{L}_0 = \{(t, m, l) | s \in \mathbb{P}, \langle s \rangle \mapsto (t, m, l)\} \\
&\quad \textstyle\bigwedge_{p=1}^{n} \mathrm{step}_m(\mathbb{Q}, p, \sigma_{p-1}, \mathbb{L}_{p-1}) \mapsto (\sigma_p, \mathbb{L}_p) \\
&\quad \langle \sigma_n, \mathbb{M}, \mathbb{L}_n \rangle^{-1} \mapsto \rho_n
\end{aligned}$$

By analogy with $\mathbb{P}$, we write $\mathbb{Q}_{p_l}$ and $\mathbb{Q}_{p_r}$ for the two $\mathcal{M}$ terms being unified at step p, and we write $\mathbb{Q}_p$ for the set $\{ \mathbb{Q}_{p_l}, \mathbb{Q}_{p_r} \}$.
The step_m procedure is made of two sub-steps: a call to the meta-language unification and a check for $\mathrm{progress}$ on the set of links, that intuitively will compensate for the weaker equational theory honored by $\simeq_m$. Procedure run_m compiles all terms in $\mathbb{P}$, then executes each step, and finally decompiles the solution. We claim:

Proposition 4.1.2 (Run equivalence)**.** $\forall \mathbb{P}, \forall n$, *if* $\mathbb{P} \subseteq \mathcal{L}$

$$\mathrm{run}_o(\mathbb{P}, n) \mapsto \rho \wedge \mathrm{run}_m(\mathbb{P}, n) \mapsto \rho' \Rightarrow \forall s \in \mathbb{P}, \rho s =_o \rho' s$$

That is, the two executions give equivalent results if all terms in $\mathbb{P}$ are in the pattern fragment. Moreover:

Proposition 4.1.3 (Simulation fidelity)**.** *In the context of* run_o *and* run_m*, if* $\mathbb{P} \subseteq \mathcal{L}$ *we have that* $\forall p \in 1 \ldots n$,

$$\mathrm{step}_o(\mathbb{P}, p, \rho_{p-1}) \mapsto \rho_p \Leftrightarrow \mathrm{step}_m(\mathbb{Q}, p, \sigma_{p-1}, \mathbb{L}_{p-1}) \mapsto (\sigma_p, \mathbb{L}_p)$$

In particular, this property guarantees that a *failure* in the $\mathcal{O}$ run is matched by a failure in $\mathcal{M}$ *at the same step*. We consider this property very important from a practical point of view since it guarantees that the execution traces are strongly related, and in turn, this enables a user to debug a logic program in $\mathcal{O}$ by looking at its execution trace in $\mathcal{M}$.

We also claim that run_m handles terms outside $\mathcal{L}$ in the following sense:

Proposition 4.1.4 (Fidelity recovery)**.** *In the context of* run_o *and* run_m*, if* $\rho_{p-1}\mathbb{P}_p \subseteq \mathcal{L}$ *(even if* $\mathbb{P}_p \notin \mathcal{L}$*) then*

$$\mathrm{step}_o(\mathbb{P}, p, \rho_{p-1}) \mapsto \rho_p \Leftrightarrow \mathrm{step}_m(\mathbb{Q}, p, \sigma_{p-1}, \mathbb{L}_{p-1}) \mapsto (\sigma_p, \mathbb{L}_p)$$

In other words, if the two terms involved in a step re-enter $\mathcal{L}$, then step_m and step_o are again related, even if $\mathbb{P} \notin \mathcal{L}$ and hence proposition 4.1.3 does not apply. Indeed, the main difference between proposition 4.1.3 and proposition 4.1.4 is that the assumption of the former is purely static, it can be checked upfront. When this assumption is not satisfied, one can still simulate a logic program and have guarantees of fidelity if, at run time, decidability of higher-order unification is restored.

This property has practical relevance since in many logic programming implementations, including *Elpi*, the order in which unification problems are tackled does matter. The simplest example is the sequence $F \simeq \lambda x.a$ and $F\,a \simeq a$: the second problem is not in $\mathcal{L}$ and has two unifiers, namely $\sigma_1 = \{ F \mapsto \lambda x.x \}$ and $\sigma_2 = \{ F \mapsto \lambda x.a \}$. The first problem picks σ_2, making the second problem re-enter $\mathcal{L}$.

4.1.3 The solution (in a nutshell)

A term s is compiled to a term t where every "problematic" subterm e is replaced by a fresh unification variable h with an accessory *link* that represents a suspended unification problem $h \simeq_m e$. As a result, $\simeq_m$ is "well behaved" on t, in the sense that it does not contradict $=_o$ as it would otherwise do on the "problematic" sub-terms.
We now define "problematic" and "well behaved" more formally. We use the $\diamond$ symbol since it stands for "possibly" in modal logic and all problematic terms are characterized by some uncertainty.

Definition 4.1.1 ($\diamond\beta$)**.** $\diamond\beta$ is the set of terms of the form $F\,x_1 \dots x_n$ such that $x_1 \dots x_n$ are distinct names (of bound variables).

An example of a $\diamond\beta$ term is the application $F\,x$. This term is problematic since the application node of its syntax tree cannot be used to justify a unification failure, i.e. by properly instantiating F, the term head constructor may become a λ, or a constant, or a name, or remain an application.

Definition 4.1.2 ($\diamond\eta$)**.** $\diamond\eta$ is the set of terms $\lambda x.s$ such that $\exists\rho, \rho(\lambda x.s)$ is an η-redex.

An example of a term s in $\diamond\eta$ is $\lambda x.\lambda y.F\,y\,x$ since the substitution $\rho = \{F \mapsto \lambda x.\lambda y.f\,y\,x\}$ makes $\rho s =_o \lambda x.\lambda y.f\,x\,y$, which is the eta-long form of f. This term is problematic since its leading λ abstraction cannot justify a unification failure against a constant f.

Definition 4.1.3 ($\diamond\mathcal{L}$)**.** $\diamond\mathcal{L}$ is the set of terms of the form $F\,t_1 \dots t_n$ such that $t_1 \dots t_n$ are not distinct names.

These terms are problematic for the very same reason terms in $\diamond\beta$ are, but they cannot be handled directly by the unification of the meta language, which is only required to handle terms in $\mathcal{L}$. Still, given $s \in \diamond\mathcal{L}$ there may exist a substitution ρ such that $\rho s \in \mathcal{L}$.

We write $\mathcal{P}(t)$ for the set of sub-terms of t, and we write $\mathcal{P}(X) = \bigcup_{t\in X} \mathcal{P}(t)$ when X is a set of terms.

Definition 4.1.4 (Well behaved set)**.** Given a set of terms X,

$$\mathcal{W}(X) \Leftrightarrow \forall t \in \mathcal{P}(X), t \notin (\diamond\beta \cup \diamond\eta \cup \diamond\mathcal{L})$$

We write $\mathcal{W}(t)$ as a short for $\mathcal{W}(\{t\})$. We claim our compiler validates the following properties:

Proposition 4.1.5 ($\mathcal{W}$-enforcing)**.** *Given a term s in $\mathcal{O}$,*

$$\langle s\rangle \mapsto (t, m, l) \Rightarrow \mathcal{W}(t)$$

Proposition 4.1.6 (compiler-useful)**.** *Given two terms s_1 and s_2, if $\exists\rho, \rho s_1 =_o \rho s_2$, then*

$$\langle s_i\rangle \mapsto (t_i, m_i, l_i) \text{ for } i \in \{1, 2\} \Rightarrow t_1 \simeq_m t_2 \mapsto \sigma$$

In other words the compiler outputs terms in $\mathcal{W}$ even if its input is not. And if two terms can be unified in the source language then their images can also be unified in the target language. Note that the property holds for any substitution, not necessarily a most general one: when applied to terms in $\mathcal{W}$ the meta-language unification $\simeq_m$ agrees with the object-language equality $=_o$. Note that a compiler that translate all terms to unification variables would satisfy this property, but wound certainly fail to verify proposition 4.1.3.

Proposition 4.1.7 ($\mathcal{W}$-preservation)**.** $\forall \mathbb{Q}, \forall \mathbb{L}, \forall p, \forall \sigma, \forall \sigma'$

$$\mathcal{W}(\sigma\mathbb{Q}) \wedge \sigma\mathbb{Q}_{p_l} \simeq_m \sigma\mathbb{Q}_{p_r} \mapsto \sigma' \Rightarrow \mathcal{W}(\sigma'\mathbb{Q})$$
$$\mathcal{W}(\sigma\mathbb{Q}) \wedge \mathrm{progress}(\mathbb{L}, \sigma) \mapsto (\_, \sigma') \Rightarrow \mathcal{W}(\sigma'\mathbb{Q})$$

Proposition 4.1.7 is key to proving propositions 4.1.2 and 4.1.3. Informally, it says that the problematic terms moved on the side by the compiler are not reintroduced by step_m, hence $\simeq_m$ can continue to operate properly. In sections 4.3, 4.4 and 4.6 we describe how to recognize terms in $\diamond\beta$, $\diamond\eta$ and $\diamond\mathcal{L}$ and how progress takes care of links and how it preserves $\mathcal{W}$ and ensures propositions 4.1.2 to 4.1.4.

We prove proposition 4.1.3 (SIMULATION FIDELITY) in sections 4.4 to 4.6 and proposition 4.1.2 (RUN EQUIVALENCE) only in section 4.3, where links are not present yet. In order to prove proposition 4.1.2 in the presence of links one needs to refine proposition 4.1.3 by relating the two substitutions ρ_p and σ_p: they are equivalent only by using decompilation to take into account terms in $\mathbb{L}$. Then, by induction on the number of steps, proposition 4.1.3 implies proposition 4.1.2. We discuss proposition 4.1.4 (FIDELITY RECOVERY) in section 4.6.

4.2 Basic compilation and simulation

4.2.1 Memory map ($\mathbb{M}$) and substitution (ρ and σ)

Unification variables are identified by a (unique) memory address. The memory and its associated operations are described below:

```
typeabbrev (mem A) (list (option A)).
type set? addr -> mem A -> A -> o.
type unset? addr -> mem A -> o.
type assign addr -> mem A -> A -> mem A -> o.
type new mem A -> addr -> mem A -> o.
```

If a memory cell is `none`, then the corresponding unification variable is not set. `assign` sets an unset cell to the given value, while `new` finds the first unused address and sets it to `none`.

Since each $\mathcal{M}$ unification variable occurs together with a scope, its assignment needs to be abstracted over it to enable the instantiation of the same assignment to different scopes. This is expressed by the `inctx` container, and in particular its `abs` binding constructor.

```
kind inctx type -> type.
type abs (tm -> inctx A) -> inctx A.
type val A -> inctx A.
typeabbrev assignment (inctx tm).
typeabbrev subst (mem assignment).
```

A solution to a $\mathcal{O}$ variable is, instead, a plain term, that is, `fsubst` is an abbreviation for `mem to`. The compiler establishes a mapping between variables of the two languages.

```
kind ovariable type.
type ov addr -> ovariable.
kind mvariable type.
type mv addr -> arity -> mvariable.
kind mapping type.
type (<->) ovariable -> mvariable -> mapping.
typeabbrev mmap (list mapping).
```

Each `mvariable` is stored in the mapping together with its arity (a number) so that the code of 4.3 below can preserve the following property:

Invariant 4.2.1 (Unification-variable arity)**.** *Each variable `A` in $\mathcal{M}$ has a (unique) arity `N` and each occurrence `(uva A L)` is such that `L` has length `N`.*

```
type m-alloc ovariable -> mvariable -> mmap -> mmap ->
  subst -> subst -> o.
m-alloc Ov Mv M M S S :- mem M (Ov <-> Mv), !.
m-alloc Ov Mv M [Ov <-> Mv|M] S S1 :- Mv = mv N _, new S N S1.
```

Figure 4.3: Malloc code

When a single `ovariable` occurs multiple times with different numbers of arguments, the compiler generates multiple mappings for it, on a first approximation, and then ensures the mapping is bijective by introducing η-`link`; this detail is discussed in section 4.5.

It is worth examining the code of `deref`, that applies the substitution to a $\mathcal{M}$ term. Notice how assignments are moved to the current scope, i.e. the `abs`-bound variables are renamed with the names in the scope of the unification variable occurrence.

```
type deref subst -> tm -> tm -> o.
deref _ (con C) (con C).
deref S (app A) (app B) :- map (deref S) A B.
deref S (lam F) (lam G) :-
  pi x\ deref S x x => deref S (F x) (G x).
deref S (uva N L) R :- set? N S A,
  move A L T, deref S T R.
deref S (uva N A) (uva N B) :- unset? N S,
  map (deref S) A B.
```

Note that `move` strongly relies on invariant 4.2.1: the length of the arguments of all occurrences of a unification variable is the same. Hence, they have the same simple type for the meta-level, and therefore the number of `abs` nodes in the assignment matches that length. This guarantees that `move` never fails.

```
type move assignment -> list tm -> tm -> o.
move (abs Bo) [H|L] R :- move (Bo H) L R.
move (val A)  [] A :- !.
```

We write $\sigma = \{\ A_{xy} \mapsto y\ \}$ for the assignment «`abs x\abs y\y`» and $\sigma = \{\ A \mapsto \lambda x.\lambda y.y\ \}$ for «`lam x\lam y\y`».

4.2.2 Links ($\mathbb{L}$)

As mentioned in section 4.1.3, the compiler replaces terms in $\diamond\eta$, $\diamond\beta$, and $\diamond\mathcal{L}$ with fresh variables linked to the problematic terms. Terms in $\diamond\beta$ do not need a link since $\mathcal{M}$ variables faithfully represent the problematic term thanks to their scope.

```
kind baselink type.
type link-eta  tm -> tm -> baselink.
type link-llam tm -> tm -> baselink.
typeabbrev links (list (inctx baselink)).
```

The right-hand side of a link, the problematic term, can occur under binders. To accommodate this situation, the compiler wraps `baselink` using the `inctx` container.

Invariant 4.2.2 (Link left hand side)**.** *The left-hand side of a suspended link is a variable.*

New links are suspended by construction. If the left-hand side is assigned during a step, then the link is considered for progress and possibly eliminated. This is discussed in section 4.4 and section 4.6.

In the examples we represent links as equations under a context. The equality sign is subscripted with the kind of `baselink`. For example $x \vdash A_x =_{\mathcal{L}} F_x\ a$ corresponds to:

```
abs x\ val (link-llam (uva A [x]) (app[uva F [x], con "a"]))
```

4.2.3 Compilation

The simple compiler described in this section serves as a base for the extensions in sections 4.3, 4.4 and 4.6. Its main task is to beta normalize the term and map one syntax tree to the other. In order to bring back the substitution from $\mathcal{M}$ to $\mathcal{O}$ the compiler builds a "memory map" connecting the $\mathcal{O}$ variables to the $\mathcal{M}$ ones.

The signature of the `comp` predicate below allows for the generation of links (suspended unification problems), which plays no role in this section but plays a major role in sections 4.3, 4.4 and 4.6.

```
type comp to -> tm -> mmap -> mmap -> links -> links ->
  subst -> subst -> o.
comp (o-con C) (con C) M M L L S S.
comp (o-lam F) (lam F1) M1 M2 L1 L2 S1 S2 :-          % r_λ
  comp-lam F F1 M1 M2 L1 L2 S1 S2.
comp (o-uva A) (uva B []) M1 M2 L L S1 S2 :-
  m-alloc (ov A) (mv B (arity z)) M1 M2 S1 S2.
comp (o-app A) (app A1) M1 M2 L1 L2 S1 S2 :-          % r_@
  fold6 comp A A1 M1 M2 L1 L2 S1 S2.
```

Figure 4.4: Comp FO

```
type compile to -> tm -> mmap -> mmap ->  links -> links ->
  subst -> subst -> o.
compile F G M1 M2 L1 L2 S1 S2 :-
  beta-normal F F', comp F' G M1 M2 L1 L2 S1 S2.
```

With respect to section 4.1, the signature also allows for updates to the substitution. The code above uses that possibility in order to allocate space for the variables, i.e. it sets their memory address to `none` (a detail not worth mentioning in the previous sections).

```
type comp-lam (to -> to) -> (tm -> tm) ->
  mmap -> mmap -> links -> links -> subst -> subst -> o.
comp-lam F G M1 M2 L1 L3 S1 S2 :-
  pi x y\ (pi M L S\ comp x y M M L L S S) =>
    comp (F x) (G y) M1 M2 L1 (L2 y) S1 S2,
  close-links L2 L3.
```

Figure 4.5: Comp FO

In the code above, the auxiliary predicate `fold6` folds a predicate with three accumulators (the memory map, the links and the substitution) over a lists to obtain a new list and the final values of the accumulators.

The auxiliary function `close-links` tests if the bound variable `v` really occurs in the link. If it does, the link is wrapped into an additional `abs` node binding `v`. In this way links generated deep inside the compiled terms can be moved outside their original context of binders.

```
type close-links (tm -> links) -> links -> o.
close-links (v\[X v|L v]) [abs X|R] :- close-links L R.
close-links (_\[]) [].
```

4.2.4 Execution

A step in $\mathcal{M}$ consists in unifying two terms and reconsidering all links for progress. If either of these tasks fails, we consider the entire step to fail. It is at this granularity that we can relate steps in the two languages.

```
type hstep tm -> tm -> links -> links -> subst -> subst -> o.
hstep T1 T2 L1 L2 S1 S3 :-
  (T1 ≃m T2) S1 S2,
  progress L1 L2 S2 S3.
```

Note that the infix notation `((A ≃m B) C D)` is syntactic sugar for `((≃m) A B C D)`.

Reconsidering links is a fixpoint process because the progress of a link can update the substitution, which may then enable another link to progress.

```
type progress links -> links -> subst -> subst -> o.
progress L L3 S S3 :-
  progress1 L L1 S S1,
  occur-check-links L1,
  scope-check L1 L2 S1 S2
  if (L = L2, S = S2) (L3 = L2, S3 = S2)
     (progress L2 L3 S2 S3).
```

Progress In the base compilation scheme, `progress1` is the identity function on both the links and the substitution, so the fixpoint trivially terminates. Sections 4.4 and 4.6 add rules to `progress1` and explain why they do not hinder termination.

Scope check The predicate `scope-check` replaces each link of the form $\Gamma \vdash X_{x_1 \ldots x_n} =_\eta t$ with a link $x_1 \ldots x_n \vdash X_{x_1 \ldots x_n} =_\eta t'$ whenever $\{x_1 \ldots x_n\} \subset \Gamma$. The term $t' = \lambda x.Y_{x_1 \ldots x_n\ x}$ is obtained after executing $\lambda x.Y_{x_1 \ldots x_n\ x} \simeq_m t$ using a fresh Y. This unification ensures that t' cannot contain variables in $\Gamma / \{x_1 \ldots x_n\}$, and if it fails then progress hence step_m fails. In turn this grants fidelity in the case where the lhs of an η`-link` is pruned of a name that occurs (rigidly) in t: links represent suspended unification problems *that may succeed*.

Occur check Since compilation moves problematic terms out of the sight of $\simeq_m$, that procedure can only perform a partial occur check. For example, the unification problem $X \simeq_m f\ Y$ cannot generate a cyclic substitution alone, but should be disallowed if $\mathbb{L}$ contains a link like $\vdash Y =_\eta$

$\lambda z.X_z$: we don't know yet if Y will feature a lambda in head position, but we surely know it contains X. The procedure `occur-check-links` is in charge of performing this check that is needed in order to guarantee proposition 4.1.3 (SIMULATION FIDELITY).

4.2.5 Substitution decompilation

Decompiling the substitution involves three steps.

First and foremost, problematic terms stored in $\mathbb{L}$ have to be moved back into the game: a suspended link must be turned into a valid assignment. This operation is possible thanks to invariant 4.2.2 (LINK LEFT HAND SIDE), and that no link causes an occur-check (4.2.4) and the fact that $\mathbb{L}$ is duplicate-free (see section 4.5).

The second step allocates in the memory of $\mathcal{O}$ new variables used to implement the pruning of higher-order variables. For example, $F\,x\,y = F\,x\,z$ requires allocating a variable G in order to express the assignment $F_{ab} \mapsto G_a$.

The final step is to decompile each assignment. The `val` node carries a term that is easy to decompile since $\mathbb{M}$ is a bijection. Since the `o-lam` node carries no additional information (other than the function body), each `abs` node can be decompiled to a `o-lam` one.

However, if the `o-lam` node was carrying more information, as is the case for *Rocq* where it holds the type of the bound variable, one would need to store this piece of information in the memory map, were we store the arity (that is a very simple function type).

Proposition 4.2.3 (Compilation round trip)**.** *If* $\langle s \rangle \mapsto (t, m, l)$ *and* $l \in \mathbb{L}$ *and* $m \in \mathbb{M}$ *and* $\sigma = \{A \mapsto t\}$ *and* $X \mapsto A^n \in \mathbb{M}$ *then* $\langle \sigma, \mathbb{M}, \mathbb{L} \rangle^{-1} \mapsto \rho$ *and* $\rho X =_o \rho s$.

We omit to sketch the proof of this property for brevity.

4.2.6 Definition of $\simeq_o$ and its properties

We already have all the ingredients to show the code of $\simeq_o$.

```
type (≃o) to -> to -> fsubst -> o.
(A ≃o B) F :-
  compile A A' [] M1 [] L1 [] S1,
  compile B B' M1 M2 L1 L2 S1 S2,
  hstep A' B' L2 L3 S2 S3,
  decompile M2 L3 S3 [] F.
```

So far the compiler is very basic. It does not really enforce that the terms passed to step_m are in $\mathcal{W}$, and indeed makes no use of the higher-order capabilities of the meta language (all generated variables have an empty scope). Still, we can prove that $\simeq_o$ is a good "first-order" unification algorithm if the input already happens to be in $\mathcal{W}$. Later, when the compiler will ensure proposition 4.1.5 ($\mathcal{W}$-ENFORCING), the proof will be refined to cover for the new cases.

Theorem 4.2.4 (Properties of $\simeq_o$ in $\mathcal{W}$)**.** *The implementation of* $\simeq_o$ *above is a good unification for* $=_o$ *(as per proposition 4.1.1) in the domain* $\mathcal{W}$.

Proof sketch.

In this setting, $=_m$ is as strong as $=_o$ on ground terms. What we have to show is that whenever two different $\mathcal{O}$ terms can be made equal by a substitution ρ, we can produce this ρ by finding a σ via $\simeq_m$ on the corresponding $\mathcal{M}$ terms and by decompiling it. If we look at the syntax of $\mathcal{O}$, the only interesting case is «`o-uva X` $\simeq_o$ `s`». In this case, after compilation, we have «`uva Y []` $\simeq_m$ `t`» that succeeds with $\sigma = \{Y \mapsto t\}$ and σ is decompiled to $\rho = \{X \mapsto s\}$ by proposition 4.2.3.

□

Theorem 4.2.5 (Fidelity in $\mathcal{W}$)**.** *Proposition 4.1.2 (RUN EQUIVALENCE) and proposition 4.1.3 (SIMULATION FIDELITY) hold if* $\mathcal{W}(\mathbb{P})$.

Proof sketch.

Since `progress1` is a no-op then step_o and step_m are the same. By theorem 4.2.4, $\simeq_m$ is equivalent to $\simeq_o$ in $\mathcal{W}$.

□

4.2.7 Notational conventions

In the following sections we use the following notation for input and output of the compiler. $\mathbb{P}$ represents the input unification problems in $\mathcal{O}$ while $\mathbb{Q}$ their corresponding compiled version with memory mapping $\mathbb{M}$ and links $\mathbb{L}$. For example:

$$\begin{array}{lllllllll}
\mathbb{P} = \{ & p_1 & \simeq_o & p_2 & & p_3 & \simeq_o & p_4 & \} \\
\mathbb{Q} = \{ & t_1 & \simeq_m & t_2 & & t_3 & \simeq_o & t_4 & \} \\
\mathbb{M} = \{ & X_1 \mapsto A_1^n & & & X_2 \mapsto A_2^m & & & & \} \\
\mathbb{L} = \{ & \Gamma \vdash a =_\eta b & & & & & & & \}
\end{array}$$

We index each sub-problem, sub-mapping, and sub-link with its position starting from 1 and counting from left to right, top to bottom. For example, $\mathbb{Q}_2$ corresponds to the $\mathcal{M}$ problem $t_3 \simeq_m t_4$.

As we deal with each category of problematic terms we weaken the assumptions of theorem 4.2.5 using the following definitions:

Definition 4.2.1 ($\mathcal{W}_\beta$)**.** $\mathcal{W}_\beta(X) \Leftrightarrow \forall t \in \mathcal{P}(X), t \notin (\Diamond\eta \cup \Diamond\mathcal{L})$

Definition 4.2.2 ($\mathcal{W}_{\beta\eta}$)**.** $\mathcal{W}_{\beta\eta}(X) \Leftrightarrow \forall t \in \mathcal{P}(X), t \notin \Diamond\mathcal{L}$

4.3 Handling of $\Diamond\beta$

The basic compiler given in the previous section is unable to make the following higher-order unification problem succeed.

$$\begin{array}{lllll}
\mathbb{P} = \{ & \lambda x.(f\ (X\ x)\ a) & \simeq_o & \lambda x.(f\ x\ a) & \} \\
\mathbb{Q} = \{ & \lambda x.(f\ (A\ x)\ a) & \simeq_m & \lambda x.(f\ x\ a) & \} \\
\mathbb{M} = \{ & X \mapsto A^0 & & & \}
\end{array}$$

The unification problem $\mathbb{Q}_1$ fails while trying to unify $A\ x$ and x, which is equivalent to «`app [uva A [], x]`» versus `x`. In order to exploit the higher-order unification algorithm of the meta language, we compile the $\mathcal{O}$ term $X\ x$ into the $\mathcal{M}$ term A_x.

4.3.1 Compilation and decompilation

We add the following rule before rule $r_@$ in 4.4, where `pattern-fragment` is a predicate checking if a list of terms is a list of distinct names.

```
comp (o-app [o-uva A|Ag]) (uva B Ag1) M1 M2 L L S1 S2 :-
  pattern-fragment Ag, !,
    fold6 comp Ag Ag1 M1 M1 L L S1 S1,
    len Ag Arity,
    m-alloc (ov A) (mv B (arity Arity)) M1 M2 S1 S2.
```

Note that compiling `Ag` cannot create new mappings nor links, since `Ag` is made of bound variables, and the hypothetical rule in 4.5 loaded by `comp-lam` grants this property.

Decompilation No change to `commit-link` since no link is added.

Progress No change to `progress` since no link is added.

Lemma 4.3.1 (Properties of $\simeq_o$ in $\mathcal{W}_\beta$)**.** *Given the updated compilation scheme, the $\simeq_o$ of section 4.2.6 is a good unification (as per proposition 4.1.1) for $=_o$ in the domain $\mathcal{W}_\beta$.*

Proof sketch.

If we look at the $\mathcal{O}$ terms, there is one more interesting case, namely `o-app[o-uva X|W]` $\simeq_o$ `s` when `W` are distinct names compiled to $\vec{w}$. In this case the $\mathcal{M}$ problem is $Y_{\vec{w}} \simeq_m t$ that succeeds with $\sigma = \{Y_{\vec{y}} \mapsto t[\vec{w}/\vec{y}]\}$, which in turn is decompiled to $\rho = \{Y \mapsto \lambda\vec{y}.s[\vec{w}/\vec{y}]\}$. Thanks to rule r_{β_l} in 4.2 we have $(\lambda\vec{y}.s[\vec{w}/\vec{y}])\ \vec{w} =_o s$.

□

Lemma 4.3.2 ($\mathcal{W}$-partial-enforcement)**.** $\forall s, \langle s\rangle \mapsto (t, m, l)$*, if* $\mathcal{W}_\beta(s)$ *then* $\mathcal{W}(t)$.

In other words the compiler eliminates all subterms that are in $\Diamond\beta$.

Theorem 4.3.3 (Fidelity in $\mathcal{W}_\beta$)**.** *Proposition 4.1.2 (*RUN EQUIVALENCE*) and proposition 4.1.3 (*SIMULATION FIDELITY*) hold if* $\mathcal{W}_\beta(\mathbb{P})$.

Proof sketch.

Thanks to lemma 4.3.2, $\simeq_m$ is as powerful as $\simeq_o$ in $\mathcal{W}_\beta$, as well as in $\mathcal{W}$ by theorem 4.2.5.

□

4.4 Handling of $\Diamond\eta$

A term $\lambda x.t\,x$ is said to be the η-redex of t if x does not occur free in t, and conversely we say that t is the η-contraction of $\lambda x.t\,x$. The equational theory of $\mathcal{O}$ identifies these terms, but the current compilation scheme does not, as shown by the following example: while $\lambda x.X\,x \simeq_o f$ does admit the solution $\rho = \{\ X \mapsto f\ \}$, the corresponding problem in $\mathbb{Q}$ does not.

$$\begin{array}{lrcl} \mathbb{P} = \{ & \lambda x.(X\ x) & \simeq_o & f \ \} \\ \mathbb{Q} = \{ & \lambda x.A_x & \simeq_m & f \ \} \\ \mathbb{M} = \{ & X \mapsto A^1 & & \ \} \end{array}$$

The reason is that «`lam x\ uva A [x]`» and «`con"f"`» start with different term constructors.

In order to guarantee proposition 4.1.2, we detect lambda abstractions that can disappear by η-contraction (section 4.4.1), and we modify the compiler so that it generates fresh unification variables in their place and moves the problematic term from $\mathbb{Q}$ to $\mathbb{L}$ (section 4.4.2). The compilation of the problem $\mathbb{P}$ above is refined to:

$$\begin{array}{lrcl} \mathbb{P} = \{ & \lambda x.(X\ x) & \simeq_o & f \ \} \\ \mathbb{Q} = \{ & A & \simeq_m & f \ \} \\ \mathbb{M} = \{ & X \mapsto B^1 & & \ \} \\ \mathbb{L} = \{ & \vdash A =_\eta \lambda x.B_x & & \ \} \end{array}$$

As per invariant 4.2.2 the η`-link` left-hand side is a variable while the right-hand side is a term in $\diamond\eta$ that has the following property:

Invariant 4.4.1 (η`-link` rhs)**.** *The rhs of any η`-link` has the shape $\lambda x.t$ and t is in $\mathcal{W}$.*

Each η`-link` is kept in the link store $\mathbb{L}$ during execution and is activated under some conditions. Activation is implemented in section 4.4.3 by extending the `progress1` predicate defined in section 4.2.4.

4.4.1 Detection of $\diamond\eta$

When compiling a term t, we need to determine if any subterm $s \in \mathcal{P}(t)$ that is of the form $\lambda x.r$ can be an η-redex, i.e. if there exists a substitution ρ such that $\rho(\lambda x.r) =_o s$. The detection of lambda abstractions that can "disappear" is not as trivial as it may seems. Here a few examples:

$$\begin{array}{lll} \lambda x.f\ (A\ x) & \in \diamond\eta & \rho = \{\ A \mapsto \lambda x.x\ \} \\ \lambda x.f\ (A\ x)\ x & \in \diamond\eta & \rho = \{\ A \mapsto \lambda x.a\ \} \\ \lambda x.f\ x\ (A\ x) & \notin \diamond\eta & \\ \lambda x.\lambda y.f\ (A\ x)\ (B\ y\ x) & \in \diamond\eta & \rho = \{\ A \mapsto \lambda x.x\ ;\ B \mapsto \lambda y.\lambda x.y\ \} \end{array}$$

The first two examples are easy, and show how a unification variable can expose or erase a variable in its scope, turning the resulting term into an η-redex or not.
The third example shows that when a variable occurs outside the scope of a unification variable, it cannot be erased and can hence prevent a term from being an η-redex.
The last example shows the recursive nature of the check we need to implement. The term starts with a spine of two lambdas, hence the whole term is in $\diamond\eta$ iff the inner term $\lambda y.f\ (A\ x)\ (B\ y\ x)$ is in $\diamond\eta$ itself. If it is, it could η-contract to $f\ (A\ x)$ making $\lambda x.f\ (A\ x)$ a potential η-redex.
We can now detail how $\diamond\eta$ terms are detected.

Definition 4.4.1 (*may-contract-to*)**.** A β-normal term s *may-contract-to* a name x if there exists a substitution ρ such that $\rho s =_o x$.

Lemma 4.4.2. *A β-normal term $s = \lambda x_1 \ldots x_n.t$* may-contract-to *$x$ only if one of the following three conditions holds:*

1. $n = 0$ *and* $t = x$*;*
2. t *is the application of* x *to a list of terms* l *and each* l_i may-contract-to x_i *(e.g.* $\lambda x_1 \dots x_n.x\ x_1 \dots x_n =_o x$*) ;*
3. t *is a unification variable with scope* W*, and for any* $v \in \{x, x_1 \dots x_n\}$*, there exists a* $w \in W$*, such that* w may-contract-to v *(if* $n = 0$ *this is equivalent to* $x \in W$*).*

Proof sketch.

Since our terms are in β-normal form the only rule that can play a role is r_{η_l} in 4.2, and note that that rule uses `beta` to add an argument to an application or to the scope of a variable. If the term s is not exactly x (case 1) it can only be an η-redex of x, or a unification variable that can be assigned to x, or a combination of both. If s begins with a lambda, then the lambda can only disappear by η contraction. In that case, the term t under the spine of binders $x_1 \dots x_n$ can either be x applied to terms that *may-contract-to* these variables (case 2), or a unification variable that can be assigned to the application $x\ x_1 \dots x_n$ (case 3).

□

Definition 4.4.2 (*occurs-rigidly*)**.** A name x *occurs-rigidly* in a β-normal term t, if $\forall \rho, x \in \mathcal{P}(\rho t)$

In other words, x *occurs-rigidly* in t if it occurs in t outside of the scope of a unification variable X; otherwise, an instantiation of X can make x disappears from t. Note that η-contracting t cannot make x disappear, since x is not a locally bound variable inside t.

We can now describe the implementation of $\diamond\eta$ detection:

Definition 4.4.3 (*maybe-eta*)**.** Given a β-normal term $s = \lambda x_1 \dots x_n.t$, *maybe-eta* s holds if any of the following holds:

1. t is a constant c or a name y applied to the arguments $l_1 \dots l_m$ such that $m \geq n$ and for every i such that $m - n < i \leq m$ the term l_i *may-contract-to* x_i, and no x_i *occurs-rigidly* in $l_1 \dots l_{m-n}\ y$;
2. t is a unification variable with scope W and for each x_i there exists a $w_j \in W$ such that w_j *may-contract-to* x_i.

Lemma 4.4.3 ($\diamond\eta$ detection)**.** *If* t *is a* β*-normal term and* $t \in \diamond\eta$ *then* maybe-eta t *holds.*

Proof sketch.

Follows from definition 4.4.2 and lemma 4.4.2

□

Remark that the converse of lemma 4.4.3 does not hold: there exists a term t satisfying the criteria (1) of definition 4.4.3 that is not in $\diamond\eta$, i.e. there exists no substitution ρ such that ρt is an η-redex. A simple counter example is $\lambda x.f\ (A\ x)\ (A\ x)$ since x does not *occur-rigidly* in the first argument of f, and the second argument of f *may-contract-to* x. In other words $A\ x$ may either use or discard x, but our analysis does not take into account that *the same term* cannot have two contrasting behaviors.

As we will see in the rest of this section, this is not a problem since it does not break proposition 4.1.2 nor proposition 4.1.3.

4.4.2 Compilation and decompilation

Compilation The following rule is inserted just before rule r_λ in 4.4.

```
comp (o-lam F) (uva A Scope) M1 M2 L1 L3 S1 S3 :-
  maybe-eta (o-lam F) [], !,
  alloc S1 A S2,
  comp-lam F F1 M1 M2 L1 L2 S2 S3,
  get-scope (lam F1) Scope,
  L3 = [val (link-eta (uva A Scope) (lam F1)) | L2].
```

Whenever `o-lam F` is detected to be in $\Diamond\eta$ it is compiled to `lam F1` and replaced by the fresh variable `A`. This variable sees all the names free in `lam F1` and is connected to `lam F1` via a η`-link`. Invariant 4.2.2 (LINK LEFT HAND SIDE) holds for this link. Moreover:

Corollary 4.4.4. *The rhs of any* η`-link` *has exactly one lambda abstraction, hence the rule above respects invariant 4.4.1 (*η`-LINK` RHS*).*

Proof sketch.

By contradiction, suppose that the rule above is applied and that the rhs of the link is $\lambda x.\lambda y.t$, where x and y occur in t. If *maybe-eta* $\lambda y.t$ holds then the recursive call to `comp` (made by `comp-lam`) must have put a fresh variable in its place, so this case is impossible. Otherwise, if *maybe-eta* $\lambda y.t$ does not hold, then also *maybe-eta* $\lambda x.\lambda y.t$ does not hold either, contradicting the assumption that the rule was applied.

□

Lemma 4.4.5 ($\mathcal{W}$-partial-enforcement)**.** $\forall s, \langle s \rangle \mapsto (t, m, l)$*, if* $\mathcal{W}_{\beta\eta}(s)$ *then* $\mathcal{W}(t)$.

Proof sketch.

By lemma 4.4.3

□

Decompilation The decompilation of a η`-link` is performed by unifying the lhs with the rhs. Note that this unification never fails, since the lhs is a flexible term not appearing in any other η`-link` (by definition 4.4.5).

4.4.3 Progress

η`-link`s are meant to delay the unification of "problematic" terms until we know for sure if their head lambdas can be η-contracted.

Definition 4.4.4 (*η-progress-lhs*)**.** A link $\Gamma \vdash X =_\eta t$ is removed from $\mathbb{L}$ when X becomes rigid. Let $y \in \Gamma$. There are two cases:

1. if $X = a$ or $X = y$ or $X = f\, a_1 \ldots a_n$ we unify the η-redex of X with t, that is we run $\lambda x.X\, x \simeq_m t$

2. if $X = \lambda x.s$ we run $X \simeq_m t$.

Definition 4.4.5 (η*-progress-deduplicate*)**.** A link $\Gamma \vdash X_{\vec{s}} =_\eta T$ is removed from $\mathbb{L}$ when another link $\Delta \vdash X_{\vec{r}} =_\eta T'$ is in $\mathbb{L}$. By invariant 4.2.1 the length of $\vec{s}$ and $\vec{r}$ is the same; hence we can move the term T' from Δ to Γ by renaming its bound variables, i.e. $T'' = T'[\vec{r}/\vec{s}]$. We then run $T \simeq_m T''$ (under the context Γ).

Lemma 4.4.6. *Let $\lambda x.t$ be the rhs of a η-`link`, then $\mathcal{W}(t)$, enforcing invariant 4.4.1.*

Proof sketch.

By lemma 4.4.5.

□

Lemma 4.4.7. *Given a η-`link` $\Gamma \vdash X =_\eta \lambda x.t$, the unification done by* η-progress-lhs *is between terms in $\mathcal{W}$.*

Proof sketch.

Let σ be a substitution such that $\mathcal{W}(\sigma X)$ (since $\simeq_m$ preserves $\mathcal{W}$). By invariant 4.4.1, we have $\mathcal{W}(t)$. If $\sigma X = \lambda x.r$, then r is unified with t and both terms are in $\mathcal{W}$. Otherwise, $\lambda x.X\ x$ is unified with $\lambda x.t$, and $X\ x$ and t are both in $\mathcal{W}$.

□

Lemma 4.4.8. *The unification done by* η-progress-deduplicate *is between terms in $\mathcal{W}$.*

Proof.

Given two η-`links` $\Gamma_1 \vdash X =_\eta \lambda x.t$ and $\Gamma_2 \vdash Y =_\eta \lambda x.t'$ the unification is performed between t and t'. By lemma 4.4.5 both terms are in $\mathcal{W}$.

□

Lemma 4.4.9. *The progress of η-`link` guarantees proposition 4.1.7 (*$\mathcal{W}$-PRESERVATION*)*

Proof sketch.

By lemmas 4.4.7 and 4.4.8, every unification performed by the activation of a η-`link` is between terms in $\mathcal{W}$.

□

Lemma 4.4.10. `progress` *terminates.*

Proof sketch.

Rules 4.4.4 and 4.4.5 remove one link from $\mathbb{L}$, hence they cannot be applied indefinitely. Moreover each rule only relies on terminating operations such as $\simeq_m$, η-contraction, η-redex, relocation (a recursive copy of a finite term).

□

Theorem 4.4.11 (Fidelity in $\mathcal{W}_{\beta\eta}$)**.** *Given a list of unification problems $\mathbb{P}$ such that $\mathcal{W}_{\beta\eta}(\mathbb{P})$, if the memory map is bijective then the introduction of η-`link` guarantees proposition 4.1.3 (SIMULATION FIDELITY).*

Proof sketch.

We want to prove that step_o succeeds iff step_m succeeds, where step_m is made by a unification step u and a link progression p. Without loss of generality we consider a $\mathcal{O}$ unification problem of the form $s_1 \simeq_o s_2$ where $s_1 \in \Diamond\eta$ and is compiled to a variable X with the accessory link $\Gamma \vdash X =_\eta \lambda x.t_1$. The unification u always succeeds, since X is a fresh variable, we only have to consider the link progression p. Two cases should be analyzed:

$s_2 \in \Diamond\eta$, in this case, the unification problem becomes $X \simeq_m Y$ with the extra link $\vdash Y =_\eta \lambda x.t_2$. After the unification of X and Y, *η-progress-deduplicate* is fired and, once the lambdas are crossed, t_1 and t_2 are unified. This unification succeeds iff $s_1 \simeq_o s_2$ succeeds by theorem 4.3.3.

$s_2 \notin \Diamond\eta$, in this case s_2 is compiled to t_2 and the unification step u assigns t_2 to X and triggers *η-progress-lhs*. If s_1 and s_2 are equal thanks to r_{η_l} in 4.2 then progression p unifies $\lambda x.t_1$ with the η-redex of t_2 namly $\lambda x.t_2\,x$, otherwise they are equal thanks to rule $r_{\lambda\lambda}$ in 4.2 and the link progress p unifies $\lambda x.t_1$ with t_2. Alternatively, if s_1 are different s_2 it cannot be because of the λ constructor in the head of t_1, since rule $r_{\lambda\lambda}$ in 4.2 or rule r_{η_l} in 4.2 move under it, eventually finding two different subterms s_1' and s_2'. By theorem 4.3.3, $\simeq_m$ fails on terms t_1' and t_2' even if *η-progress-lhs* η-expands t_2.

□

Corollary 4.4.12 (Properties of $\simeq_o$ in $\mathcal{W}_{\beta\eta}$)**.** *Given the updated compilation scheme, procedure $\simeq_o$ of section 4.2.6 is a good unification (as per proposition 4.1.1) for $=_o$ in the domain $\mathcal{W}_{\beta\eta}$.*

Example of *η-progress-lhs* The example at the beginning of section 4.4, once $\sigma = \{\ A \mapsto f\ \}$, triggers *$\eta$-progress-lhs* since the link becomes $\vdash f =_\eta \lambda x.B_x$ and the lhs is a constant. This rule runs $\lambda x.f\,x \simeq_m \lambda x.B_x$, resulting in $\sigma = \{\ A \mapsto f\ ;\ B_x \mapsto f\ \}$. Since X is mapped to B and B_x is decompiled into $\lambda x.f\,x$, by the definition of η-`link` decompilation σ is decompiled to $\rho = \{\ X \mapsto \lambda x.f\,x\ \}$.

4.5 Making $\mathbb{M}$ a bijection

We want to allow for partial application of functions. As a consequence the same unification variable X can be used multiple times with different arities. When it is the case the memory map $\mathbb{M}$ generated by the compiler is not a bijection. For example:

$$\begin{array}{lllllllll}
\mathbb{P} = \{ & \lambda x.(X\,x) & \simeq_o & f & X & \simeq_o & \lambda x.a & \} \\
\mathbb{Q} = \{ & A & \simeq_m & f & C & \simeq_m & \lambda x.a & \} \\
\mathbb{M} = \{ & X \mapsto C^0 & X \mapsto B^1 & & & & & \} \\
\mathbb{L} = \{ & x \vdash A =_\eta \lambda y.B_y & & & & & & \}
\end{array}$$

in the unification problems $\mathbb{P}$ above, X is used with arity 1 in $\mathbb{P}_1$ and with arity 0 in $\mathbb{P}_2$. In order to preserve invariant 4.2.1 (UNIFICATION-VARIABLE ARITY) the compiler did generate two entries in $\mathbb{M}$ for X, namely B and C. However, there is no connection between these two variables and any incompatible assignements to them would not be detected, in turn breaking proposition 4.1.3. To address this issue, we post-process $\mathbb{M}$ to ensure the following property:

Proposition 4.5.1 ($\mathbb{M}$ is a bijection)**.** *After compilation, for each $\mathcal{O}$-variable X in $\mathbb{P}$ and for each $\mathcal{M}$-variable A in $\mathbb{Q}$ there is exactly one entry $X \mapsto A^n$ in $\mathbb{M}$.*

Note that running $\simeq_m$ may require allocating new variables in $\mathcal{M}$ as explained in section 4.2.5. The property above ignores these variables since they are fresh and have no corresponding variable in $\mathcal{O}$.

The core procedure of this post-processing step is *align-arity* that is iterated by *map-deduplication*:

Definition 4.5.1 (*align-arity*)**.** Given two mappings $m_1 : X \mapsto A^m$ and $m_2 : X \mapsto C^n$ where $m < n$ and $d = n - m$, *align-arity* m_1 m_2 generates the following d links, one for each i such that $0 \leq i < d$,

$$x_0 \dots x_{m+i} \vdash B^i_{x_0 \dots x_{m+i}} =_\eta \lambda x_{m+i+1}.B^{i+1}_{x_0 \dots x_{m+i+1}}$$

where B^i is a fresh variable of arity $m + i$, and $B^0 = A$ as well as $B^d = C$.

The intuition is that we η-expand the occurrence of the variable with lower arity to match the higher arity. Since each η-`link` can add exactly one lambda, we need as many links as the difference between the two arities.

Definition 4.5.2 (*map-deduplication*)**.** Forall mappings $m_1, m_2 \in \mathbb{M}$ such that $m_1 : X \mapsto A^m$ and $m_2 : X \mapsto C^n$ and $m < n$ we remove m_1 from $\mathbb{M}$ and add to $\mathbb{L}$ the result of *align-arity* m_1 m_2.

Theorem 4.5.2 (Fidelity with *map-deduplication*)**.** *Given a list of unification problems $\mathbb{P}$, such that $\mathcal{W}_{\beta\eta}(\mathbb{P})$, if $\mathbb{P}$ contains the same $\mathcal{O}$-variable used at different arities, then* map-deduplication *guarantees proposition 4.1.3 (*SIMULATION FIDELITY*)*

Proof sketch.

Let X be a $\mathcal{O}$-variable appearing in two different subterms s_1 and s_2. In s_1 variable X is applied to the list of terms $x_1 \dots x_m$ while, in s_2, it is applied to $y_1 \dots y_n$. Without loss of generality we take $m < n$. After *map-deduplication* we have two distinct $\mathcal{M}$-variables for X, say A and C, related by a chain $\vec{c}$ of η-`links` of length $n - m$. We prove that if A and C are assigned respectively to the $\mathcal{W}$ terms $\lambda x_1 \dots x_p.t_1$ and $\lambda x_1 \dots x_q.t_2$, then a failure occurs iff these terms do not unify. By *η-progress-lhs*, the assignment of A activates p links of $\vec{c}$. In particular, when $p \geq n$, then the entire chain collapses and unification is performed between $\lambda x_n \dots x_p.t_1$ and $\lambda x_1 \dots x_q.t_2$. By theorem 4.3.3 and since we work with $\mathcal{W}$ terms, this unification succeeds iff it does in $\mathcal{O}$. Otherwise if $p < n$ then we have to consider two cases. 1) If t_1 is a rigid term then t_2 must be the η-redex of t_1, implying $q = 0$. If it is not the case, then *η-progress-lhs* would eventually cause a failure. 2) If t_1 is a unification variable then `scope-check` guarantees that t_2 does not use $x_1 \dots x_p$ as free variables, causing otherwise a unification failure. After this check, $m - n - p$ links of $\vec{c}$ remain suspended, as we expect: decompilation is charged to wake all links in $\mathbb{L}$ and eventually relate t_1 with t_2.

□

Example of *map-deduplication* If we take back the example given at the beginning of this section, *map-deduplication* produces the following result:

$$\begin{array}{lllllll}
\mathbb{P} = \{ & \lambda x.(X\ x) & \simeq_o & f & X & \simeq_o & \lambda x.a & \} \\
\mathbb{Q} = \{ & A & \simeq_m & f & C & \simeq_m & \lambda x.a & \} \\
\mathbb{M} = \{ & X \mapsto B^1 & & & & & & \} \\
\mathbb{L} = \{ & \vdash C =_\eta \lambda x.B_x & & x \vdash A =_\eta \lambda y.B_y & & & & \}
\end{array}$$

Note that $X \mapsto C^0$ disappears from $\mathbb{M}$ in favour of the auxiliary $\eta\texttt{-link} \vdash C =_\eta \lambda x.B_x$. The resolution of $\mathbb{Q}_1$ assignes a to A. This wakes up $\mathbb{L}_2$ by *η-progress-lhs*, assigning «$f\ x$» to B_x. The resolution of $\mathbb{Q}_2$ instantiates C to $\lambda x.a$, which, in turn triggers $\mathbb{L}_1$ by *η-progress-lhs*. In turn, this performs $\lambda x.a \simeq_m \lambda x.(f\ x)$ that fails as expected.

4.6 Handling of $\Diamond\mathcal{L}$

As observed in [43] it is worth handling terms in $\Diamond\mathcal{L}$ since, in practice, these terms often re-enter $\mathcal{L}$ at runtime.

In the following example problem $\mathbb{P}_2$, namely $X\ a \simeq_o a$, admits two different solutions: $\rho_1 = \{ X \mapsto \lambda x.x \}$ and $\rho_2 = \{ X \mapsto \lambda x.a \}$. The unification algorithm alone cannot chose the right one, since no solution is more general than the other, but for the first problem the choice is obvious, and in turn this choice makes $\mathbb{P}_2 \subseteq \mathcal{L}$:

$$\begin{array}{lllllll}
\mathbb{P} = \{ & X & \simeq_o & \lambda x.a & (X\ a) & \simeq_o & a & \} \\
\mathbb{Q} = \{ & A & \simeq_m & \lambda x.a & (A\ a) & \simeq_m & a & \} \\
\mathbb{M} = \{ & X \mapsto A^0 & & & & & & \}
\end{array}$$

In order to support this scenario we have to improve a little our compiler and progress routines. In particular $\mathbb{Q}_1$ generates $\sigma = \{ A \mapsto \lambda x.a \}$ making $\sigma\mathbb{Q}_2$ equal to $(\lambda x.a)\ a \simeq_m a$ that $\simeq_m$ cannot solve since it lacks rules r_{β_l} and r_{β_r} in 4.2.

To address this problem the compiler must recognize $\Diamond\mathcal{L}$ terms and replace them with fresh variables and generate a new kind of links that we call $\mathcal{L}\texttt{-link}$.

In addition to invariant 4.2.2 (LINK LEFT HAND SIDE), the term on the rhs of a $\mathcal{L}\texttt{-link}$ has the following property:

Invariant 4.6.1 ($\mathcal{L}\texttt{-link}$ rhs)**.** *The rhs of any $\mathcal{L}\texttt{-link}$ has the shape $X_{s_1 \ldots s_n}\ t_1 \ldots t_m$ where X is a unification variable in $\mathcal{L}$ (with scope $s_1 \ldots s_n$) and $t_1 \ldots t_m$ is a list of terms such that $m > 0$ and t_1 is either a variable occurring in $s_1 \ldots s_n$ or a term other than a variable.*

Note that the shape of such as rhs is «`app [uva X S | L]`», where `S` is `[`$\texttt{s}_1$`, ..., `$\texttt{s}_n$`]` and `L` is `[`$\texttt{t}_1$`, ..., `$\texttt{t}_m$`]`.

4.6.1 Compilation and decompilation

We insert this rule just before rule $r_@$ in 4.4

```
comp (o-app [o-uva A|Ag]) (uva B Sc) M1 M3 L1 L3 S1 S4 :- !,
  pattern-fragment-prefix Ag Pf Extra, alloc S1 B S2,
  len Pf Ar, m-alloc (ov A) (mv C (arity Ar)) M1 M2 S2 S3,
  fold6 comp Pf    Pf1    M2 M2 L1 L1 S3 S3,
```

```
fold6 comp Extra Extra1 M2 M3 L1 L2 S3 S4,
Beta = app [uva C Pf1 | Extra1],
get-scope Beta Sc,
L3 = [val (link-llam (uva B Sc) Beta) | L2].
```

The list `Ag` is split into two parts: `Pf` is in $\mathcal{L}$, and can be empty; `Extra` cannot be empty and is such that «`append Pf Extra Ag`». The rhs of the $\mathcal{L}$`-link` is the application of a fresh variable `C` having in scope all names in `Pf1` (the compilation of `Pf`). The variable `B`, returned as the compiled term, is a fresh variable having in scope all the free variables occurring in `Pf1` and `Extra1` (the compilation of `Extra`). This construction enforces invariant 4.6.1 ($\mathcal{L}$`-LINK RHS`).

Lemma 4.6.2 ($\mathcal{W}$-enforcement)**.** $\forall s, \langle s \rangle \mapsto (t, m, l)$, *then* $\mathcal{W}(t)$.

Proof sketch.

By lemmas 4.3.2 and 4.4.5 and the rule above.

□

Decompilation All $\mathcal{L}$`-link` should be solved before decompilation. If any $\mathcal{L}$`-link` remains in $\mathbb{L}$, decompilation fails.

4.6.2 Progress

We add the followins rules to `progress1`. Let l be a $\mathcal{L}$`-link` of the form $\Gamma \vdash T =_{\mathcal{L}} X_{s_1 \dots s_n} t_1 \dots t_m$

Definition 4.6.1 ($\mathcal{L}$*-progress-refine*)**.** Let σ be a substitution such that σt_1 is a name s not occurring in $s_1 \dots s_n$. If $m = 1$, then l is removed and the lhs is unified with $X_{s_1 \dots s_n\ s}$. If $m > 1$, then l is replaced by the link $\Gamma \vdash T =_{\mathcal{L}} Y_{s_1 \dots s_n\ s} t_2 \dots t_m$ (where Y is a fresh variable of arity $n+1$) and link $\Gamma \vdash X_{s_1 \dots s_n} =_\eta \lambda x. Y_{s_1 \dots s_n\ x}$ is added to $\mathbb{L}$.

Definition 4.6.2 ($\mathcal{L}$*-progress-rhs*)**.** Link l is removed from $\mathbb{L}$ if $X_{s_1 \dots s_n}$ is instantiated to a term t and $t\, t_1 \dots t_m$ β-reduces to a $t' \in \mathcal{L}$.

Definition 4.6.3 ($\mathcal{L}$*-progress-fail*)**.** progress fails when either

there exists another link $l' \in \mathbb{L}$ with the same lhs as l; or

the lhs of l become rigid.

We relax this condition and accommodate for heuristics in 4.6.3.

Finally we introduce the following η`-link` progress rule.

Definition 4.6.4 (η*-progress-rhs*)**.** A link $\Gamma \vdash X =_\eta T$ is removed from $\mathbb{L}$ when either

maybe-eta T does not hold (anymore), in this case X is unified with T; or

T η-contracts to a term T' not starting with the `lam` constructor. In this case X is unified with T'

The idea behind this rule is to instantiate the lhs variable of a η−`link` whenever its rhs is for sure a $\mathcal{W}$ term.

Lemma 4.6.3. `progress` *terminates*

Proof sketch.

Let $l = \Gamma \vdash T =_{\mathcal{L}} X_{s_1 \dots s_n} \cdot t_1 \dots t_m$ a $\mathcal{L}$−`link` in the store $\mathbb{L}$. The progression made by $\mathcal{L}$-*progress-rhs* terminates: l is removed from $\mathbb{L}$ after having performed terminating instructions. If l is not activated by $\mathcal{L}$-*progress-rhs*, then X is a variables. The activation of $\mathcal{L}$-*progress-refine* replaces l with the new $\mathcal{L}$−`link` $\Gamma \vdash T =_{\mathcal{L}} Y_{s_1 \dots s_n\ s} \cdot t_2 \dots t_m$. This progression can be triggered at most m times, since at each time the number of terms applied to X decreases. In the end l_m will be removed from $\mathbb{L}$ after the unification between lhs with rhs. We also note that each $\mathcal{L}$-*progress-refine* generates a η−`link`, yet, according to lemma 4.4.10, this does not affect termination. $\mathcal{L}$-*progress-fail* terminates since it stop progression with failure. Finally, η-*progress-rhs* performs terminating operations and, if its permises succeed, the considered η−`link` is removed from $\mathbb{L}$, ensuring termination.

□

Theorem 4.6.4 (Fidelity in $\mathcal{O}$)**.** *The introduction of* $\mathcal{L}$−`link` *guarantees proposition 4.1.4 (*FIDELITY RECOVERY*) if* $\mathbb{P} \notin \mathcal{L}$.

Proof sketch.

Let $\mathbb{P}_i$ be the first problem in $\mathbb{P}$ such that it is not in $\mathcal{L}$ and let σ be the substitution obtained solving $\mathbb{Q}_1 \dots \mathbb{Q}_{i-1}$. After the execution of step_m for the $i-1$ time, each variable X corresponding to a $\diamond\eta$ subterm in the original $\mathcal{O}$ unification problem, is instantiated iff it is known for sure that X is no more in $\diamond\eta$. If $\sigma\mathbb{Q}_i$ is in $\mathcal{L}$, then by definitions 4.6.1 and 4.6.2 the associated $\mathcal{L}$−`link` is solved and removed. In this case, all calls to $\simeq_m$ are between terms in $\mathcal{L}$ and by theorem 4.3.3 fidelity is guaranteed. If $\sigma\mathbb{Q}$ is still in $\diamond\mathcal{L}$, then by definition 4.6.3 unification fails as the corresponding unification in $\mathcal{O}$ would (it is called outside its domain).

□

4.6.3 Relaxing definition 4.6.3 (*$\mathcal{L}$-progress-fail*)

Working with terms in $\mathcal{L}$ is sometime too restrictive [1] and we could find in literature a few strategies to go beyond $\mathcal{L}$ without implementing Huet's algorithm [29]. Some implementations of λProlog [44] such as Teyjus [43] delay the resolution of $\diamond\mathcal{L}$ unification problems until the substitution makes them reenter $\mathcal{L}$. Other systems apply heuristics like preferring projection over mimic, and commit to that solution. This is the case for the unification algorithm *Rocq* uses in its type-class solver [52],

In this section we show how we can implement these strategies by simply adding (or removing) rules to the `progress` predicate. In the example below $\mathbb{P}_1$ is in $\diamond\mathcal{L}$.

$$
\begin{array}{lccccccc}
\mathbb{P} = \{ & (X\ a) & \simeq_o & a & X & \simeq_o & \lambda x.Y & \} \\
\mathbb{Q} = \{ & A & \simeq_m & a & B & \simeq_m & \lambda x.C & \} \\
\mathbb{M} = \{ & Y \mapsto C^0 & X \mapsto B^0 & & & & & \} \\
\mathbb{L} = \{ & x \vdash A =_{\mathcal{L}} (B\ a) & & & & & & \}
\end{array}
$$

If we want the object-language unification to delay the first unification problem (waiting for X to be be instantiated), we can relax definition 4.6.3. Instead of failing when the lhs of $\mathbb{L}_1$ becomes rigid (equal to a), we keep it in $\mathbb{L}$ until the head of its rhs also become rigid. In this case, since both the lhs and rhs have rigid heads, they can be unified. While this relaxed rule does not break proposition 4.1.3 (SIMULATION FIDELITY) per se, the `occur-check-links` procedure becomes incomplete since invariant 4.2.2 (LINK LEFT HAND SIDE) is broken. Also note that delaying unification outside $\mathcal{L}$ can leave $\mathcal{L}$`-link` for the decompilation phase. Therefore `commit-links` should be modified accordingly.

If instead we want $\simeq_o$ to follow the second strategy and pick an arbitrary solution we can modify progress by applying the desired heuristic instead of failing. For instance, in $X\,a\,b = Y\,b$, the last argument of the two terms is the same and unification can succeed by assigning $X\,a$ to Y. This heuristic is used by [52].

4.7 Actual implementation in *Elpi*

In this paper we did study a compiler and a simulation loop on a minimal language. The actual implementation uses the *Rocq-Elpi* meta-language to both compile the sequence of problems (the rules) and execute them, that is *Elpi* plays the role of $\mathcal{M}$ as well.

The main difference is that we cannot implement run_m since it is the runtime of the programming language. In particular the runtime iterates $\simeq_m$, but step_m also needs to check links for progress. Luckily *Elpi* extends λProlog with constraints (suspended goals) and Constraint Handling Rules (*CHR*) to operate on them section 2.2.5. Similarly to [39] a constraint is a goal suspended on a list of variables that is resumed *as soon as one of these variable is assigned*, before any other existing goal is considered. In turn link activation grants proposition 4.1.3 (SIMULATION FIDELITY). We point out that [39] delays $\Diamond\mathcal{L}$ unification problems while we also delay problems that are in $\Diamond\eta$.

In the following snippet, we show how η`-link` is either suspended or allowed to make progress.

```
link-eta L R :- not (var L), !, eta-progress-lhs L R.
link-eta L R :- not (maybe-eta R), !, eta-progress-rhs L R.
link-eta L R :- declare_constraint (link-eta L R) [L,R].
```

The first two clauses describe when progress can occur. If `L` is no longer a variable, the invariant invariant 4.2.2 is violated; therefore, we trigger unification between `L` and `R`. Similarly, if `R` is no longer a $\Diamond\eta$ term, the invariant invariant 4.4.1 does not hold anymore, and we again force unification of `L` and `R`.

If neither condition holds, no progress is possible, and the constraint is suspended again. Similar rules are implemented for $\mathcal{L}$`-link`.

```
rule (N1 ▷ G1 ?- link-eta (uvar X LX1) T1)    % match
  /  (N2 ▷ G2 ?- link-eta (uvar X LX2) T2)    % remove
  |  (relocate LX1 LX2 T2 T2')                % condition
 <=> (N1 ▷ G1 ?- T1 = T2').                   % new goal
```

In the snippet above, we illustrate the deduplication of η`-link`. The syntax «`N ▷ G ?- P`» denotes a λProlog sequent, that is a goal `P` under a set `G` of hypothetical rules (introduced by

`=>`) and where the program context binds (via the `pi` operator) eigenvariables in `N`. Sequents are presented to *CHR* with their higher-order unification variables replaced by "frozen constants", that is A_{xyz} becomes «`uvar` f_A `[x,y,z]`» for a fresh constant f_A. Frozen constants are "defrost" when they are part of a new goal (see [25, section 4.3]).
The first directive matches a constraint whose lhs is a variable `X`; the second matches and removes a constraint on the same variable; the third relocates `T2` and the latter crafts the new goal that unifies `T1` with `T2'` under `G1` and the set of heigenvariables `N1`.

4.8 Related work and conclusion

Object-language terms can be unified using different strategies.

The first approach that comes to mind consist in implementing $\simeq_o$ as a regular routine, i.e. write rules as follows

```
tc-Provable {{impl lp:F lp:B}} {{Pimpl lp:F lp:B lp:Q}} :-
  pi p \ tc-Provable {{fact lp:F}} p => tc-Provable B X,
  unify X {{lp:Q lp:p}}.
```

where `unify` is a regular predicate. This method fails to take advantage of the logic programming engine provided by the meta language since it removes all the data from the rule's heads and makes indexing degrade. Additionally, implementing a unification procedure in the meta language is likely to be significantly slower compared to the built-in one on the common domain of first order terms.

Another possibility is to avoid having application and abstraction nodes in the syntax tree, and use the meta language ones, as in:

```
tc-Provable ({{impl}} F B) ({{Pimpl}} F B Q}} :-
  pi p \ tc-Provable ({{fact}} F) p => tc-Provable B (Q p).
```

However, this encoding has two big limitations. First it is not always feasible to adopt it for *Rocq* due to the fact that the type system of the meta language is too limited to accommodate the one of the object language, e.g. *Rocq* can typecheck variadic functions [8].
Second, the encoding of *Rocq* terms provided by *Elpi* is primarily used for meta programming, i.e. to extend the *Rocq* system. Consequently, it must be able to manipulate terms that are not known in advance without relying on introspection primitives such as Prolog's `functor/3` and `arg/3`. To avoid that constants cannot be symbols of the meta language, but must rather live in an open world, akin to the `string` data type used in the `con` constructor.

In the literature we could find a related encoding of the Calculus of Constructions (CC) [15]. The goal of that work is to exhibit a logic program performing proof checking for CC and hence relate the proof system of intuitionistic higher-order logic (that animates λProlog programs) with the one of CC. The encoding is hence tailored toward a different goal, for example it utilizes three relations to represent the equational theory of CC, and that choice alone makes things harder for us. Section 6 contains a discussion about the use of the unification procedure of the meta language in presence of non ground goals, but the authors are not interested in exploiting a decidable fragment of higher-order unification but are rather leaning towards an interactive system where the user is given control over the search procedure.

Another work that is, somewhat surprisingly, only superficially related to ours is the type-class engine built in Isabelle's meta language. In [63] classes identify (simple) types, they are not higher order predicates as in *Rocq*, hence the solver does not require a higher-order unification procedure.

The approach presented in this paper addresses all the concerns mentioned earlier. It takes advantage of the unification capabilities of the meta language at the price of handling problematic sub terms on the side. As a result our encoding takes advantage of indexing data structures and mode analysis for clause filtering. It is worth mentioning that we replace terms with variables only when it is strictly needed, leaving the rest of the term structure intact and hence indexable. Some preliminary benchmarks on the Stdpp library [30] show encouraging results: our type-class solver has some overhead on small goals but is consistently faster than the *Rocq* one starting from goals made of 32 nodes. Moreover our approach is flexible enough to accommodate different strategies and heuristics to handle terms outside the pattern fragment. Finally, our encoding is sufficiently shallow to: 1) allow to lift forms of static analysis for the meta language, such as determinacy, also to the object language; 2) take advantage of future improvements to the logic-programming engine of *Elpi*, such as tabled search.

We considered mechanizing our results with Abella [20], especially in the early phase of this work, but unfortunately we could not. The main reason is that the logic of Abella does not let one quantify over predicates nor use the cut operator. Eliminating cut or higher order combinators like `map` or `forall2` is surely possible, but has a high cost in verbosity given how pervasively they are used. Moreover, and more importantly, it creates a distance between the verified and the actual code, which we found to hinder our exploration. However, we did establish a test suite with about a hundred cases. It is available at https://github.com/FissoreD/ho-unif-for-free.

CHAPTER 5

Determinacy checking for *Elpi*

Over the years *Elpi* was put in the hands of quite a number of *Rocq* users and the most widely cited barrier to entry has been the paradigm shift to a logic programming one. *Rocq* users are comfortable with typed and functional programming but often lack experience with backtracking and its control. Uncontrolled backtracking can produce slower programs and complicate debugging.

To lower this barrier we design a system of determinacy signatures that allows programmers to declare when a predicate behaves like a function: at most one result is produced and the runtime will not consider alternative results. This notion of determinism is called *semidet* by Henderson et al. [28] and named *operationally deterministic* by Nakamura [45]. Alternative notions exist, for example Warren [12] considers predicates *observably deterministic* when they produce the same result multiple times or diverge. We chose operational determinacy since it matches the behavior of functions in mainstream functional languages.

Our determinacy checker covers *Elpi*'s higher-order features, including higher-order predicates (for example map and fold, see section 5.1.1) and dynamic predicates that extend themselves at runtime (see section 5.1.2).

We apply these signatures to a large portion of publicly available *Elpi* code in the *Rocq* ecosystem and report our experience. Our findings indicate that most *Elpi* programs are intended to be backtracking-free and require only minimal signatures for the static analyzer we implemented to verify determinacy.

Our contributions are: 1) we design a language of signatures to assert the determinacy of predicates that covers higher-order logic programming constructs such as first-class predicates and dynamic predicates; 2) we implement a static analyzer for that language; 3) we apply the static analyzer to most of the *Elpi* code available in the *Rocq* ecosystem.

5.1 *Elpi* and determinacy signatures by examples

As in most logic programming languages, *Elpi* programs are organized in rules. When rules are not *mutually exclusive*, i.e. multiple rules apply on the same query, the runtime continues the computation using the rule with highest priority. Moreover it stores the other rules as a *choice point* and can backtrack to that point to continue the computation. What makes backtracking tricky to *Elpi* users is that it is an opt-out feature of logic programming: it is left to the programmer to tell the runtime to commit to one rule and discard the choice point, rather than the other way around. If the user forgets to commit to a rule, his program still works but becomes much harder to debug and typically slower to run in case of failures.

The simplest example is the list-membership *relation* that can succeed more than once if the element we are looking for occurs multiple times in the list:

```
mem X [Y|_ ] :- X = Y.     % the head is the X we are looking for, or
mem X [_|YS] :- mem X YS. % recurse on the tail
```

The first rule for `mem` is the base case: it succeeds if the head of the list is equal to the `X` we are looking for. The second one discards the list's head and continues looking for `X` in the tail. A query such as `mem 2 [2,3,2]` can be solved in two different ways: either using rule 1 immediately, or using rule 2 twice to discard the first two items, and then conclude with rule 1.

Elpi users often have a background in functional programming and simply do expect `mem` to be a *function* testing membership, and not a relation. Moreover one needs to look, carefully, at how `mem` is implemented in order to see the peril. This misconception becomes a problem when they use `mem` in a larger piece of code and observe its execution.

```
main L :- code_before L, mem 1 L, code_after L.
```

For example, in the code above, one may see `code_after` being executed multiple times: each choice point left by `mem` enables the runtime to backtrack to that point in time and continue from there. Even worse, if `code_after` is expensive and fails, one pays its cost multiple times.

It must be said that backtracking can be very useful in some situations and that a programmer with experience in logic programming can take advantage of that. We want the *Elpi* casual user to be able to write code without mastering all that, and simply declare that they expect *their code* to compute functions.

```
func main list int -> . % the signature for main
```

The signature starts by a keyword declaring the determinacy of the predicate: **`func`** for deterministic predicates, that we also call functions, and **`pred`** for any predicate, including both functions and relations. Inputs are separated from outputs by the arrow symbol, and in the case of `main` there is no output.

```
main L :- code_before L, mem 1 L, code_after L.
%                        ^^^^^^ error: non-deterministic atom
```

Since `main` was declared as a function, our static analysis invites the user to discard the choice points left by `mem`, for example by inserting a **`!`** (a *cut* directive) after it. Assuming `code_after` is a functions, this code is accepted, since the cut discards all choice points generated by `mem` and `code_before L`.

```
main L :- code_before L, mem 1 L, !, code_after L. % OK
```

Note that the programmer may also write a functional membership test as follows, and use it in functional code with no additional safeguard. In this case the cut discards the second rule as soon as `X = Y` succeeds.

```
func in int, list int -> .
in X [Y|_ ] :- X = Y, !. % commit to this rule, discard the next one
in X [_|YS] :- in X YS.

main L :- code_before L, in 1 L, code_after L. % OK
```

Statically tracking the use of backtracking is a well-known technique in logic programming, usually referred to as determinacy analysis [12, 51]. However, the literature does not provide a detailed treatment of features that are specific to *Elpi* and not available in plain *Prolog*, such as higher-order programming, dynamic programming, and meta-programming.

These features were introduced in section 2.2.4 and play an important role in determinacy. In the following three subsections, we revisit those examples and analyze them from the perspective of determinacy.

5.1.1 Higher-order programs

A notable higher-order program is the one turning relations into functions.

```
func once (pred) -> . % a function taking a fully applied predicate
once P :- P, !.       % the ! commits to the first success
```

We say that the signature of the `once` predicate marks it as *unconditionally* a function: `once` leaves no choice points, no matter what it receives as the higher-order argument `P`.

An example of a *conditional* function is the predicate to map over a list:

```
func map (func A -> B), list A -> list B.
map _ [] [].
map F [X|XS] [Y|YS] :- F X Y, map F XS YS.
```

We say that the signature of `map` has a *precondition*, namely that `F` is a function, and a *postcondition*, namely that `map` is a function, i.e., the call `map F L L'` leaves no choice points. If the precondition is not met the postcondition is not guaranteed to hold and we say that `map` is *miscalled*. In this case we weaken its signature to*:

```
pred map (func A -> B), list A -> list B.
```

Our static analysis tracks miscalls to functions and treats them as relations when analyzing the surrounding code. This avoids code duplication, since only one definition of `map` is needed in the library. We illustrate this with four scenarios involving correct calls and miscalls to `map`.

```
% a function                          % a relation
func incr int -> int.                 pred get_pdiv int -> int.
incr 1 2.                             get_pdiv 6 3. get_pdiv 6 2.

       % four predicates calling map or miscalling map
func p list int -> list int.          pred q list int -> list int.
p L R :- map incr L R. % OK           q L R :- map get_pdiv L R. % OK

func r list int -> list int.          func r1 list int -> list int.
r L R :- map get_pdiv L R.            r1 L R :- map get_pdiv L R, !. % OK
           % ^^^^^^^^ error: non-deterministic atom
```

In this example, `incr` is a function, while `get_pdiv` is a relation. In particular, `get_pdiv` associates more than one result to the input `6`, making it non-deterministic.

The predicate `p` is deterministic because it satisfies the precondition of `map`: the argument `incr` is a function, so the call does not introduce choice points.

*We explain in detail this weakining operation in section 5.4

The predicate `q` instead contains a miscall to `map`, since `get_pdiv` is a relation. As a result, the execution may leave choice points. However, this is accepted because `q` is declared as a relation (with `pred`), and therefore non-determinism is allowed.

The situation is different for `r`. Its body is the same as `q`, but `r` is declared as a function. For this reason, the static analysis reports an error: a deterministic predicate cannot contain a non-deterministic computation.

Finally, `r1` has the same signature as `r` and also miscalls `map`, but it explicitly removes choice points using a cut. This makes the overall behavior deterministic, and the definition is accepted by the analyzer. Similarly, an implementation such as `once (map get_pdiv L R)` would also be valid, since the `once` wrapper ensures that any potential choice points are discarded.

5.1.2 Dynamic programs

In section 2.2.4, we pointed out the interest of dynamic programming in a language like *Elpi*. In this paragraph, we point out why the predicate `copy` can be considered as a deterministic predicate.

```
func copy tm -> tm.
copy (app A B) (app C D) :- copy A C, copy B D.
copy (lam F) (lam G)     :- pi x\ copy x x => copy (F x) (G x).
```

`copy` is a function because no two rules can apply to the same input: `lam` and `app` are different constants, and each constant `x` generated dynamically is guaranteed to be fresh by the `pi` operator, hence each `copy x x` rule is mutually exclusive with the other rules.

5.1.3 Meta programs

The meta-program we have show in section 2.2.4 is a good example of how our determinacy checker interacts well in program inspecion and composition.

We take back the implementation for the predicates `comp` and `fuse`, but this time, we pay atention to their determinacy in the signatures.

```
func comp (func A → B), (func B → C), A → C.
comp F G X Z :- F X Y, G Y Z.

func fuse (func list A → list B), (func list B → list C) →
          (func list A → list C).
fuse (map F) (map G) (map (comp F G)).
```

The signature of `comp` infroms the checker that `comp` behaves deterministically if its two higher-order arguments are functions. This piece of informarmation is important when the checker analyses the rule for `fuse`.

The signature of `fuse` has two preconditions, both inputs should be functions, and two post-conditions: `fuse` is a function and its output is also a function. The code of `fuse` inspects two programs: if both are mapping a list, respectively with `F` and `G`, then it generates a new program `map (comp F G)`. The static analysis is able to verify that this output is a function following this reasoning: since `map F` and `map G` are functions then also `F` and `G` must be; from that we have that `comp F G` is a function and so is `map (comp F G)`.

5.2 *Elpi*'s abstract syntax and semantics

Any static analysis to rule out non-determinism must depend on the operational semantics of the language. In particular the order in which rules are applied and the order in which rule premises are executed directly impact the scope of the cut directive.

The *Elpi* syntax and semantics considered in this section are those presented in sections 2.2.1 and 2.2.2. To simplify the presentation, we treat `pi` and `=>` as a single operator. In this setting, the construct `pi` x. $r \Rightarrow a$ represents the introduction of a fresh symbol x, which is visible in both the rule r and the atom a. The local rule r is added to the scope of the atom a.

Using the combined form `pi` x. $r \Rightarrow a$ does not reduce the expressiveness of *Elpi*. The behavior of the `pi` operator can be simulated by introducing a dummy rule r in a, while the `=>` operator can be recovered by using a vacuous variable `x`.

We omit from the syntax the construction of data with binders, since it does not play a relevant role in the static analysis of determinacy.

We use the following notation: s ranges over symbol names (including predicates, data constructors, and variables), v over variables, k over predicate and data constructors, a and b over atoms, and r over rules.

In summary, the algorithm proceeds as follows:

it checks that each rule is mutually exclusive with the others in the program;

it verifies that the sequence of calls in the body consists of correctly used functional predicates;

it ensures that the preconditions of rules entail their postconditions.

5.3 Mutual Exclusion

We focus on how to address the first source of non-determinism. The interesting aspects for this section are the `Cut` operator and the `matching`/`unify` procedure depending on the modes of a predicate.

In the next definition we distinguish between global rules, i.e., rules that are part of the program $\mathcal{P}$, and local rules that are loaded via the `pi` operator. For example, the `copy` predicate from section 5.1.2 has two global rules and one local rule.

Definition 5.3.1 (Mutual exclusion (mut_excl $\mathbb{P}$ $\mathbb{R}$))**.** Given a program $\mathcal{P}$ and a rule r, such that $r \notin \mathcal{P}$, we say that r is mutually exclusive with the rules in $\mathcal{P}$ if it has the highest priority and one of the following additional conditions holds:
(a) there is a `Cut` in the body of r, or
(b) r is a global rule and all rules r' in $\mathcal{P}$ have an input argument not unifying with the corresponding input in r
(c) r is a local rule introduced via `pi` x r b, x is an input of r and for all rules r' in the program, the corresponding input is not a variable

```
pred p int -> int.
p 1 3.
p 1 2.
```

```
func q int -> int.
q 1 3.
q 2 4.
```

```
func r int -> int.
r X R :- p X R, !.
r X R :- q X R.
```

For example, the rules for predicate `p` above are not mutually exclusive, so choice points may remain after a query resolution, hence introducing non-determinism. For instance, `p 1 X` has two solutions, namely `X = 3` and `X = 2`.
Conversely, `q` satisfies mutual exclusion: no query can trigger both rules. Note that, since inputs are matched, a query like `q X Y` has no solution in *Elpi*. Both rules for `r` match any input. If the first rule succeeds, the `Cut` removes the alternatives created by the call to `p` and also discards the other rule `r X R :- q X R`. If the first rule fails, the `Cut` is not reached hence the second rule is not discarded. Thus, at most one rule can succeed for `q`.

The predicates `s` and `s1` below illustrate the role of `pi`.

```
func s int -> int.                    func s1 int -> int.
s X R :- pi y\ q y 3 => q X R.        s1 X R :- pi y\ q 1 5 => q X R.
                              % error: mutxecl violated  ^^^^^
```

The local rule `q y 3` is mutually exclusive with the rules for `q` above since `y` is fresh: each time the rule is loaded `y` is different from `1` and `2` and any other fresh constants generated before. On the contrary the local rule `q 1 5` is not mutually exclusive: the query `s1 1 R` has two results, namely `R = 5` and `R = 3`.

Note that condition *(c)* also rules out the following example, since the `pi`-quantified symbol `y` is not used in input position.

```
func s2 int -> int.
s2 1 R :- pi y\ q 3 5 => s2 2 R.
s2 2 R :- pi y\ q 3 6 => q 3 R.
%                 ^^^^^ error: mutxecl violated
```

Both rules for `s2` load a local rule for `q`. All these local rules are mutually exclusive with the global rules in the program, but the query `s2 2 R` has two results namely `R = 5` and `R = 6`.

5.3.1 Checking mutual exclusion: discrimination trees

This subsection briefly describes our implementation of mutual exclusion verification in *Elpi*. It is not required for understanding the rest of the chapter and can be skipped on a first reading.

To support mutual exclusion checking in *Elpi*, we rely on the discrimination tree data structure [38], a standard mechanism for indexing rules in logic programming systems. Discrimination trees are widely used in *Prolog*-like systems to improve the efficiency of clause retrieval. Our implementation follows, in part, the presentation given in Pfenning's notes [†].

A discrimination tree is a trie-based structure. Inner nodes are labeled, while leaves store buckets of objects. Tries represent dictionaries by sharing common prefixes of keys. A *path* is defined as the sequence of labels encountered from the root to a leaf.

Two basic operations are supported: insertion and retrieval. To insert an element e with key k, the algorithm traverses the trie symbol by symbol, starting from the root. For each symbol, it follows the corresponding edge, creating a new node if necessary. Once all symbols have been consumed, the element e is stored in the resulting leaf. Retrieval works similarly: the key is read symbol by symbol, and the tree is traversed accordingly. If the path exists, the elements stored in the corresponding leaf are returned.

[†] https://www.cs.cmu.edu/~fp/courses/99-atp/lectures/lecture28.html

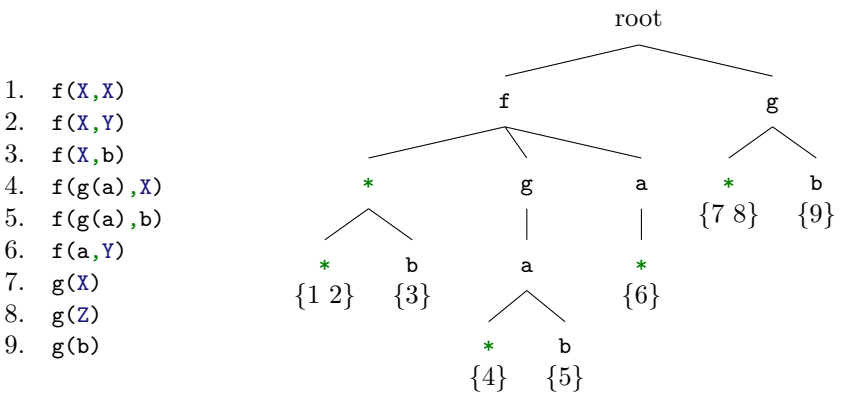


Figure 5.1: Example of a discrimination tree, from [38]

In *Prolog* systems, the indexed objects are the rules of the database, and the head of each rule serves as its key. In figure 5.1, the database (left) and its corresponding discrimination tree (right) are shown. Leaves are annotated with the rules stored in each bucket. The purpose of the tree is to enable fast clause retrieval.

Variables are represented in an approximate way. In particular, they are encoded as generic symbols that match any term. For instance, in figure 5.1, rules 1 and 2 share the same path and are stored in the same bucket. Retrieval must also take modes into account. For example, if the predicate `g` has one input and one output argument, the query `g X Y` returns only rules 7 and 8. If both arguments are outputs, then rules 7, 8, and 9 are returned.

For efficiency, terms are encoded as sequences of integers, where each integer represents multiple pieces of information at the bit level. Modes are included in this encoding, allowing the system to determine whether a rigid term should be matched or unified during retrieval.

Unification in the discrimination tree is only an approximation of the full *Elpi* unification algorithm, which is more expensive. By construction, the set of rules returned by the tree is a superset of the rules that actually match the query. A second phase applies the full *Elpi* unification to filter out non-applicable rules.

We also optimize the encoding of query paths. In practice, query terms are often larger than the terms stored in the database, so encoding them entirely is not useful. Instead, we use a hit-map to decide which subterms should be encoded and at which depth. Furthermore, to avoid complex unification, terms headed by a λ-symbol are treated as variables. For example, `x\ X x` may unify with `id` or `(x\ 3)`, but all such terms are encoded as variables in the tree.

Discrimination trees are effective for indexing rules in the *Elpi* database, making retrieval faster in many cases. In this chapter, however, their main role is to support mutual exclusion checking.

During compilation, an auxiliary discrimination tree is built. Each rule of a deterministic predicate is inserted into this structure. When a new rule is added, the tree is queried to retrieve the set of potentially overlapping rules r. By invariant, r is an ordered (possibly empty) list in which all rules, except possibly the last one, contain a `Cut`. The mutual exclusion check then verifies that the new rule is inserted either at the end of the list; otherwise it should contain the `Cut` operator.

5.4 Static analysis of determinacy signatures

The abstract syntax of signatures is given below. Predicate signatures $\mathbb{T}\mathrm{y}$ start with input arguments from the $\mathbb{S}_i$ non-terminal, continue with output arguments from the $\mathbb{S}_o$ non-terminal and end with a determinacy marker $\mathbb{D}$: `Fun` for functions and `Rel` for relations (respectively `func` and `pred` in the concrete syntax).

$$\begin{array}{llll} \mathbb{S}_i & ::= \mathbb{S}_o \mid \mathbb{S}_i \xrightarrow{\mathrm{i}} \mathbb{S}_i \mid \bigstar \xrightarrow{\mathrm{i}} \mathbb{S}_i & \text{sign. start} \\ \mathbb{S}_o & ::= \mathbb{D} \mid \mathbb{S}_i \xrightarrow{\mathrm{o}} \mathbb{S}_o \mid \bigstar \xrightarrow{\mathrm{o}} \mathbb{S}_o & \text{sign. stop} \\ \mathbb{T}\mathrm{y} & ::= \mathbb{S}_i & \text{signature} \end{array} \qquad \begin{array}{lll} \mathbb{D} & ::= \texttt{Fun} \mid \texttt{Rel} & \text{determinacy} \\ \bigstar & ::= \star & \text{all data} \end{array}$$

We collapse all data types into a single type $\bigstar$ and we disallow to store predicates into data, i.e., one cannot form a list of predicates. Note that the grammar of $\mathbb{T}\mathrm{y}$ forces all input arguments to come first. We reserve d for instances of type $\bigstar$, ρ and τ for signatures and $\mathcal{D}$ for instances of $\mathbb{D}$. For example the signature `func map (func A -> B), list A -> list B` corresponds to $(\star \xrightarrow{\mathrm{i}} \star \xrightarrow{\mathrm{o}} \texttt{Fun}) \xrightarrow{\mathrm{i}} \star \xrightarrow{\mathrm{i}} \star \xrightarrow{\mathrm{o}} \texttt{Fun}$.

We denote booleans $\mathbb{B} = \{\top, \bot\}$; contexts Γ being partial maps from symbols to signatures. We use $\varnothing$ for the empty list and $x :: xs$ for prepending the element x to a list xs. We reserve Γ for contexts.

5.4.1 Operations on signatures

We compare signatures using the standard subtyping relation of functional languages. In particular, we compare inputs contravariantly and outputs covariantly. It is similar to [27], except for the absence of the `Any` terminal. `Fun` is smaller than `Rel` since functions generate a "smaller number of results" than relations.

$$\begin{array}{lllll} \star \subseteq \star & \texttt{Fun} \subseteq \texttt{Fun} & \texttt{Rel} \subseteq \texttt{Rel} & \texttt{Fun} \subseteq \texttt{Rel} & \text{base cases} \\ \multicolumn{4}{l}{\rho \xrightarrow{\mathrm{i}} \tau \subseteq \rho' \xrightarrow{\mathrm{i}} \tau' \text{ iff } \rho' \subseteq \rho \wedge \tau \subseteq \tau'} & \text{covariance} \\ \multicolumn{4}{l}{\rho \xrightarrow{\mathrm{o}} \tau \subseteq \rho' \xrightarrow{\mathrm{o}} \tau' \text{ iff } \rho \subseteq \rho' \wedge \tau \subseteq \tau'} & \text{contravariance} \end{array}$$

The static analysis needs often to keep the stronger or weaker signature among two. We hence derive from $\subseteq$ a `min` and `max` operators defined below.

$$\begin{array}{ll} \texttt{min} \star \star = \star & \texttt{min} \star \star = \star \\ \texttt{min Rel Rel} = \texttt{Rel} & \texttt{min Rel Rel} = \texttt{Rel} \\ \texttt{min Rel Fun} = \texttt{Fun} & \texttt{min Rel Fun} = \texttt{Rel} \\ \texttt{min Fun Rel} = \texttt{Fun} & \texttt{min Fun Rel} = \texttt{Rel} \\ \texttt{min Fun Fun} = \texttt{Fun} & \texttt{min Fun Fun} = \texttt{Fun} \\ \texttt{min}\ (\rho \xrightarrow{\mathrm{i}} \tau)\ (\rho' \xrightarrow{\mathrm{i}} \tau') = \texttt{max}\ \rho\ \rho' \xrightarrow{\mathrm{i}} \texttt{min}\ \tau\ \tau' & \texttt{max}\ (\rho \xrightarrow{\mathrm{i}} \tau)\ (\rho' \xrightarrow{\mathrm{i}} \tau') = \texttt{min}\ \rho\ \rho' \xrightarrow{\mathrm{i}} \texttt{max}\ \tau\ \tau' \\ \texttt{min}\ (\rho \xrightarrow{\mathrm{o}} \tau)\ (\rho' \xrightarrow{\mathrm{o}} \tau') = \texttt{min}\ \rho\ \rho' \xrightarrow{\mathrm{o}} \texttt{min}\ \tau\ \tau' & \texttt{max}\ (\rho \xrightarrow{\mathrm{o}} \tau)\ (\rho' \xrightarrow{\mathrm{o}} \tau') = \texttt{max}\ \rho\ \rho' \xrightarrow{\mathrm{o}} \texttt{max}\ \tau\ \tau' \end{array}$$

When a predicate is miscalled we need to weaken its signature.

We define strong and weak as follows:

$$\begin{array}{ll} \text{strong } \star = \star & \text{weak } \star = \star \\ \text{strong Rel} = \texttt{Fun} \quad \text{strong Fun} = \texttt{Fun} & \text{weak Fun} = \texttt{Rel} \quad \text{weak Rel} = \texttt{Rel} \\ \text{strong } (\rho \xrightarrow{\mathrm{i}} \tau) = \text{weak } \rho \xrightarrow{\mathrm{i}} \text{strong } \tau & \text{weak } (\rho \xrightarrow{\mathrm{i}} \tau) = \text{strong } \rho \xrightarrow{\mathrm{i}} \text{weak } \tau \\ \text{strong } (\rho \xrightarrow{\mathrm{o}} \tau) = \text{strong } \rho \xrightarrow{\mathrm{o}} \text{strong } \tau & \text{weak } (\rho \xrightarrow{\mathrm{o}} \tau) = \text{weak } \rho \xrightarrow{\mathrm{o}} \text{weak } \tau \end{array}$$

The set of signatures that only differ by a leaf in $\mathcal{D}$ form a lattice w.r.t. the order relation $\subseteq$, the join operator `max` and meet operator `min`. The infimum of the lattice is computed by strong and the supremum is computed by weak.

In section 5.1.1 we have weakened the signature of `map` from $\tau = (\star \xrightarrow{\mathtt{i}} \star \xrightarrow{\mathtt{o}} \mathtt{Fun}) \xrightarrow{\mathtt{i}} \star \xrightarrow{\mathtt{i}} \star \xrightarrow{\mathtt{o}} \mathtt{Fun}$ to $(\star \xrightarrow{\mathtt{i}} \star \xrightarrow{\mathtt{o}} \mathtt{Fun}) \xrightarrow{\mathtt{i}} \star \xrightarrow{\mathtt{i}} \star \xrightarrow{\mathtt{o}} \mathtt{Rel}$.

One could define the supremum of τ as $\tau' = (\star \xrightarrow{\mathtt{i}} \star \xrightarrow{\mathtt{o}} \mathtt{Rel}) \xrightarrow{\mathtt{i}} \star \xrightarrow{\mathtt{i}} \star \xrightarrow{\mathtt{o}} \mathtt{Rel}$, which intuitively amounts to considering all `Fun` as `Rel`, i.e., dropping all `Fun` in case of error. However, this would contradict the lattice property: `max` τ (weak τ) = weak τ, since we would have `max` $\tau\ \tau' = \tau'$, but $\tau \notin \tau'$. Moreover, since we compare input signatures using $\subseteq$, this alternative definition of weak (resp. strong) would not make the checker accept more programs.

5.4.2 The idea

The static analyzer accumulates knowledge on variables in a context Γ initially only containing the signature of predicates. The checking algorithm considers one rule at a time, following their priorities. It begins by updating Γ using the preconditions on the *inputs* of the head, then it scans all body atoms and finally checks the postconditions: the determinacy of the body and the determinacy of the *outputs* of the head.

For example, consider the predicates `comp` and `compose` below: `comp` is a function that takes as input two functions `F` and `G`, and calls them in sequence, whereas `compose` builds a new function from the two received in input. At first, the checker assumes that the inputs of `comp` satisfy the preconditions given in its signature. In particular, `F` and `G` are marked as functions in Γ. Then the algorithm scans the atoms in the body. The determinacy of the whole rule is computed as the `max` of the determinacies of its premises. Since both `F X Y` and `G Y Z` have determinacy `Fun`, their `max` is also `Fun`. Finally, the analyzer verifies that the determinacy of the body is consistent with the one declared in the signature.

For `compose`, two conditions must be checked: the term `comp F G` must be a function, and `compose` itself must also be a function. The algorithm assumes again that `F` and `G` are functions. The term `comp F G` is a partial application, and it returns a function when its arguments are functions. Since this holds for `F` and `G`, the preconditions of `comp` are satisfied, and the result is a function. Finally, the body of `compose` is empty, so it is deterministic. Therefore, `compose` behaves deterministically.

We now move to an example that involes a miscall to a predicate, that is, a rule wih a non-deterministic body atom.

```
func r (func int -> int), (pred int -> int), int -> int.
r F G X Y :- map G [X] [Z], !, compose F F H, H Z Y.
```

After analyzing the head of the rule, the context asserts that `F` (resp. `G`) is a function (resp. a relation). In the body of `r`, the first atom is non-deterministic, since `G` violates the preconditions of `map`. The signature of `map G [X] [Z]` is hence weakened to `Rel`. Even if this atom could generate multiple alternatives, the `Cut` commits to the first one restoring the determinism of the body of `r` up to that point. The call to `compose` satisfies its preconditions, so its postconditions apply: the entire atom `compose F F H` is considered deterministic *and* `H` is marked as a function in Γ. The final atom `H Z Y` is also deterministic because of the knowledge about `H` accumulated so far in Γ.

$$\frac{\Gamma \left|\overset{\mathsf{i}}{\mathsf{hd}}\right. c = \mathcal{D}_1,\ \Gamma_1 \quad \forall i \in 1 \ldots n \text{ do } \mathcal{P},\ \Gamma_i,\ \mathcal{D}_i \left|\overset{}{\mathsf{a}}\right. b_i = \mathcal{D}_{i+1},\ \Gamma_{i+1} \quad \Gamma_{n+1} \left|\overset{\mathsf{o}}{\mathsf{hd}}\right. c : \mathcal{D}_{n+1}}{\mathcal{P},\ \Gamma \left|\overset{}{\mathsf{r}}\right. c\ \texttt{:-}\ b_1 \ldots b_n}\ (\mathsf{r})$$

(a) Check rule

$$\frac{x \text{ fresh in } \Gamma \qquad \mathcal{P},\ \Gamma + \{x \mapsto \star\} \left|\overset{}{\mathsf{r}}\right. r}{\dfrac{\text{mut\_excl } \mathcal{P}\ \Gamma\ x\ r \qquad (r :: \mathcal{P}),\ \Gamma + \{x \mapsto \star\},\ \mathcal{D} \left|\overset{}{\mathsf{a}}\right. a = \mathcal{D}',\ \Gamma'}{\mathcal{P},\ \Gamma,\ \mathcal{D} \left|\overset{}{\mathsf{a}}\right. \texttt{pi}\ x.\ r \Rightarrow a = \mathcal{D}',\ \Gamma'}\ (\mathsf{a}_\forall)}$$

$$\frac{}{\mathcal{P},\ \Gamma,\ \_ \left|\overset{}{\mathsf{a}}\right. \texttt{Cut} = \texttt{Fun},\ \Gamma}\ (\mathsf{a}_!) \qquad \frac{\Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. c = \texttt{Rel},\ \bot}{\mathcal{P},\ \Gamma,\ \_ \left|\overset{}{\mathsf{a}}\right. c = \texttt{Rel},\ \Gamma}\ (\mathsf{a}_\bot)$$

$$\frac{\Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. c = \mathcal{D}',\ \top \qquad \Gamma \left|\overset{\mathsf{o}}{\mathsf{ca}}\right. c : \mathcal{D}' = \Gamma'}{\mathcal{P},\ \Gamma,\ \mathcal{D} \left|\overset{}{\mathsf{a}}\right. c = \texttt{max}\ \mathcal{D}\ \mathcal{D}',\ \Gamma'}\ (\mathsf{a}_c)$$

(b) Rules for **check atom**: its type is $\mathbb{P},\ \mathbb{\Gamma},\ \mathbb{D} \left|\overset{}{\mathsf{a}}\right. \mathbb{A} = \mathbb{D},\ \mathbb{\Gamma}$

$$\frac{\Gamma\ s = \tau}{\Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. s = \tau,\ \top}\ (\mathsf{tm}^{\mathsf{i}}_k) \qquad \frac{\Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. f = \star \xrightarrow{\mathsf{i}} \tau,\ b}{\Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. f\ d = \tau,\ b}\ (\mathsf{tm}^{\mathsf{i}}_\star) \qquad \frac{\Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. f = \_ \xrightarrow{\mathsf{o}} \tau,\ b}{\Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. f\ t = \tau,\ b}\ (\mathsf{tm}^{\mathsf{i}}_o)$$

$$\frac{\Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. f = \rho \xrightarrow{\mathsf{i}} \tau,\ \top \qquad \Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. t = \rho',\ \top \qquad \rho' \subseteq \rho}{\Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. f\ t = \tau,\ \top}\ (\mathsf{tm}^{\mathsf{i}}_\top)$$

$$\frac{\Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. f = \rho \xrightarrow{\mathsf{i}} \tau,\ b_1 \qquad \Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. t = \rho',\ b_2 \qquad b_1 = \bot \vee b_2 = \bot \vee \rho' \not\subseteq \rho}{\Gamma \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. f\ t = \text{weak } \tau,\ \bot}\ (\mathsf{tm}^{\mathsf{i}}_\bot)$$

(c) Rules for **check input**: its type is $\mathbb{\Gamma} \left|\overset{\mathsf{i}}{\mathsf{tm}}\right. \mathbb{tm} = \mathbb{ty},\ \mathbb{B}$

$$\frac{\tau' = \text{if } X \in \Gamma \text{ then } \Gamma\ X \text{ else } \tau}{\Gamma \left|\overset{\mathsf{o}}{\mathsf{tm}}\right. X : \tau = \Gamma + \{X \mapsto \texttt{min}\ \tau\ \tau'\}}\ (\mathsf{tm}^{\mathsf{o}}_X) \qquad \frac{\Gamma\ k = \tau' \quad \tau' \subseteq \tau}{\Gamma \left|\overset{\mathsf{o}}{\mathsf{tm}}\right. k : \tau = \Gamma}\ (\mathsf{tm}^{\mathsf{o}}_k)$$

$$\frac{\Gamma \left|\overset{\mathsf{o}}{\mathsf{tm}}\right. f : \star \xrightarrow{\mathsf{i}} \tau = \Gamma'}{\Gamma \left|\overset{\mathsf{o}}{\mathsf{tm}}\right. f\ d : \tau = \Gamma'}\ (\mathsf{tm}^{\mathsf{o}}_\star) \qquad \frac{\Gamma \left|\overset{\mathsf{o}}{\mathsf{tm}}\right. f : \_ \xrightarrow{\mathsf{o}} \tau = \Gamma'}{\Gamma \left|\overset{\mathsf{o}}{\mathsf{tm}}\right. f\ t : \tau = \Gamma'}\ (\mathsf{tm}^{\mathsf{o}}_o)$$

$$\frac{\Gamma \left|\overset{\mathsf{o}}{\mathsf{tm}}\right. f : \rho \xrightarrow{\mathsf{i}} \tau = \Gamma' \qquad \Gamma' \left|\overset{\mathsf{o}}{\mathsf{tm}}\right. t : \rho = \Gamma''}{\Gamma \left|\overset{\mathsf{o}}{\mathsf{tm}}\right. f\ t : \tau = \Gamma''}\ (\mathsf{tm}^{\mathsf{o}}_i)$$

(d) Rules for **assume output**: its type is $\mathbb{\Gamma} \left|\overset{\mathsf{o}}{\mathsf{tm}}\right. \mathbb{tm} : \mathbb{ty} = \mathbb{\Gamma}$

Figure 5.2: Determinacy checker algorithm part 1

$$\frac{\Gamma\ k = \tau}{\Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. k = \tau,\ \Gamma}\ (\mathsf{hd}^{\ \mathsf{i}}_{k}) \qquad \frac{\Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. f = \_ \xrightarrow{\circ} \tau,\ \Gamma' \quad \vee \quad \Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. f = \star \xrightarrow{\mathsf{i}} \tau,\ \Gamma'}{\Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. f\ t = \tau,\ \Gamma'}\ (\mathsf{hd}^{\ \mathsf{i}}_{o})$$

$$\frac{\Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. f = \rho \xrightarrow{\mathsf{i}} \tau,\ \Gamma' \qquad \Gamma' \left|\frac{\mathsf{o}}{\mathsf{tm}}\right. t : \rho = \Gamma''}{\Gamma \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. f\ t = \tau,\ \Gamma''}\ (\mathsf{hd}^{\ \mathsf{i}}_{i})$$

(a) Rules for **assume head input**: its type is $\mathbb{\Gamma} \left|\frac{\mathsf{i}}{\mathsf{hd}}\right. \mathbb{tm} = \mathbb{ty},\ \mathbb{\Gamma}$

$$\frac{\Gamma\ k = \tau' \qquad \tau \subseteq \tau'}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. k : \tau}\ (\mathsf{hd}^{\ \circ}_{k}) \qquad \frac{\Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. f : \_ \xrightarrow{\mathsf{i}} \tau \quad \vee \quad \Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. f : \star \xrightarrow{\circ} \tau}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. f\ t : \tau}\ (\mathsf{hd}^{\circ}_{i})$$

$$\frac{\Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. f : \rho \xrightarrow{\circ} \tau \qquad \Gamma \left|\frac{\mathsf{i}}{\mathsf{tm}}\right. t = \rho',\ \top \qquad \rho' \subseteq \rho}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. f\ t : \tau}\ (\mathsf{hd}^{\circ}_{o})$$

(b) Rules for **check head output**: its type is $\mathbb{\Gamma} \left|\frac{\mathsf{o}}{\mathsf{hd}}\right. \mathbb{tm} : \mathbb{ty}$

$$\frac{\Gamma\ s = \tau}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{ca}}\right. s : \tau = \Gamma}\ (\mathsf{a}^{\circ}_{k}) \qquad \frac{\Gamma \left|\frac{\mathsf{o}}{\mathsf{ca}}\right. f : \rho \xrightarrow{\circ} \tau = \Gamma' \qquad \Gamma' \left|\frac{\mathsf{o}}{\mathsf{tm}}\right. t : \rho = \Gamma''}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{ca}}\right. f\ t : \tau = \Gamma''}\ (\mathsf{a}^{\circ}_{o})$$

$$\frac{\Gamma \left|\frac{\mathsf{o}}{\mathsf{ca}}\right. f : \_ \xrightarrow{\mathsf{i}} \tau = \Gamma}{\Gamma \left|\frac{\mathsf{o}}{\mathsf{ca}}\right. f\ t : \tau = \Gamma}\ (\mathsf{a}^{\circ}_{i})$$

(c) Rules for **assume term output**: its type is $\mathbb{\Gamma} \left|\frac{\mathsf{o}}{\mathsf{ca}}\right. \mathbb{tm} : \mathbb{ty} = \mathbb{\Gamma}$

Figure 5.3: Determinacy checker algorithm part 2

5.4.3 The algorithm

We present the static analyzer using derivation rules, they are listed in figures 5.2 and 5.3. Each procedure has an associated entails-like symbol, e.g., context $\vdash^{\text{flag}}_{\text{name}}$ intake = result. We use the equal sign to "mode" the rule, i.e., separate what is given to the rule from what is generated by it. The static analyzer is composed of seven procedures for a total of 24 rules.

The entry point is `check:-` (figure 5.2a). This procedure takes a program and a context of signatures, it verifies if a rule validates the signature ascribed to its predicate. The procedure stores in Γ all the knowledge available on variables and verifies that the precondition made by the signature entails the postcondition.

In particular, the **assume head input** procedure (figure 5.3a) loads into Γ_1 the information assumed from the head inputs. This procedure is applied under the assumption that the predicate is well-called. Hence, all variables that occur (recursively) in input positions are assumed to satisfy the determinacy specified in the signature.

On the other hand, the **check head output** procedure (figure 5.3b) verifies that Γ_{n+1} entails the required postcondition. More precisely, the inferred type of the terms appearing in output positions must be smaller or equal (with respect to $\subseteq$) than the type declared in the signature. In addition, the determinacy of the body must also by smaller or equal than the expected one. For instance, if the predicate is declared deterministic, then the determinacy flag inferred for the body must be `Fun`.

Each of the n body atoms b_i is processed by **check atom** (figure 5.2b). This procedure updates the context, step by step, and computes the determinacy of the whole body, producing $\mathcal{D}_{n+1}$.

The **check atom** procedure includes specific rules to handle `Cut` ($a_!$ in figure 5.2b) and `pi` ($a_\forall$ in figure 5.2b), as well as rules for predicate calls (a_c) and miscalls ($a_\bot$).

The `Cut` operator resets the current determinacy to `Fun`. This follows from its semantics: once a `Cut` is executed, all alternative choice points are discarded, so no nondeterminism can arise.

The `pi` operator introduces a fresh symbol together with a local rule. This rule is checked to ensure that determinacy is preserved. In particular, it must satisfy `checkc` in the current context and must not violate mutual exclusion. Then, the underlying atom is analysed in the program extended with this local rule.

Rule a_c checks that the preconditions of the predicate c are satisfied. This is ensured by **check input**, which verifies that the inputs meet the expected conditions. If this holds, the post-condition is assumed, producing an updated context Γ'. The determinacy is updated as the maximum between the current one and that declared for the predicate.

If the preconditions are not satisfied, the context remains unchanged and the determinacy is set to `Rel`. This value reflects that, in case of a miscalled predicate, no deterministic behaviour can be guaranteed.

The most relevant procedure is the one used to check whether the preconditions of a predicate call are satisfied, namely **check input** (figure 5.2c). This procedure ignores data arguments and output arguments, and simply retrieves from the context the signature of constants and variables (tm^i_k in figure 5.2c).

Rule $tm^i_\top$ corresponds to the standard typing rule for application. When the input arguments match the expected signature, it computes the resulting type and reports no miscall (i.e. $\top$).

Otherwise, the head or the argument contain a miscall, or when the argument does not match the expected signature, rule $tm^i_\bot$ is applied. In this case, a miscall is reported and the resulting

signature is obtained by weakening the signature inferred from the head. Intuitively, the predicate call is treated as a relation, and the types of the output arguments are weakened accordingly.

The procedure to update the context when an atom is found to be a good call is **assume term output** (figure 5.3c). It is an iterator over output arguments: other than ignoring input arguments (a°_i) and fetching the signature of symbols in the context (a°_k), it calls **assume output** on output arguments.

The **assume output** procedure (figure 5.2d) updates the context when it encounters variables (rule tm^o_X). If a variable already has a signature in the context, the stronger one is kept.

Data and output arguments are ignored, while input arguments are traversed recursively. In this way, the signatures of variables occurring in input positions are also updated in the context.

The processing of the head of a rule is similar to that of body atoms, but symmetric with respect to how arguments are treated. The **assume head input** procedure (figure 5.3a), called at the beginning of figure 5.2a, ignores output arguments and applies **assume output** to the inputs.

The **check head output** procedure (figure 5.3b) is invoked at the end. It ignores input arguments and checks that the output ones satisfy the declared signature in the current context. This context results from assuming the rule preconditions together with the postconditions of all well-called body atoms.

5.5 Determinism guarantees

In order to express *operational determinacy* we need to use the operational semantics for *Elpi*. The semantics is given in section 2.2.2.

The properties of `runE` relevant to the current presentation are:

1. `Cut` behaves as per section 2.2.3
2. `pi` $x.\ r \Rightarrow a$ generates a fresh constant for x available in r and a
3. `pi` $x.\ r \Rightarrow a$ locally loads r in a giving it the highest priority
4. input arguments are matched against the query as per definition 2.2.4

Definition 5.5.1 (Operationally deterministic execution (deterministic$_{\mathsf{a}}$ $\mathbb{P}$ $\mathbb{A}$))**.** We say that a query atom b is operationally deterministic in a program $\mathcal{P}$ iff

$$\forall \sigma, \texttt{runE}\ ((\sigma,\ (\mathcal{P}, b) :: \varnothing) :: \varnothing) \rightsquigarrow (\_, a)\ \rightarrow\ a = \varnothing$$

In other words a deterministic query leaves no alternatives when it succeeds. We now link the static check to the deterministic execution of a query.

Definition 5.5.2 (Checked program (checked$_{\mathsf{p}}$ $\mathbb{\Gamma}$ $\mathbb{P}$))**.** A program $\mathcal{P}$ is checked against a context of signatures Γ iff it validates these rules:

$$\frac{}{\text{checked}_{\mathsf{p}}\ \Gamma\ \varnothing} \qquad \frac{\text{checked}_{\mathsf{p}}\ \Gamma\ \mathcal{P} \qquad \text{mut\_excl}\ \mathcal{P}\ r \qquad \mathcal{P},\ \Gamma \vdash_{\mathsf{r}} r}{\text{checked}_{\mathsf{p}}\ \Gamma\ (\text{r} :: \mathcal{P})}$$

The intuition is that we build the program from the end, always adding new rules with highest priority.

Software	LOC	% `Fun`	`Cut` added	`Cut` needed (bugs)
Elpi	1448	97%	1	1 (100%)
Rocq-Elpi	13164	79%	91	78 (85%)
Hierarchy Builder	5326	94%	33	30 (90%)
Algebra Tactics	1799	–	1	1 (100%)
Trocq	2918	–	4	4 (100%)
BRiCk (public)	2500	–	11	11 (100%)
BRiCk (private)	7500	–	6	6 (100%)
Total	34655	85%	147	131 (89%)

Table 5.1: Experience report

Definition 5.5.3 (Checked query atom (checked$_{\mathsf{a}}$ $\mathbb{\Gamma}$ $\mathbb{A}$))**.** An atom b is checked against a context of signatures Γ iff

$$\varnothing,\ \Gamma,\ \texttt{Fun} \vdash_{\mathsf{a}} b = \texttt{Fun},\ \_$$

Theorem 5.5.1 (Static analysis)**.** *Given program $\mathcal{P}$, a context of signatures Γ and a query atom b, checked$_{\mathsf{p}}$ Γ $\mathcal{P}$ $\wedge$ checked$_{\mathsf{a}}$ Γ $b \Rightarrow$ deterministic$_{\mathsf{a}}$ $\mathcal{P}$ b.*

The proof of theorem 5.5.1 has been mechanized in the *Rocq* proof assistant, for the first-order part and is available at `https://github.com/FissoreD/elpi-formalization`. We discuss of the first-order formalization in deterail at chapter 6. The mechanization for the higher-order part is ongoing.

5.6 Experience report

The static analyzer we describe was integrated in the *Elpi* language version 3.0. It consists of about 200 lines of *OCaml* code for the implementation of mut_excl and about 800 lines of code for the implementation of `check`$_{:-}$, along with all the related auxiliary functions. We have ported some *Rocq* libraries written using *Elpi*: *Rocq-Elpi*, *Hierarchy Builder*, *Algebra Tactics*, *Trocq*, and *BRiCk*.

We can divide the porting of these libraries into two categories: libraries in which we have traversed all the predicates and carefully changed **`pred`** markers to **`func`** (*Elpi*, *Rocq-Elpi*, and *Hierarchy Builder*), and libraries that have been ported to just pass the static analyzer. In particular, the latter class of libraries measures the minimal cost for adapting code to the changes we made to the *Elpi* and *Rocq-Elpi* standard libraries. The typical example of code requiring a fix is code that loads a local rule containing no cut, typically `copy`.

We summarize these results in table 5.1. In the *Elpi* standard library, we annotated 272 predicates out of 281 as functions, with 37 occurrences of higher-order arguments, as in `map`, `filter`, `fold`, etc. We identified 1 error, i.e., a missing cut. In *Rocq-Elpi*, we have 928 functions out of 1181, with 62 higher-order predicates. We added 91 cuts to please the static analyzer, 78 of which were clearly bugs in the code. The remaining 13 cuts are due to mut_excl, which is not powerful enough to discriminate among rules. The algorithm could be improved to better distinguish mutual exclusion by extending definition 5.3.1, as in [34], but the problem is, in general, undecidable. An example of this issue is given below:

Authors	System	Mode analysis	Dynamic	H.O.	Miscalls
[12]	SB-*Prolog*	required	✗	✗	✗
[34]	Ciao	required	✗	✗	✗
[33]	Red Alert	required	✗	✗	✗
[51]	Mercury	required	✗	✓‡	✗†
[27]	Curry	no need	✗	✓	✓
Fissore and Tassi	*Elpi*	no need	✓	✓	✓

Table 5.2: Comparison with related work

```
func fact int -> int.
fact 0 1.
fact N R :- N > 0, N' is N - 1, fact N' R', R is N * R'.
```

The two rules for `fact` are in mutual exclusion: the input cannot be *0* and *greater than 0* at the same time. But a complete algorithm identifying all of these scenarios does not exists, and we require an explicit `Cut` in the first rule.

In *Hierarchy Builder*, we have 460 functions out of 488, with 33 higher-order arguments, and we have corrected 30 bugs in the code.

Overall, we can conclude that 85% of the predicates were functions.

In *Algebra Tactics*, we added 1 cut over 1799 lines of codes; in *Trocq*, we added 4 cuts over 2918 lines of code. None of these libraries were using `copy` as a relation, i.e. to generate all possible copies of a term. The *BRiCk* application has some public code we ported ourselves and some closed source code ported by the BlueRock company, that we thank for sharing these numebrs. In the public code we added 11 cuts over 2500 lines of code, while in the private one they added 11 cuts over 7500 lines.

5.7 Related work and concluding remarks

We summarize several related works in table 5.2. One key difference between our work and [12, 34, 51, 33, 27] is that they focus on *closed* predicates, that do not evolve during execution, while λ-*Prolog*, hence *Elpi*, allows rules to be dynamically loaded. For closed predicates, programs can be transformed into *super-homogeneous* form by merging all rules into one with disjunctions. This simplifies determinacy inference and may allow to compile the code to a faster binary. However, this transformation is not feasible in our dynamic setting (section 5.1.2).

Regarding modes, *Elpi*'s matching procedure over input arguments allows us to dispense with explicit mode analysis – an essential component in [12, 34, 51, 33].

We differ from Red Alert [33] in that we pick an operational semantics that explicitly represents choice points as a list of alternatives, and that hence is very close to the actual behavior of *Elpi*'s runtime. They pick a denotational semantics over the super-homogeneous form that in turn lets them formulate the static analyzer as an abstract interpreter. It is unclear if a denotational semantics can be written without transforming the program in super-homogeneous form.

Some of the higher-order features of the *Elpi* language are available in Mercury [51], although in a more constrained way (‡). For example `once` from section 5.1.1 is considerably more useful if the input relation is allowed to be non-ground, e.g, `once (map get_pdiv [6] L)`. We

allow higher-order predicates to be miscalled, while Mercury does not. Instead, it requires higher-order functions to be annotated with explicit mode and determinacy signatures for each possible behavior. As a result, Mercury ascribes 11 signatures to the `foldr` predicate and any call that does not match exactly one of these signatures results in a compilation failure (†). *Elpi* targets less technically inclined users, more familiar with functional programming languages than logic ones, hence we strive for simplicity, defining only one determinacy class and immediately classifying wrong calls as non-deterministic. As a result the signature for `foldr` is just `func foldr (func L, A -> A), list L, A -> A`, familiar to our audience.

The "determinism types" of Curry [27] are much more similar to our signatures, and Curry supports miscalled functions as well. In the work of Hanus and Prott the `Any` type stands for non-determinism and like a black hole it absorbs the determinism of the surrounding sub-terms. This is a convenient way to capture all miscalls and treat them uniformly. In our presentation, we have no equivalent; instead, we use weak to change the signature of a term when a miscall is detected (such as, $tm^i_\perp$ in 5.2c or atom calls $a_\perp$ in 5.2b). The advantage is that weak only changes the leaves ($\mathbb{D}$) of the signature while leaving its structure intact. Although we do not analyze data in this paper, our set of rules can be complemented to perform full type checking (in addition to determinacy checking) in one pass.

In Curry the `allValues` builtin can be used to turn a relation into a function. *Prolog*, and also *Elpi*, provide a similar `findall` builtin, but in addition to that they also let one commit to the first result by either using `once`, or by placing a `Cut` after the non-deterministic call. It is this "after the fact" commitment that make the analysis a bit more complex. In fact, in the body of a deterministic predicate we can have calls to relations (or miscalled functions) and still consider the rules valid with respect to determinacy (see $a_!$ in 5.2b). The notion of *functional context* in [12] is closely related to this difficulty.

Our experience report confirms most of the *Elpi* code out there is made of functional predicates and that the static analyzer accepts the vast majority of it requiring no changes. The main improvement we will consider is about *mode subtyping* for higher-order arguments. In the definition of $\subseteq$ we currently accept only arrows that have the same input/output mode. However, it seems possible to relax this condition. For example we could allow passing to `map` a function of type $\star \overset{\circ}{\to} \star \overset{\circ}{\to} \texttt{Fun}$, since the modes of higher-order arguments seem to play no role in the determinacy of code that receives and calls them.

CHAPTER 6

A Mechanized First-Order Static Determinacy Checker

In the previous chapter we have explained the implementation of the determinacy checker algorithm that we have implementated in *Elpi*, starting from version 3. The checker helps users control backtracking: the user asserts which predicates should be considered deterministic and then the checker verifies that these predicates behave like functions, i.e., they commit to their first solution, leaving no *choice points*.

This chapter reports our first steps towards a mechanization, in the *Rocq* prover, of the determinacy analysis from [17]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for *Prolog* with cut. The first is a stack-based semantics that closely models a sublanguage of *Elpi*: we exclude from the semantics the interpretation for the `pi` and `=>` operators. This implementation is similar to the semantics mechanized by Pusch in *Isabelle* [46] and to the model of Debray and Mishra [11, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations used by standard *Prolog* abstract machines [62, 2], but it makes reasoning about the scope of the cut operator difficult. To address this limitation we introduce a tree-based semantics in which the branches pruned by `Cut` are explicit and we prove the two semantics equivalent. Using the tree-based semantics, we then prove some properties about the impact of evaluating the cut in disjunctive queries. For example, unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut-away by q_1.

Finally, we use the formal semantics to prove the correctness of a static determinacy analysis chapter 5. Intuitively, nondeterminism arises when a computation can be completed by applying two different rules. We exclude this situation in two ways. Firstly, at most one rule should be used on a given query. This condition is satisfied if (i) the rules have non-overlapping heads: if two heads do not overlap in the inputs they accept, then no query can unify with both, or if (ii) we account for the presence of cut in rule premises: once a rule whose premises contain a cut succeeds, all remaining rules are discarded. Secondly, each rule of a deterministic predicate should (i) either call functions, this ensure that no choice point is left after the execution of the premises of the chosen rule, (ii) or call relations provided that they are followed by the cut operator, so that any allocated choice points preceding the cut are discarded.

We show that if every rule defining a deterministic predicate passes this analysis, then any call to that predicate leaves no choice points.

```
Inductive Tm := Symb of S | Pred of P | Var of V | App of Tm & Tm.
Inductive Atom := cut | call of Tm.
Record R := mkR { head : Tm; premises : list Atom }.
Record ℙ := { rules : seq R; sig : sigT }.
Definition Σ := {fmap V -> Tm}.
```

Figure 6.1: Definition of term, rule, program and substitution

6.1 Preliminaries: syntax and informal semantics of cut

In this chapter we use figure 2.4 as our running example, and we start by describing how we represent a program like that one.

The description of a program is given by the record $\mathbb{P}$ (in figure 6.1) and is shared by both semantics. A program consists of a list of rules `R` together with a signature environment `sig`. We delay the discussion of signatures to section 6.5 where we describe the static analysis that concerns them. The order in which rules appear in the list represent their priorities: rules coming first have higher priority.

A rule consists of a head term `Tm` and a list of premises. Each premise is an `Atom`, i.e. either the cut operator or a predicate call. Terms `Tm` include predicate symbols, data symbols, variables, and term application. The syntax tree allows variables to be applied and predicates to be mixed with data. These higher-order features play no role in the current chapter that focuses on first-order logic programming, but allows future extensions to full *λ-Prolog* to be based on the same mechanization.

The set of variables `V`, predicates `P` and data symbols `S` are countable. We represent substitutions Σ as finitely supported maps from variables to terms, using the finmap library [37]. The empty substitution is written ε, while the composition of two substitutions σ_1 and σ_2 with disjoint supports as $\sigma_1 + \sigma_2$.

The backchaining operation applies all rules to a query term; it is central to both semantics. Its implementation is very similar to the function $\mathcal{B}$ from section 2.2.2.

```
Definition backchain : ℙ → ℱν → Tm → Σ → ℱν * list (Σ * list Atom)
```

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the set of variable names used so far ($\mathcal{F}_\nu$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

In our mechanization, we abstract over the unifier and its properties. The function `unify` takes two terms t_1 and t_2, together with a substitution σ, as input, and optionally returns a substitution σ' that extends σ. The expected behavior of `unify` is the standard one (see for example [13, section 2.5]), limited to the first order. Therefore, if t_1 and t_2 unify under σ, then $\sigma'\ t_1 = \sigma'\ t_2$, where $\sigma\ t$ denotes substitution application and $=$ denotes syntactic equality.

6.2 And-Or Tree semantics

```
Inductive tree :=
  | KO | OK                                  (* failure and success *)
  | Todo of Atom                             (* unexplored branch *)
  | Or   of option tree & Σ & tree           (* A ∨ B_σ *)
  | And  of tree & list Atom & tree.         (* A ∧_{B_0} B *)
```

Figure 6.2: Definition of And-Or tree

An And-Or tree is a stratified, variable-width tree, defined in *Rocq* as in figure 6.2. It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, and all subqueries in the corresponding And-layer.

The tree is built incrementally. The `Todo` node represents an unexplored branch (an `Atom`). When the atom is a `call`, we use the backchaining procedure from section 6.1 to compute the two layers to attach to the tree: alternatives form the Or-layer, and the premises of each applicable rule form the corresponding And-layer. This expansion step is performed by the `step` procedure described in section 6.2.2.

In addition to the `Todo` node we have two distinguished nodes, `KO` and `OK`, which represent respectively the smallest failed and successful tree.

The `Or` node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A. The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ, which becomes the current substitution when backtracking to B.

The `And` node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to the first choice point in A. We call B_0 the *reset point* for B: it represents the continuation for all alternatives of A except the first. That is, when backtracking occurs within A, the subtree B is replaced by the atoms in B_0. An example is shown in section 6.2.3.

A graphical representation of a tree is shown in figure 6.4b. For compactness, we depict `And` and `Or` nodes as n-ary, with children associating to the right. The `KO` node acts as the terminating element of an `Or` list, while `OK` is the terminating element of an `And` list.

6.2.1 The semantics

The tree is constructed incrementally by two recursive functions, `step` and `prune`. Both traverse the tree depth-first, left-to-right. The node to be explored in a tree is retrieved thanks to the `next_tree` (in figure 6.3). When this node is `OK` we say the entire tree is a *success*, when it is `KO` we say the tree *failed*, otherwise the tree is *incomplete*. In figure 6.4, we provide some graphic representations of trees and mark with a black arrow the result of `next_tree`.

```
Inductive tag := Expanded | CutBrothers | Failed | Success.
Definition step : ℙ → 𝓕_ν → Σ → tree → (𝓕_ν * tag * tree)
Definition prune : 𝔹 → tree → option tree
Inductive runT : ℙ →  𝓕_ν → Σ → tree →
                          option (Σ * option tree) → 𝔹 → 𝓕_ν → Prop
```

```
Fixpoint next σ t : Σ * tree :=
  match t with
  | Todo _ | KO | OK ⇒ (σ, t)
  | Or □ σ' B ⇒ next σ' B
  | Or ·A _ _ ⇒ next σ A
  | And A _ B ⇒
    let (σ', pA) := next σ A in
    if pA == OK then next σ' B
    else (σ', pA)
  end.

Definition next_subst σ t := (next σ t).1.
Definition next_tree t := (next ε t).2.

Definition success t := next_tree t == OK.
Definition failed t := next_tree t == KO.
Definition incomplete t :=
    if next_tree t is Todo _ then ⊤ else ⊥.
```

Figure 6.3: `next` and derived functions

Since all functions must terminate in *Rocq*, we define the transitive closure of `step` and `prune` as the inductive relation `runT`. More precisely `runT` iterates calls to `step` on incomplete trees to expand `Todo` nodes. When a tree fails it calls `prune` to find the next alternative and continues from there, if one exists.

6.2.2 The `step` procedure

The `step` procedure takes as input a program, a set of variables, a substitution, and a tree, and returns an updated set of variables, a `tag`, and an updated tree.

The set of variables is assumed to contain all variables occurring in the tree. It is passed to backchain so that rules can be refreshed correctly.

The `tag` indicates which kind of step was performed. `Failure` and `Success` are returned for trees that satisfy, respectively, the `failed` and `success` tests. `CutBrothers` indicates the interpretation of a superficial cut: `step` was called on an And-layer and its `next_tree` is the cut atom. Finally, `Expanded` accounts for the interpretation of predicate calls and non-superficial cuts.

The `step` procedure evaluates atoms. In particular, it leaves the tree unchanged when the tree is *success* or *failed*.

Lemma 6.2.1 (succ_step_iff). `∀ p v σ t, success t ↔ step p v σ t = (v, Success, t).`

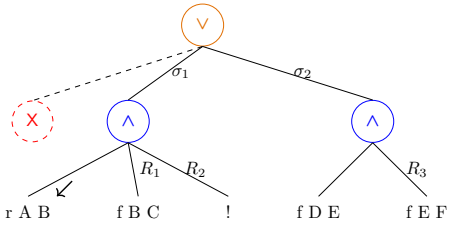
Lemma 6.2.2 (fail_step_iff). `∀ p v σ t, failed t ↔ step p v σ t = (v, Failed, t).`

The `step` procedure expands `Todo` nodes by calling `backchain`, which returns a list l of alternatives. If l is empty, then the current call atom is replaced by `KO`. Otherwise, it is replaced by a right-skewed tree of `Or` nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf is `OK`; otherwise, it is a right-skewed tree of `And` nodes.

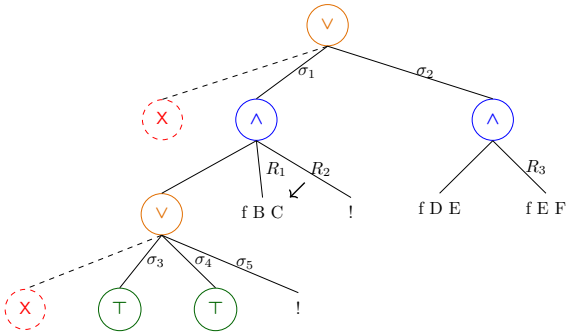
Figure 6.4 depicts the tree corresponding to the running example (section 2.2.3) as it undergoes calls to `step` and `prune`. We run query `q 0 R` against the program P in figure 2.4. Let c_0, c_1, and c_2 be the children of the root. By construction, c_0 is the $\Box$ tree, while c_1 and c_2 correspond respectively to the premises of rules r_1 and r_2 in P. Note that c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in section 2.2.3. Reset points are defines as follows: $R_1 :=$ `[f Y Z, !]`, $R_2 :=$ `[!]`, and $R_3 :=$ `f Y Z`. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of `step` performing a tree expansion via backchain appear in figures 6.4c to 6.4e.

g 0 R

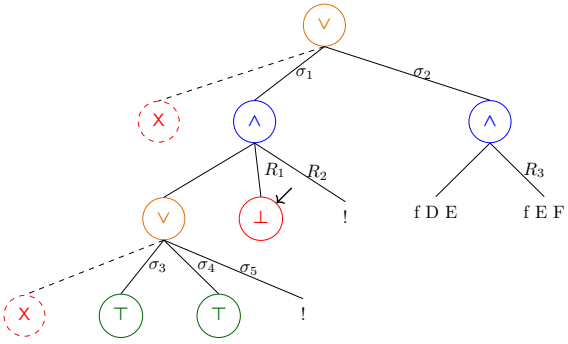
(a) Initial query



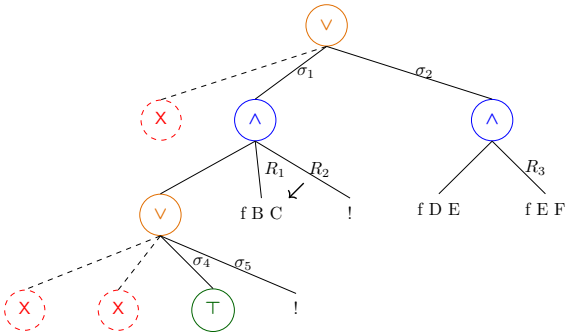
(b) Tree after `step` on `g 0 R`



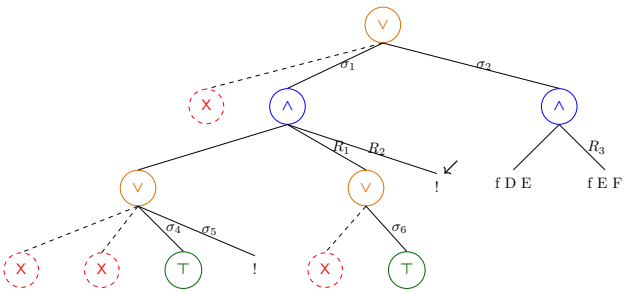
(c) `step` on figure 6.4b



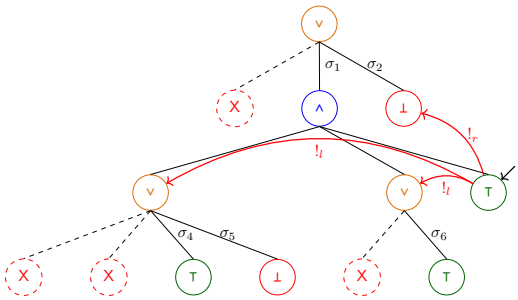
(d.1) `step` on figure 6.4c



(d.2) `prune` on figure 6.4d.1



(e) `step` on figure 6.4d.2



(f) `step` on figure 6.4e

Figure 6.4: Execution in the tree semantics. The substitutions $\sigma_1 \dots \sigma_6$ are from section 2.2.3. R_1, R_2, and R_3 are reset points and correspond respectively to the lists of atoms [`f Y Z, !`], [`!`], and [`f Y Z`].

The interpretation of a cut The step corresponding to a cut performs two actions: (i) the cut node is replaced by `OK`; and (ii) the subtrees in the scope of `Cut` are pruned. More precisely, (ii) prunes the left siblings of the `Cut` node (in the same And-layer, denoted $!_l$) and prunes the right uncles of `Cut` (in the one-level-up Or-layer, denoted $!_r$).

An example of the impact of cut is shown in figure 6.4f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. Note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by `KO`. This amounts to discarding the third rule of predicate `r`.

6.2.3 The `prune` procedure

When `backchain` returns an empty list, `step` produces a tree that *failed* and the computation must backtrack. The `prune` procedure is responsible for producing the new tree from which the computation can continue.

In figure 6.4d.1 we encounter a failure because no rule applies to the atom `f B C` under substitution σ_3, which maps `B` to `0`. The `prune` procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the `OK` node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the `KO` node with the information stored in the reset point R_1. This restores the call `f B C`, which can then be reconsidered under substitution σ_4.

Furthermore, `prune` serves another important purpose. Its behavior depends on the boolean flag it receives as input: `prune` $\bot$ t recursively removes all failures (if any) from t, whereas `prune` $\top$ t removes the first success in t. For example, the tree in figure 6.4f contains a successful path that maps `R` to `2`. Calling `prune` on this tree returns $\square$, since there are no alternatives in the tree after the success.

Some interesting properties of `prune` are shown below; they illustrate how `prune` complements `step` with respect to failed, successful, and incomplete trees.

Lemma 6.2.3 (incomplete_prune_id). `∀ b t, incomplete t → prune b t = ·t.`

Lemma 6.2.4 (prune_Some). `∀ b t t', prune b t = ·t' → failed t' = ⊥.`

Lemma 6.2.5 (prune_None). `∀ t, prune ⊥ t = □ → failed t.`

6.2.4 The `runT` relation and its properties

The inductive relation `runT` relates four inputs to three outputs. The inputs are a program, a set of variables, an initial substitution σ, and a tree t. The outputs are an optional result r, a boolean b, and an updated set of variable names.

The main result r is $\square$ when the execution fails; otherwise when $r = \cdot(\sigma', t')$ the execution succeeded and produced the solution σ'; t' represents the remaining, not-yet-explored alternatives. The boolean b records whether a superficial cut was evaluated during execution.

The `runT` predicate has four cases, depicted as inference rules in figure 6.5. ($\texttt{runT}_\top$) If `next_tree` t is a success, then the substitution σ' is extracted from t (starting from σ) and the input tree is cleaned of its successful path. ($\texttt{runT}_\mathcal{B}$) If `next_tree` t is an atom, then `step` is invoked to evaluate t, and `runT` is called recursively on the updated tree. ($\texttt{runT}_\circlearrowleft$) If `next_tree` t is a failure, then `prune` is called to clear the failed path; if the resulting cleaned tree exists, `runT` is called recursively on it. Finally, ($\texttt{runT}_\bot$) accounts for trees with no solutions.

The set of rules defining `runT` is deterministic.

$$\frac{\texttt{success}\ t \qquad \texttt{next\_subst}\ \sigma\ t = \sigma' \qquad \texttt{prune}\ \top\ t = t'}{\texttt{runT}\ \mathcal{P}\ v\ \sigma\ t\ \cdot(\sigma', t')\ \bot\ v}\ \texttt{runT}_\top$$

$$\frac{\begin{array}{cc}\texttt{incomplete}\ t & b' = (\text{tg} = \texttt{CutBrothers}) \vee b \\ \texttt{step}\ \mathcal{P}\ v\ \sigma\ t = (v', \text{tg}, t') & \texttt{runT}\ \mathcal{P}\ v'\ \sigma\ t'\ r\ b\ v''\end{array}}{\texttt{runT}\ \mathcal{P}\ v\ \sigma\ t\ r\ b'\ v''}\ \texttt{runT}_{\mathcal{B}}$$

$$\frac{\texttt{failed}\ t \qquad \texttt{prune}\ \bot\ t = \cdot t' \qquad \texttt{runT}\ \mathcal{P}\ v\ \sigma\ t'\ r\ \text{n}\ v'}{\texttt{runT}\ \mathcal{P}\ v\ \sigma\ t\ r\ \text{n}\ v'}\ \texttt{runT}_{\circlearrowleft}$$

$$\frac{\texttt{prune}\ \bot\ t = \Box}{\texttt{runT}\ \mathcal{P}\ v\ \sigma\ t\ \Box\ \bot\ v}\ \texttt{runT}_\bot$$

Figure 6.5: Rule system for runT

Lemma 6.2.6 (runT_det). $\forall\ p\ v_0\ \sigma\ t\ r\ r'\ b\ b'\ v\ v',\ runT\ p\ v_0\ \sigma\ t\ r\ b\ v \to$

$runT\ p\ v_0\ \sigma\ t\ r'\ b'\ v' \to r' = r \wedge v' = v \wedge b' = b.$

The proof is by induction on the first runT derivation and is immediate, because the rules are selected based on next_tree t. In particular, the hypotheses success t, incomplete t, and failed t are mutually exclusive. Moreover, by lemma 6.2.5, the hypothesis of $\texttt{runT}_\bot$ implies that t *failed*; hence $\texttt{runT}_\bot$ is exclusive with $\texttt{runT}_\top$ and $\texttt{runT}_{\mathcal{B}}$, and it is also exclusive with $\texttt{runT}_{\circlearrowleft}$ since they impose different requirements on the output of prune.

The boolean output of runT allows us to state interesting properties about the Or node in the presence of cut. Here we report only a selection of those we proved.

Lemma 6.2.7 (runSST_or). $\forall\ p\ v\ v'\ \sigma\ \sigma'\ A\ A'\ \sigma_1\ B,\ runT\ p\ v\ \sigma\ A\ \cdot(\sigma',\ \cdot A')\ \top\ v' \to$

$runT\ p\ v\ \sigma\ (\cdot A \vee B_{\sigma_1})\ \cdot(\sigma',\ \cdot(\cdot A' \vee KO_{\sigma_1}))\ \bot\ v'.$

Lemma 6.2.8 (runNT_or). $\forall\ p\ v\ v'\ \sigma\ A\ \sigma_1\ B,\ runT\ p\ v\ \sigma\ A\ \Box\ \top\ v' \to$

$runT\ p\ v\ \sigma\ (\cdot A \vee B_{\sigma_1})\ \Box\ \bot\ v'.$

Lemma 6.2.9 (runNF_or). $\forall\ p\ v_0\ v_1\ v_2\ \sigma\ A\ \sigma_1\ \sigma_2\ B\ B'\ b,$

$runT\ p\ v_0\ \sigma\ A\ \Box\ \bot\ v_1 \to runT\ p\ v_1\ \sigma_1\ B\ \cdot(\sigma_2,\ B')\ b\ v_2 \to$
$\quad \textbf{let}\ sR := omap\ (\textbf{fun}\ x \Rightarrow \Box \vee x_{\sigma_1})\ B'\ \textbf{in}$
$\quad runT\ p\ v_0\ \sigma\ (\cdot A \vee B_{\sigma_1})\ \cdot(\sigma_2, sR)\ \bot\ v_2.$

Lemmas 6.2.7 to 6.2.9 correspond to the introduction rules for disjunction. If A executes a cut, one *cannot* obtain $A \vee B$ from B: the execution of A, whether it ultimately succeeds or fails, makes the success of B irrelevant (the cut discards it). In the first lemma, B is forced to be KO; in the second one, the failure of A absorbs B. The last lemma is connected to the depth-first exploration of the tree: proving a disjunction from its right-hand side can only occur after the left-hand side has been disproved. Depicted as rules:

$$\frac{A \quad (A \text{ cuts})}{A \vee B}\ (\vee_{il}) \qquad \frac{\neg A \quad (A \text{ cuts})}{\neg(A \vee B)}\ (\vee_{ir1}) \qquad \frac{\neg A \qquad B \quad (A \text{ does not cut})}{A \vee B}\ (\vee_{ir2})$$

Lemmas 6.2.10 and 6.2.11 are elimination rules for disjunction.

Lemma 6.2.10 (run_orSST). $\forall\ p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ A\ A'\ B\ B',$

$runT\ p\ v\ \sigma\ (\cdot A \vee B_{\sigma_1})\ \cdot(\sigma',\ \cdot(\cdot A' \vee B'_{\sigma_1}))\ \bot\ v' \to$
$\quad \exists\ b,\ runT\ p\ v\ \sigma\ A\ \cdot(\sigma',\ \cdot A')\ b\ v' \wedge B' = \textbf{if}\ b\ \textbf{then}\ KO\ \textbf{else}\ B.$

Lemma 6.2.11 (run_orSNT). $\forall\ p\ v\ v'\ \sigma\ \sigma'\ \sigma_1\ A\ B\ B',$

$$\mathcal{B}_S(\mathcal{P}, v, t, \sigma, a, g) = \left[\sigma', ([(\mathcal{P}, q, a) \mid q \in b] +\!\!+ g) \;\middle|\; (\sigma', b) \in \mathcal{B}(\mathcal{P}, v, t, \sigma)\right].$$

$$\frac{}{\texttt{runS}\ \mathcal{P}\ v\ ((\sigma, \varnothing) :: a)\ {\cdot}(\sigma, a)}\ \text{StopS} \qquad \frac{\texttt{runS}\ \mathcal{P}\ v\ ((\sigma, g) :: ca)\ \mathrm{r}}{\texttt{runS}\ \mathcal{P}\ v\ ((\sigma, (\text{cut}, ca) :: g) :: \_)\ \mathrm{r}}\ \text{CutS}$$

$$\frac{\mathcal{B}_S\ (\mathcal{P}, v, t, \sigma, a, g) = (v_1, a') \qquad \texttt{runS}\ \mathcal{P}\ v_1\ (a' +\!\!+ a)\ \mathrm{r}}{\texttt{runS}\ \mathcal{P}\ v\ ((\sigma, (\text{call}\ t, \_) :: g) :: a)\ \mathrm{r}}\ \text{CallS} \qquad \frac{}{\texttt{runS}\ \mathcal{P}\ \_\ \varnothing\ \square}\ \text{FailS}$$

Figure 6.6: Rule system for `runS`

```
runT p v σ (·A ∨ B_σ1) ·(σ', ·(□ ∨ B'_σ1)) ⊥ v' →
  (∃ b, runT p v σ A ·(σ', □) b v' ∧ if b then B' = KO else prune ⊥ B = ·B') ∨
  (∃ v2 b, runT p v σ A □ ⊥ v2 ∧ runT p v2 σ1 B ·(σ', ·B') b v').
```

From $A \vee B$, when A is not inert (i.e. not already explored), we cannot deduce A or B independently. Instead, we can only derive a weakened conclusion of the form $\top \vee B'$, where B' provides no information in the case where the exploration of A performs a cut. This yields an elimination rule that is stronger than the usual one (depicted on the left). If A does not cut, the elimination rule has the usual two premises, together with the additional constraint that whenever B holds, A must have failed. This again reflects the depth-first traversal of the proof tree.

$$\frac{A \vee B \qquad \overset{[A]}{C} \qquad (A \text{ cuts})}{C}\ (\vee_{e1}) \qquad \frac{A \vee B \qquad \overset{[A]}{C} \qquad \overset{[\neg A, B]}{C} \quad (A \text{ does not cut})}{\mathrm{C}}\ (\vee_{e2})$$

It is worth recalling that the "A cuts" side-condition in the rules above is precisely the boolean result returned by `runT`. Without this information, it is impossible to formulate these rules: it is not merely a matter of whether A contains a syntactic occurrence of cut, but whether that cut is actually executed during the (dis)proof of A (in rules $(\vee_{ir1})$ and $(\vee_{ir2})$). Indeed, an atom preceding the cut may fail, so the cut is never reached.

As anticipated in the introduction, these lemmas contradict the commutativity of disjunction.

6.3 Stack-based semantics

The stack-based semantics we present is very close to what is implemented by the *Elpi* interpreter (see section 2.2.2). Since we encode a first-order interpreter, we have no `pi` and `=>` operators, therefore the corresponding rules ($\texttt{runE}_\forall$ and $\texttt{runE}_\Rightarrow$) are fired. This semantics is similar to the one proposed by Pusch in [46], which is in turn closely related to the semantics given by Debray and Mishra in [11, Section 4.3]. Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract Machine [62, 2].

The execution state is a stack of alternatives. Each alternative is a list of goals. Intuitively it amounts to a disjunctive normal form: each stack item is a self contained computation, coupling a substitution with all the goals to be solved.

Goals pair an atom with a list of the *cut-to* alternatives, making the two datatypes mutually recursive, but these alternatives have a very different role. They have to be understood as pointers to lower levels of the stack itself. This datum is used to implement the cut, i.e. to prune the stack of alternatives.

```
Inductive alts  := nilA | consA of (Σ * goals) & alts
with       goals := nilG | consG of (Atom * alts)  & goals
```

The stack-based semantics is described by the `runS` inductive predicates.

```
Inductive runS (p: ℙ) : F_ν → alts → option (Σ * alts) → Prop
```

The `runS` predicates relates a set of variables and a list of alternatives to optional result. $\square$ stands for failure, i.e., no solution, whereas ${\cdot}(\sigma, a)$ represent success with σ begin the solution and a as the list of non-yet-explored alternatives.

The rule system for `runS` is given in figure 6.6 and is made by 4 cases. We distinguish two main cases. If we run out of alternatives we fail: (FailS) stops the execution and return $\square$. If the list of alternatives is $(\sigma_0, h) :: a$, then we look at the list of goals h (under the substitution σ_0). If h is empty, (StopS) stops the execution with a success σ_0 leaving a as the unexplored alternatives. If h is not empty we look at its head goal (t, ca) where t is the atom and ca is its cut-to alternatives. If t is a cut rule (CutS) continues to evaluate the tail of h under with alternatives ca. Finally, when t is a call, rule (CallS) backchains: for each rule it pairs each premise with the current stack of alternatives, and prepends the obtained goals to the tail of h. Note that if backchain returns the empty list, (CallS) second premise amounts to exploring the next item in the current stack.

```
Fixpoint valid_tree A :=
  match A with
  | Todo _ | OK | KO ⇒ T
  | Or □ _ B ⇒ valid_tree B
  | Or ·A _ B ⇒ valid_tree A &&
       ((B == KO) || B.base_or B)
  | And A B0 B ⇒ valid_tree A &&
      if success A then valid_tree B...
      else B == big_and B0
  end.
```

```
Fixpoint t2l t σ (cp: alts) : alts :=
match t with
| OK      ⇒ [(σ, ∅)]
| KO      ⇒ ∅
| TA a    ⇒ [(σ, [(a, ∅)])]
```

Figure 6.7: Valid tree and Tree to stack

6.3.1 Inadequacy of the stack-based semantics for formal reasoning

The lemmas that we presented in section 6.2.4 are hard to express in the stack-based semantics, due to the lack of structure in the stack to delimit the scope of the cut operator. For example, consider a stack $a_0, a_1, \ldots, a_n$. If a_0 succeeds but executes a cut, it is hard to relate the remaining alternatives $a_1, \ldots, a_n$ to the alternatives $b_1, \ldots, b_m$ that survive the cut, since the only simple invariant is that $b_1, \ldots, b_m$ is a suffix of $a_1, \ldots, a_n$. If $a_1, \ldots, a_n$ were generated before the call whose execution generates the cut in a_0, then they all stay; but if some were generated afterwards, they are discarded. Expressing this precisely would require attaching, at the very least, time-stamps (or depths) to goals, which amounts to a cumbersome encoding of a tree.

6.4 Equivalence of the two semantics

In order to relate the two semantics we have to relate the computations states, i.e., the And-Or tree and the stack of alternatives. We define a translation from trees to stacks, called `t2l`, but we do not explicitly provide the converse translation, an economy allowed by the proof technique we employ, inspired by [4].

Before describing the translation of trees to stacks (in section 6.4.2) we need to define the notion of *valid tree*. The `tree` datatype indeed contains trees that cannot be generated by `runT` via `step` or `prune`, for example all the trees that do not correspond to a depth-first left-to-right tree construction strategy.

6.4.1 Valid trees

The class of valid trees is captured by the `valid_tree` function shown in Figure 6.7. While all base cases of `tree` are valid, we need to analyze the `Or` and `And` constructors more carefully.

For the `Or` constructor, we distinguish two cases depending on whether the left-hand side is present. If it is not, then the right-hand side must be a valid tree. Otherwise, the left-hand side must be a valid tree, and the right-hand side must either be the `KO` tree or be unexplored. The former case occurs when the right-hand side is cut away by the evaluation of a superficial cut in the left-hand side. In the latter case we use the `base_or` predicate to check that `B` is the right-skewed tree formed by a disjunction of conjunctions, generated after a successful backchain for a `call`.

For the `And` constructor, the left hand side is required to be a valid tree. The shape of the right hand side depends on whether the left hand side is a success. If it is not successful, then the right hand side must not be explored, i.e. `B` must be the right-skewed tree containing conjunctions of premise atoms. This is enforced using the `big_and` procedure, which generates a tree from the reset point B_0 (the same procedure used to transform each alternative output by `backchain` into the And-layer of the resulting tree). When the left hand side is successful, then the right hand side may have been explored, and consequently must be a valid tree.

6.4.2 From trees to stacks

The `t2l` recursive function translates a tree to a stack. For space constraints figure 6.7 only gives the base cases, while we describe the recursive cases in the following text.

The function `t2l` takes as input the tree t_0, a substitution σ_0, and a list of choice points *cp*. These choice points represent alternatives that appear at the same level of the current tree, such as, for example, the right siblings of an `Or` node. The substitution is used to determine under which substitution the subtree being computed lives.

The `OK` node represents one successful alternative; therefore it is translated into a singleton list whose only element has an empty list of goals. The `KO` node represents failure, hence it is mapped to the empty list of alternatives. Finally, atoms are mapped to a singleton list containing one alternative whose only goal is that atom, with an empty cut-to list. At this stage, the cut-to list cannot point to *cp*: atoms are not right-uncle subtrees, whereas *cp* represents alternatives that will actually be discarded if the atom turns out to be a cut. The correct cut-to list is therefore determined later, once the trees corresponding to sibling subtrees have been inspected.

The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B. The alternatives of B are valid choice points for A and should be passed to the translation of A. The two lists of alternatives can then be concatenated. Moreover, every atom occurring one level below *cp* (i.e. in A or B) should add *cp* as its cut-to alternatives.

The translation of $A \wedge_{B_0} B$ starts with the translation of A. If the translation of A is an empty list we return the empty list of alternatives without even touching B nor B_0, since an empty translation for A indicates failure and this in turn implies the failure of the entire conjunction. When the translation of A is a list $(\sigma_0, g) :: a$, the B node represents the continuation of the first alternative g, whereas B_0 is the continuation for each alternative in a.

Example of `t2l` execution The translations of the trees in figure 6.4 are shown in figure 2.5. The letters in the figures relate each tree to the corresponding stack. For example, 6.4b is translated into 2.5b. We also note that 6.4d.1 and 6.4d.2 are both translated into 2.5d, showing that *t2l* is not injective: different trees can map to the same stack. Consequently, there are transitions in `runT` that are not visible in `runS`. As an example of a call to `t2l`, we take the tree and the stack in figure 6.8, which are respectively taken from figures 2.5c and 6.4c. The red arrows point from the head of an alternative in the tree to the head of the corresponding cell in the stack. We focus on the deepest `Or` node in the tree. This subtree is a disjunction with four children. The first is ignored, since it is $\square$. The success node below σ_3 has `f B C` and the cut operator as continuation, corresponding to the first cell in the stack. We note that the cut operator points outside the stack, since the scope of this cut eliminates its right uncle. The success node below σ_4 has R_1 as continuation, where R_1 is the list of goals `f B C, !`; this is translated into the second cell of the stack. Finally, the cut operator under σ_5 also has R_1 as continuation. It corresponds to the third alternative. We can see that the first cut points to the translation of the subtree under σ_2, since it is one `Or`-layer above its right uncles.

6.4.3 Equivalence proof

The tree-based semantics exposes more information than needed, hence we hide it behind existentials.

Definition 6.4.1 (runT' `p v σ t r`). `∃ b v', runT p v σ t r b v'.`

From figures 2.5 and 6.4, we observe that, upon backtracking, `runT` may perform more steps than `runS` before returning a solution or a failure. These silent transitions must be skipped when proving the equivalence between the two semantics. Therefore, we prove the following lemma.

Lemma 6.4.1 (pruneF_t2l). `∀ t t' b, valid_tree t → failed t → prune b t = ·t' →`

`∀ l σ, t2l t σ l = t2l t' σ l.`

Proof.

By induction on the tree t.

□

The first direction of the equivalence states that the execution of a valid tree is matched by the execution of the stack obtained by translating the tree. Moreover, the unexplored alternatives returned as a tree correspond to those returned as a stack.

Theorem 6.4.2 (tree_to_elpi). `∀ p t σ r, let v := vars_tree t ∪ vars_sigma σ in`

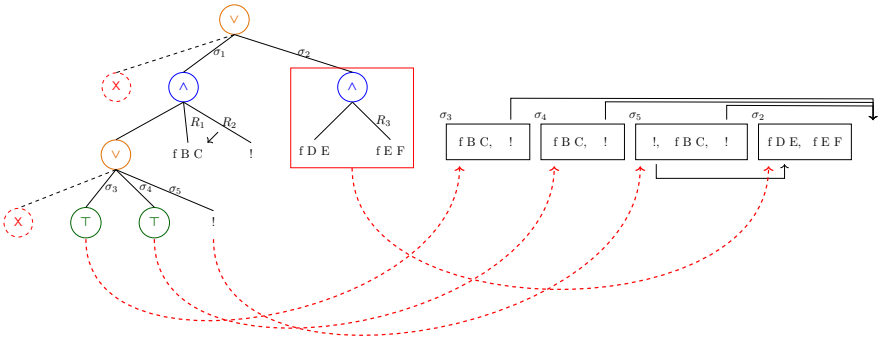


Figure 6.8: Example of `t2l` execution

```
valid_tree t →
  runT' p v σ t r →
    let a := t2l t σ ∅  in
    let r' :=  if r is ·(σ', t) then  ·(σ', t2l (odflt KO t) σ ∅ )
               else □ in
    runS p v a r'.
```

Proof.

We proceed by induction on the execution of `runT'`. There are four cases to consider. The first case arises from $\texttt{runT}_\top$; it deals with successful trees. A successful tree must start with an empty list, and therefore we conclude using StopS. The case for $\texttt{runT}_\mathcal{B}$ requires treating two subcases: `step` evaluates either a cut or a call. Accordingly, we apply CutS or CallS, followed by the induction hypothesis. The case for $\texttt{runT}_\circlearrowleft$ is silent: it represents the non-injective behavior of `t2l`; however, thanks to lemma 6.4.1 and the induction hypothesis, the conclusion is proved. The last case arises from the derivation $\texttt{runT}_\bot$. It is easily proved using FailS and the fact that if `prune` returns □ when trying to erase failure in t, then `t2l` applied to t returns the empty list.

□

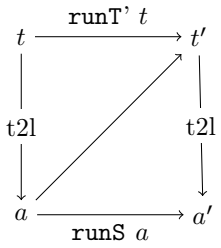
The converse direction is stated following [4]: instead of using a function to translate a stack into a tree it states that any tree that translates to the given stack has an equivalent execution, i.e. gives the same solution and returns a unexplored tree that translates to the unexplored stack.

Theorem 6.4.3 (elpi_to_tree). `∀ p v a r, runS p v a r →`

```
∀ σ t, valid_tree t → t2l t σ ∅  = a →
  if r is ·(σ', a') then
    ∃ t', runT' p v σ t ·(σ', t') ∧ t2l (odflt KO t') σ ∅  = a'
  else runT' p v σ t □.
```

Proof.

The proof is based on the idea explained in [4, Section 3.5.1] and depicted in the figure on the right: we have a tree t whose translation is a, we proceed by induction on the execution of `runS`. This gives us not only the output a' but that hypothesis that it exists a tree t' such that its translation to a stack is a'.



□

Stating the converse direction in a symmetric way would require a function `l2t` transforming a stack a into a tree t. Writing this function is far from trivial, since the structure of the tree is lost by `save_as` and `runT` that intuitively keep the state as a big disjunction of conjunctions by distributing conjunctions over disjunctions. Moreover, one would need to define a notion of valid stack that is equally intricate.

The constructive nature of the logic of *Rocq* tells us that the proof above provides an effective function translating stacks into trees. However, that function also uses an execution trace of the stack-based semantics, from which it recovers the missing tree structure.

6.5 Case study: determinacy analysis

An interesting application of the tree semantics is determinacy checking, introduced in version 3 of the *Elpi* language [17]. The checker takes as input the signatures of the predicates declared by the user and verifies that the rules of a deterministic predicate generate at most one solution; we call this particular kind of predicate a function. Thanks to this static verification, the user can better control unexpected backtracking.

A predicate behaves deterministically if two conditions are satisfied: *mutual exclusion* guarantees that at most one rule can be successfully applied to any given query, whereas *rule checking* ensures that each rule does not create choice points, for example by calling non-deterministic predicates.

Determinacy checking is strongly related to the modes of predicates: we differentiate between input and output arguments. A deterministic call relates to its inputs only one output. In the literature, we find several works on determinacy checking, namely [51, 35, 12]. These works need a mode checker that statically ensures that, in a query with ground inputs, each recursive call is performed with ground inputs.

In our mechanization, and in particular in the *Elpi* language, we adopt a special unification routine for input arguments, called `matching` [2, 64]. Similarly to `unify`, `matching` takes the query term t_1 and the pattern t_2 and returns an optional substitution σ' that does not assign any variable in the query.

The function `matching` has the following property:

Axiom 6.5.1 (matching_disj). `∀ σ σ' t1 t2, vars t1 # domf σ → vars t1 # vars t2 →`

`matching t1 t2 σ = ·σ' → ∃ e, domf σ' = domf σ ∪ e ∧ e ⊆ vars t2.`

If the variables (`vars`) of two terms are disjoint (#) and the variables in the support of σ are disjoint from the variables in t_1, then, if matching succeeds, only the variables in t_2 are assigned; that is, $t_1 = \sigma' t_2$, and the variables in t_1 are not assigned in σ'.

The `backchain` function uses `matching` or `unify` on the arguments q depending if the argument is in *input* or *output* mode. We assume two important properties of our unification routines.

Axiom 6.5.2 (unif_trans). `∀ t1 t2 t3 σ, unify t1 t2 σ → unify t2 t3 σ → unify t1 t3 σ.`

Axiom 6.5.3 (match_unif). `∀ t1 t2 σ, matching t1 t2 σ → unify t1 t2 σ.`

We also assume that refreshing variables does not affect unification. In particular, a refreshing renaming f for a term t is a function from variables to variables that: (i) f is defined on all the variables in t, i.e. all variables get renamed; (ii) f is injective, i.e. does not confuse variables; and (iii) f has non-overlapping domain and codomain, i.e. it maps the variables of the term to a set of completely fresh variables.

Below, we define `refresh_for` to test the validity of a refreshing renaming. Axiom 6.5.4 states the property of term unifiability with respect to term renaming. That is, if the renamings of terms t_1 and t_2 are unifiable under two respective valid refreshing renamings, and they have disjoint variables, then for any pair of renamings z and x that are valid renamings for t_1 and t_2, respectively, and are disjoint from each other, unification also succeeds. In other words, this theorem accounts for α-equivalence of terms and the unification of terms with disjoint variables.

Definition 6.5.1 (refresh_for `x t`). `(vars t ⊆ domf x) ∧ injective x ∧ (domf x # codomf x).`

Axiom 6.5.4 (unif_ren). `∀ x y z w t1 t2,`

```
refresh_for w t1 → refresh_for y t2 → refresh_for z t1 → refresh_for x t2 →
codomf w # vars (rename y t2) → codomf z # vars (rename x t2) →
unify (rename w t1) (rename y t2) ε → unify (rename z t1) (rename x t2) ε.
```

In figure 2.4, the rules of the program are equipped with three signatures, one per predicate. Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, their types, and whether the predicate is deterministic. They indicate that `f` and `g` are expected to be functions, as specified by the **`func`** keyword, whereas `r` is a relation, as indicated by the **`pred`** keyword. The **`->`** symbol separates inputs from outputs; therefore, all three predicates are expected to take an `int` as input and return an `int` as output.

The signatures of the program are stored in the `sig` field of the program and are encoded as described in [17]: we distinguish data from predicates, and predicates are classified into deterministic a non-deterministic.

A program execution behaves deterministically if, given a program, a substitution, and an input tree for the `runT` inductive, the execution either fails or, if it succeeds, the resulting tree of alternatives is □.

Definition 6.5.2 (is_det `p σ v t`). `∀ r, runT' p v σ t r → r = □ ∨ ∃ σ, r = ·(σ, □).`

The theorem we want to prove is the following one:

Theorem 6.5.5 (det_check_call). `∀ p σ t v,`

`check_ℙ p → tm_is_det p.(sig) t → is_det p σ v (Todo (call t)).`

Here, `check_ℙ` denotes the function that checks the program with respect to mutual exclusion and rule checking, and `tm_is_det` denotes the function that tests whether the head of the term refers to a rigid constant with a deterministic type.

The proof of this lemma proceeds by induction on the program execution. However, without further work, the induction hypothesis is too weak since it talks about `Todo` trees (i.e. atoms), whereas we need it on any tree.

To address this issue, we introduce the notion of deterministic trees (`det_tree`), show that a tree satisfying this condition enjoys the desired properties, and then prove the following more general theorem:

Lemma 6.5.6 (det_check_tree). `∀ σ v p t, check_ℙ p → det_tree p.(sig) t → is_det p σ v t.`

Our target theorem `det_check_call` is a corollary of `det_check_tree`.

6.6 Related work

We studied a *Prolog* semantics in which input arguments are *matched*, rather than unified, against rule heads. This choice agrees with the *Elpi* interpreter, which is in turn inspired by B-Prolog's "matching-clauses" [2, 64]. The two semantics we presented (and proved equivalent) also apply to standard *Prolog* if one forces all predicate signatures to have only output arguments.

For determinacy analysis, matching input arguments affects the proofs only as follows: axiom 6.5.1 does require the query q to be ground. Adapting our results to standard *Prolog* therefore requires the insertion of two additional hypotheses in theorem 6.5.5. First, one has to require that t has ground inputs. Second, that the program p is well-moded. Well-moded predicates produce ground outputs when given ground inputs; hence, starting from an initial query with ground inputs, well-moded programs behave exactly as if input arguments were matched. Therefore, the proofs in section 6.5 still apply.

The only mechanization of Prolog's semantics we could find is the one by Pusch in *Isabelle* [46] that is based on the model of Debray and Mishra [11, Sec. 4.3]. Their work starts from that semantics and refines it by introducing some optimizations usually found in the Warren abstract machine [62, 2]. They do not use the semantics to prove properties on the execution of programs as we do in our use case, hence they do not face the difficulty described in section 6.3 that pushed us to develop a more explicit semantics (with respect to cut). In a sense, our work is complementary. By combining both works one can show the tree-based semantics is equivalent to an optimized stack-based one.

Another relevant work is by King [33], but we could not find their *Rocq* source code. Their semantics is denotational and cannot easily cover all the features that [17] covers, but it would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by [4], and we thank Y. Bertot for insightful discussions on the topic. In [4], Bertot mechanizes the correctness of a compiler. He relates the semantics of an imperative language to that of its compiled assembly code and proves their equivalence. His approach uses the compiler as the translation function from one language to the other, but does not introduce a decompiler, just as we do not define a function from stacks to trees. To show that the assembly code behaves like the imperative language, he proceeds by induction on the execution of the assembly program. We adopt the same strategy in the proof of theorem 6.4.3.

6.7 Conclusion

This paper describes two operational semantics for Prolog with cut. The first semantics is based on And–Or trees: it is well suited to graphically illustrating the scope of cut, and to proving lemmas about the impact of its evaluation when reasoning about disjunctions. The second semantics is based on a stack of alternatives: it is computationally more efficient, but it loses the structural expressiveness of the tree-based semantics. Thanks to a dedicated function, we translate trees into stacks and prove the two semantics equivalent. This stack-based semantics corresponds to the first-order fragment of *Elpi*. Using the tree-based semantics, we mechanize the first-order part of the determinacy checker that has been available in *Elpi* since version 3.0, and we prove that a call to a deterministic predicate returns at most one solution.

The formal development consists of approximately 2,500 lines of specifications and about 5,000 lines of *ssreflect* proofs. Roughly 30% of the code is dedicated to the tree semantics (in particular, the proofs in section 6.2.4), 15% to the stack semantics, and 35% to the equivalence proof. The remaining 20% is devoted to the determinacy checker. The mechanization took us approximately 8 months.

To fully model the *Elpi* language and mechanize [17] entirely, we would need to account for higher-order variables, i.e. variables representing predicates. This extension is ongoing and largely orthogonal to the present paper, since cut and higher-order features do not interact in subtle ways.

CHAPTER 7

Conclusion and Perspectives

Dire che il discrimination is a piece of code that can be reused for the implementation of [50, 48]

Bibliography

[1] Andreas Abel and Brigitte Pientka. Extensions to miller's pattern unification for dependent types and records. Preprint, https://www.cs.mcgill.ca/~bpientka/papers/unif_miller60.pdf, 2018.

[2] Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.

[3] Gilles Barthe. Implicit coercions in type systems. In Stefano Berardi and Mario Coppo, editors, *Types for Proofs and Programs*, pages 1–15, Berlin, Heidelberg, 1996. Springer Berlin Heidelberg.

[4] Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: https://inria.hal.science/inria-00073199v1.

[5] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development. Coq'Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer Verlag, 2004.

[6] Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.

[7] Arthur Charguéraud. TLC: A library for classical coq. https://github.com/charguer/tlc, 2021. GitHub repository.

[8] Adam Chlipala. *Certified Programming with Dependent Types: A Pragmatic Introduction to the Coq Proof Assistant*. The MIT Press, 2013.

[9] Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages 239–268, Cham, 2024. Springer Nature Switzerland.

[10] Thierry Coquand and Gérard Huet. The calculus of constructions. *Information and Computation*, 76(2):95–120, 1988. URL: https://www.sciencedirect.com/science/article/pii/0890540188900053, doi:10.1016/0890-5401(88)90005-3.

[11] Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J. Log. Program.*, 5(1):61–91, March 1988. doi:10.1016/0743-1066(88)90007-6.

[12] Saumya K. Debray and David S. Warren. Functional computations in logic programs. *ACM Trans. Program. Lang. Syst.*, 11(3):451–481, July 1989. doi:10.1145/65979.65984.

[13] Gilles Dowek. *Higher-order unification and matching*, page 1009–1062. Elsevier Science Publishers B. V., NLD, 2001.

[14] Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast, embeddable, λProlog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages 460–468. Springer, 2015. URL: https://inria.hal.science/hal-01176856v1, doi:10.1007/978-3-662-48899-7_32.

[15] Amy Felty. Encoding the calculus of constructions in a higher-order logic. In M. Vardi, editor, *lics93*, pages 233–244. IEEE, June 1993. doi:10.1109/LICS.1993.287584.

[16] Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024. doi:10.1145/3678232.3678233.

[17] Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.

[18] Thom Frühwirth. Constraint handling rules - what else? In Nick Bassiliades, Georg Gottlob, Fariba Sadri, Adrian Paschke, and Dumitru Roman, editors, *Rule Technologies: Foundations, Tools, and Applications*, pages 13–34, Cham, 2015. Springer International Publishing.

[19] Thom Frühwirth. Theory and practice of constraint handling rules. *The Journal of Logic Programming*, 37(1):95–138, 1998. URL: https://www.sciencedirect.com/science/article/pii/S0743106698100055, doi:10.1016/S0743-1066(98)10005-5.

[20] Andrew Gacek. The abella interactive theorem prover (system description). In *Proceedings of the 4th International Joint Conference on Automated Reasoning*, IJCAR '08, page 154–161, Berlin, Heidelberg, 2008. Springer-Verlag. doi:10.1007/978-3-540-71070-7_13.

[21] Jean H. Gallier. *Logic for Computer Science: Foundations of Automatic Theorem Proving*. Dover Publications, Mineola, NY, 2003.

[22] Georges Gonthier. A computer-checked proof of the Four Color Theorem. Technical report, Inria, March 2023. URL: https://inria.hal.science/hal-04034866.

[23] Georges Gonthier, Andrea Asperti, Jeremy Avigad, Yves Bertot, Cyril Cohen, François Garillot, Stéphane Le Roux, Assia Mahboubi, Russell O'Connor, Sidi Ould Biha, Ioana Pasca, Laurence Rideau, Alexey Solovyev, Enrico Tassi, and Laurent Théry. A machine-checked proof of the odd order theorem. In Sandrine Blazy, Christine Paulin-Mohring, and David Pichardie, editors, *Interactive Theorem Proving - 4th International Conference, ITP 2013, Rennes, France, July 22-26, 2013. Proceedings*, volume 7998 of *Lecture Notes in Computer Science*, pages 163–179. Springer, 2013. URL: https://inria.hal.science/hal-00816699v1, doi:10.1007/978-3-642-39634-2_14.

[24] Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing Machinery, 2023. doi:10.1145/3573105.3575683.

[25] Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory in higher order constraint logic programming. *Mathematical Structures in Computer Science*, 29:1–26, 03 2019. doi:10.1017/S0960129518000427.

[26] Cordelia V. Hall, Kevin Hammond, Simon L. Peyton Jones, and Philip L. Wadler. Type classes in haskell. *ACM Trans. Program. Lang. Syst.*, 18:109–138, March 1996. doi:10.1145/227699.227700.

[27] Michael Hanus and Kai-Oliver Prott. Determinism Types for Functional Logic Programming. In *Proceedings of PPDP*, pages 47–59, 2025.

[28] Fergus Henderson, Zoltan Somogyi, and Thomas Conway. Determinism analysis in the mercury compiler. In *Proceedings of Australian Computer Science Conference*, pages 337–346, 08 1996.

[29] G.P. Huet. A unification algorithm for typed λ-calculus. *Theoretical Computer Science*, 1(1):27–57, 1975. URL: https://www.sciencedirect.com/science/article/pii/0304397575900110, doi:10.1016/0304-3975(75)90011-0.

[30] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018. doi:10.1017/S0956796818000151.

[31] Robbert Krebbers, Jacques-Henri Jourdan, Ralf Jung, et al. std++: An extended standard library for coq. https://gitlab.mpi-sws.org/iris/stdpp, 2023. GitLab repository.

[32] Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27, doi:10.4230/LIPIcs.ITP.2025.27.

[33] Jael Kriener and Andy King. Redalert: Determinacy inference for prolog. *Theory and Practice of Logic Programming*, 11(4-5):537–553, 2011.

[34] P. López-García, F. Bueno, and M. Hermenegildo. Determinacy analysis for logic programs using mode and type information. In Sandro Etalle, editor, *Logic Based Program Synthesis and Transformation*, pages 19–35, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[35] Lunjin Lu and Andy King. Determinacy inference for logic programs. *Lecture Notes in Computer Science*, 3444:108–123, 04 2005. doi:10.1007/978-3-540-31987-0_9.

[36] Assia Mahboubi and Enrico Tassi. *Mathematical Components*. Zenodo, September 2022. doi:10.5281/zenodo.7118596.

[37] math-comp. finmap — finite maps for mathcomp, 2026. URL: https://github.com/math-comp/finmap.

[38] William McCune. Experiments with discrimination-tree indexing and path indexing for term retrieval. *J. Autom. Reason.*, 9(2):147–167, October 1992. doi:10.1007/BF00245458.

[39] Spiro Michaylov and Frank Pfenning. Higher-order logic programming as constraint logic programming. In *Position Papers for the First Workshop on Principles and Practice of Constraint Programming*, pages 221–229, Newport, Rhode Island, April 1993. Brown University.

[40] Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.

[41] Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge University Press, New York, NY, USA, 2012.

[42] Dale A. Miller. Mechanized metatheory revisited. In *Journal of Automated Reasoning*, volume 63, pages 625–665. Springer, 2018.

[43] Gopalan Nadathur. The metalanguage λprolog and its implementation. In Herbert Kuchen and Kazunori Ueda, editors, *Functional and Logic Programming*, pages 1–20, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg.

[44] Gopalan Nadathur and Dale A. Miller. An overview of lambda-prolog. In *Proceedings of the Fifth International Conference and Symposium on Logic Programming*, pages 810–827, Seattle, WA, USA, 1988. MIT Press.

[45] Katsuhiko Nakamura. Control of logic program execution based on the functional relations. In *Proceedings of ICLP*, page 505–512. Springer, 1986.

[46] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL - A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002.

[47] Frank. Pfenning and Conal Elliott. Higher-order abstract syntax. In *Proceedings of PLDI*, page 199–208. Association for Computing Machinery, 1988. doi:10.1145/53990.54010.

[48] Brigitte Pientka. Tabling for higher-order logic programming. In Robert Nieuwenhuis, editor, *Automated Deduction – CADE-20*, pages 54–68, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.

[49] Rocq Development Team. Class. https://rocq-prover.org/doc/v9.1/refman/addendum/type-classes.html#coq:cmd.Class. Rocq Prover Reference Manual, version 9.1. Accessed: 2026-03-11.

[50] Daniel Selsam, Sebastian Ullrich, and Leonardo de Moura. Tabled typeclass resolution. *CoRR*, abs/2001.04301, 2020. URL: https://arxiv.org/abs/2001.04301, arXiv:2001.04301.

[51] Zoltan Somogyi, Fergus Henderson, and Thomas Conway. The execution algorithm of mercury, an efficient purely declarative logic programming language. *The Journal of Logic Programming*, 29(1):17–64, 1996. High-Performance Implementations of Logic Programming Systems. URL: https://www.sciencedirect.com/science/inproceedings/pii/S0743106696000684, doi:10.1016/S0743-1066(96)00068-4.

[52] Matthieu Sozeau and Nicolas Oury. First-class type classes. In *Proceedings of TPHOLs*, volume 5170 of *LNCS*, pages 278–293. Springer, 2008.

[53] Christopher Strachey. Fundamental concepts in programming languages. *Higher Order Symbol. Comput.*, 13(1–2):11–49, April 2000. doi:10.1023/A:1010000313106.

[54] SWI-Prolog Documentation. Predicate `!/0` — cut (built-in predicate), 2026. Accessed via SWI-Prolog PlDoc reference. URL: https://www.swi-prolog.org/pldoc/doc_for?object=!/0.

[55] Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λProlog dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January 2018. URL: https://inria.hal.science/hal-01637063.

[56] Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of *LIPIcs*, pages 29:1–29:18, September 2019. URL: https://inria.hal.science/hal-01897468, doi:10.4230/LIPIcs.CVIT.2016.23.

[57] Enrico Tassi. Elpi: rule-based extension language. https://github.com/gares/HDR/blob/main/hdr.pdf, 2025.

[58] Enrico Tassi and Freek Wiedijk. ? In ?, editor, *?*, 2026.

[59] Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi. Master's thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.

[60] Philip Wadler. Theorems for free! In *Proceedings of FPCA*, FPCA '89, page 347–359. Association for Computing Machinery, 1989. doi:10.1145/99370.99404.

[61] Philip Wadler and Stephen Blott. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '89, page 60–76, New York, NY, USA, 1989. Association for Computing Machinery. doi:10.1145/75277.75283.

[62] David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309, SRI International, Artificial Intelligence Center, Computer Science and Technology Division, Menlo Park, CA, USA, October 1983. URL: https://www.sri.com/wp-content/uploads/2021/12/641.pdf.

[63] Markus Wenzel. Type classes and overloading in higher-order logic. In Elsa L. Gunter and Amy Felty, editors, *Theorem Proving in Higher Order Logics*, pages 307–322, Berlin, Heidelberg, 1997. Springer Berlin Heidelberg.

[64] Neng-fa Zhou. The language features and architecture of b-prolog. *Theory Pract. Log. Program.*, 12(1–2):189–218, January 2012. doi:10.1017/S1471068411000445.

[65] Beta Ziliani and Matthieu Sozeau. A unification algorithm for coq featuring universe polymorphism and overloading. In *Proceedings of the 20th ACM SIGPLAN International Conference on Functional Programming*, ICFP 2015, page 179–191, New York, NY, USA, 2015. Association for Computing Machinery. doi:10.1145/2784731.2784751.

Appendix

APPENDIX A

Simple compiler full implementation

We provide here the complete code for the simple elpi compiler.

```
From elpi Require Import elpi.
Require Import Reals.

Elpi Db tc.db lp:{{
  pred tc term → term.
}}.

Elpi Command Compiler.
Elpi Accumulate Db tc.db.
Elpi Accumulate lp:{{
  shorten std.{rev}.
  shorten coq.{mk-app}.

  % Inst Ty Args Prems Res
  pred comp term, term, list term, list prop → prop.
  comp I {{lp:T → lp:Bo}} Ag P (pi y\ R y) :- !,  % r$\to$
    pi x\ comp I Bo [x|Ag] [tc T x | P] (R x).
  comp I {{∀ x, lp:(Bo x)}} Ag P (pi y\ R y) :- !, % r$\forall$
    pi x\ comp I (Bo x) [x|Ag] P (R x).
  comp I T Ag P (tc T Proof :- [true | Body]) :-   % rB
    mk-app I {rev Ag} Proof,
    rev P Body.

  pred compile gref →.
  compile G :- coq.env.typeof G Ty,
    comp (global G) Ty [] [] R,
    coq.elpi.accumulate _ "tc.db" (clause _ _ R).

  main [str S] :- coq.locate S GR, compile GR.
}}.

Elpi Tactic Solver.
Elpi Accumulate Db tc.db.
Elpi Accumulate lp:{{
solve (goal _ _ Ty _ _ as G) S :- tc Ty P, refine P G S.
}}.
```

```
Class Add T := {plus: T → T → T}.

Instance addNat : Add nat := {plus := Nat.add}.
Instance addR : Add R := {plus := Rplus}.
Instance addProd T1 T2 : Add T1 → Add T2 → Add (T1 * T2) :=
  {plus '(x1,y1) '(x2, y2) := (plus x1 x2, plus y1 y2)}.

Elpi Compiler addNat.
Elpi Compiler addR.
Elpi Compiler addProd.

Goal Add (nat * (nat * nat)).
Proof. elpi Solver. Qed.
```

Tritre francais

Davide FISSORE

Résumé

Les macarons c'est bon par nature, c'est le propre même du macaron que d'être bon. Au vue de l'affluence constante aux divers magasins Ladurée on peut supposer que les macarons qu'on y trouve sont meilleurs qu'ailleurs. Cette thèse vise à donner des idées et pistes sur le pourquoi du comment que les macarons de Ladurée sont si bons.

Mots-clés : Pâtisserie, Macaron, Ladurée.

Abstract

Macarons are so good.

Keywords: Pastry, Macaron, Ladurée.